

开发示例分享

自研枪炮内弹道计算程序集成示例

FastCAE-自研枪炮内弹道计算程序集成示例（可视化定制）

一、概述

1.1 案例介绍

本案例实现了自研枪炮内弹道计算程序的产品化集成，集成的过程中，使用可视化定制功能对前后处理进行个性化定制，数据规范采用了 FastCAE 定义的标准数据接口，对计算程序的输入与输出进行改动以适应标准接口。最终实现了 FastCAE 与计算程序的无缝集成，形成了完整的平台化的枪炮内弹道分析计算软件

本案例名称为枪炮内弹道计算程序集成案例，其中所有资料包含源码和相关文档均可以在本站[“下载”](#)菜单栏目内进行下载。

1.2 集成目标

- 通过 FastCAE2.0 集成框架实现对枪炮内弹道计算程序核心求解器以及几何建模、网格剖分、参数设置可视化的全面集成。
- 通过本案例能够使用户快速了解使用 FastCAE2.0 进行定制插件开发的过程以及特性，为使用定制插件开发软件集成提供指导。

1.3 环境说明

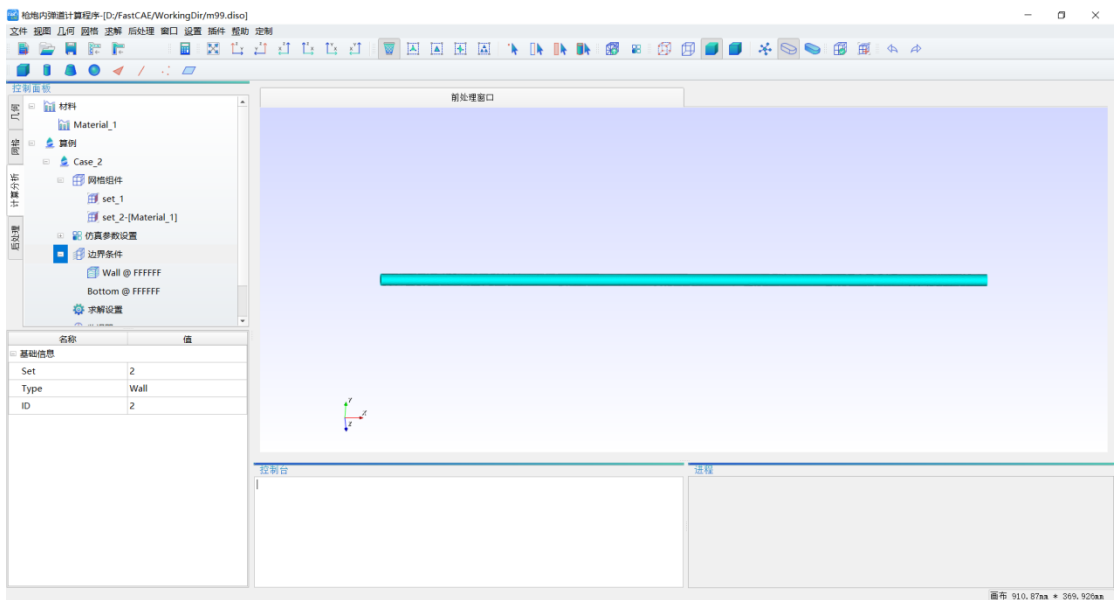
- 开发环境
- Windows 10
- FastCAE 2.0

1.4 注意事项

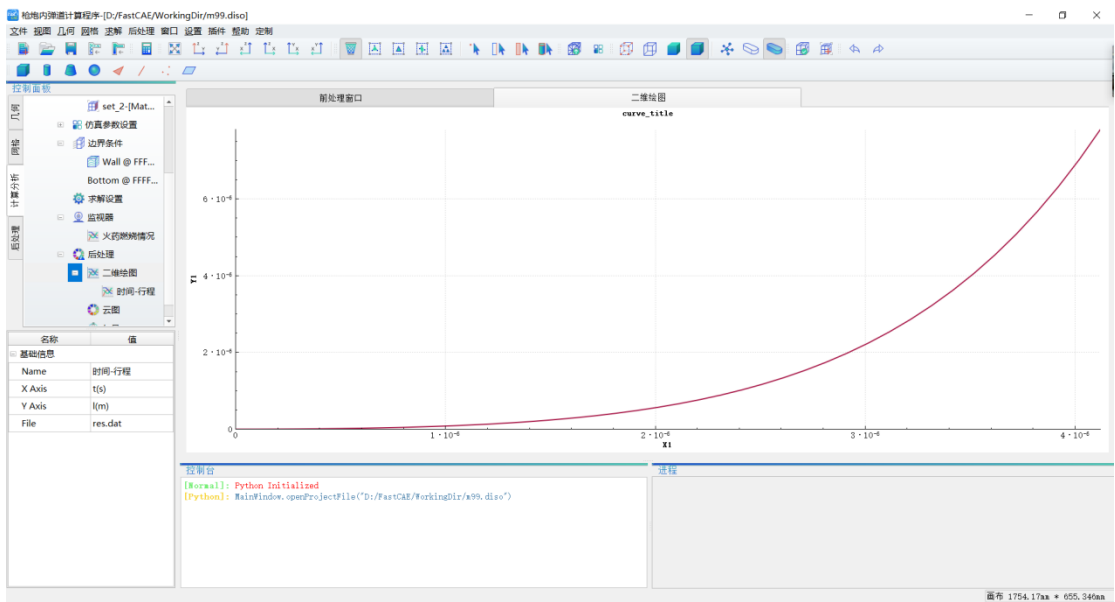
- 建议开展本案例学习之前，对 FastCAE 框架的使用有一定的了解。

二、软件实现效果

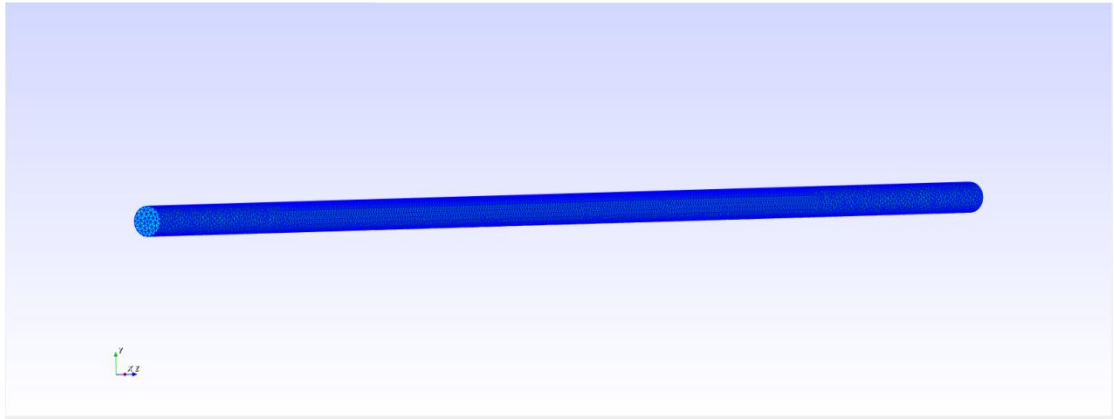
本案例最终形成了枪炮内弹道分析计算软件, 该软件数值计算部分采用显式计算的方法实现弹丸膛内运动的动力学仿真计算。该自研计算程序直接采用 FastCAE 定义的数据规范, 与平台实现无缝对接, 形成了完整的平台化的仿真分析软件。软件界面如下图所示:



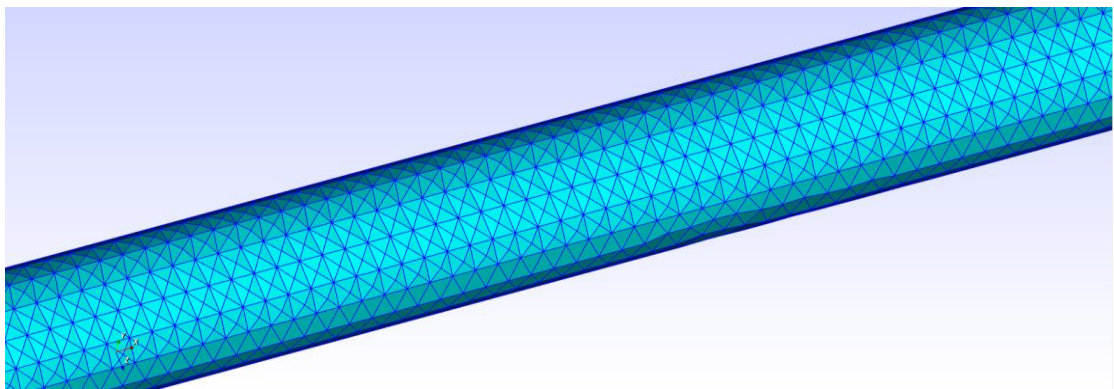
整体效果图



后处理曲线



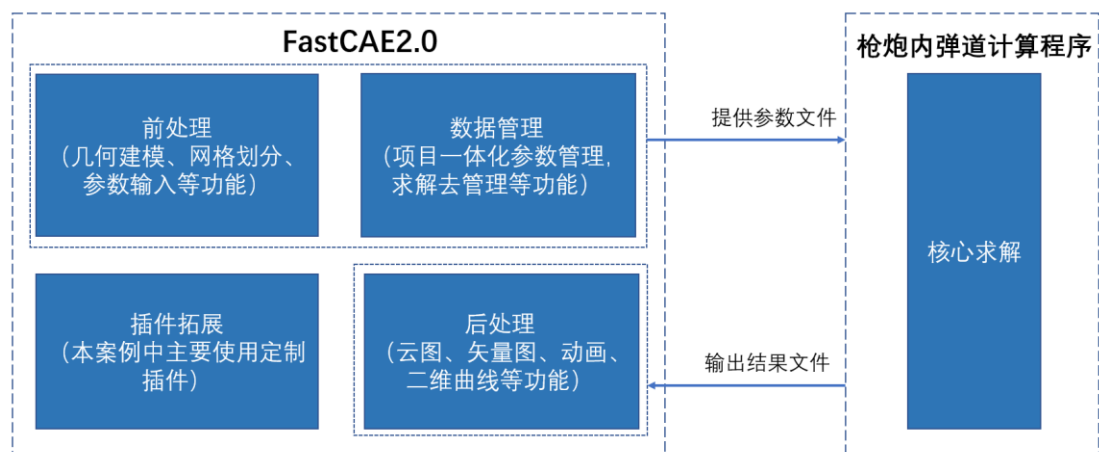
网格剖分结果



网格局部放大效果

三、原理说明

本案例的核心集成原理，通过 FastCAE 平台软件定制界面，生成求解器可读的 xml 格式数据文件，后台驱动求解器进行枪炮内弹道计算并对计算结果进行展示。



3.1 枪炮内弹道计算部分

1.内弹道（internal ballistics）是弹道的一部分，内弹道研究弹丸从点火到离开发射器身管的行为。内弹道学研究对各种身管武器都有重要意义。

2.拉格朗日假设和几何燃烧定律，是研究枪炮膛内弹道参量平均值变化规律的内弹道学的理论基础。它是在 19 世纪到 20 世纪初发展并形成的一种较完整的内弹道学体系。在近一个世纪的实践中，经典内弹道学在武器火力系统的设计、发射装药设计以及弹道预测等方面，一直起着重要的指导作用。

3.本程序通过输入身管形状及其参数、弹丸参数、火药参数、计算控制参数等实现对身管内火药燃气压强、弹丸行程与弹丸速度的计算。程序根据拉格朗日基本假设和几何燃烧定律建立偏微分方程组，采用四阶龙格库塔法进行显式求解。

3.2 FastCAE 部分

根据 3.原理说明-3.1 枪炮内弹道计算部分内容，我们可以分析得出|FastCAE 部分需要实现枪膛建模与网格划分功能以及生成枪炮内弹道计算求解器需要的数据文件，使枪炮内弹道计算求解器在后台运行，并可以将求解生成的后处理文件进行可视化展示。

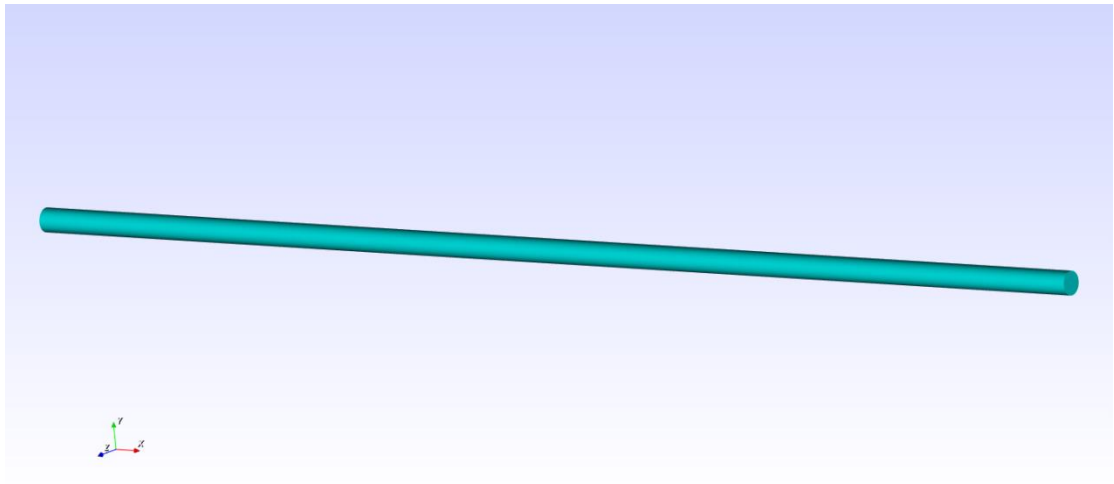
FastCAE 定制化插件部分将提供软件参数的设置、网格的导入及展示、边界条件设置、后处理结果展示功能。这部分功能不需要用户重新开发，只需要进行配置就能够使用。

四、基于 FastCAE 定制软件

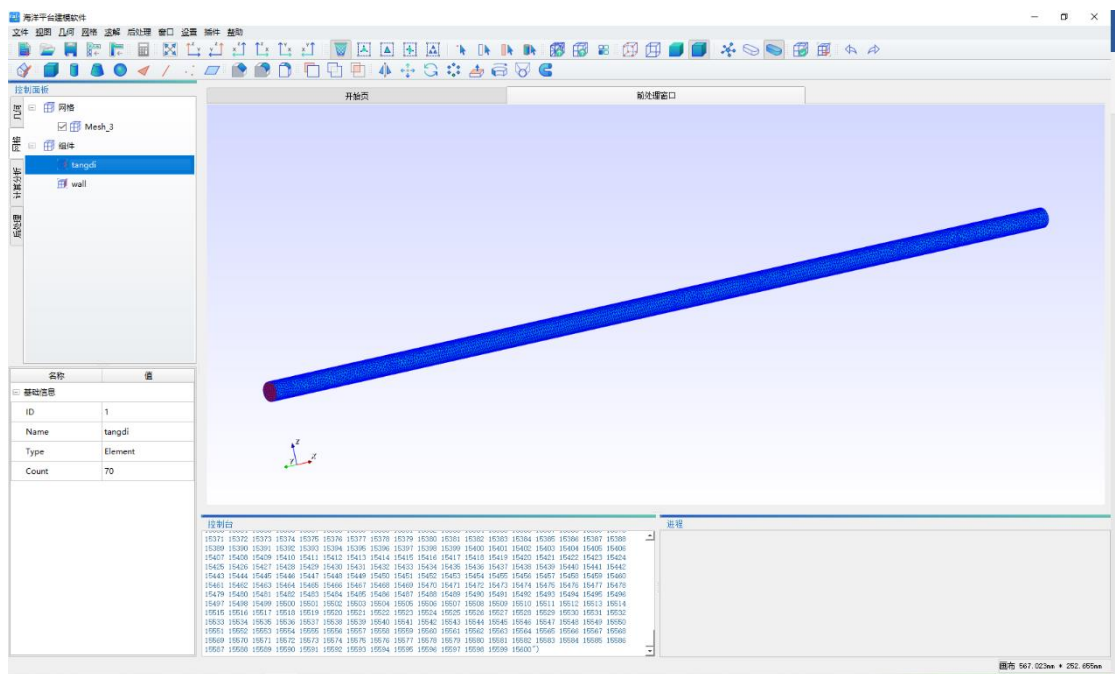
在操作/开发步骤中我们要对 FastCAE 的几何建模以及网格划分模块功能进行验证是否满足求解调用；开发步骤是在定制插件中完成，对求解所需的材料、仿真参数、边界条件、求解设置、求解过程监控曲线、后处理展示等全部求解器所需求解环节定制开发，此过程无需编写代码。

4.1 枪膛建模与网格划分

使用定制插件，打开几何建模模块以及网格划分模块，同时激活网格组件与网格质量检查功能。对 FastCAE 的几何建模与网格划分功能进行实验，验证网格划分的结果能否满足要求。



几何建模



网格剖分结果

经过测试，几何建模功能能够很快的完成建模操作，功能符合预期；网格能够区分膛底和膛壁，网格支持三角形和四边形，满足计算的需要。

4.2 软件定制

在软件定制中，我们先进行通用项的定制，包括软件名称、公司信息、需要的导入、导出网格功能，需要的几何功能。

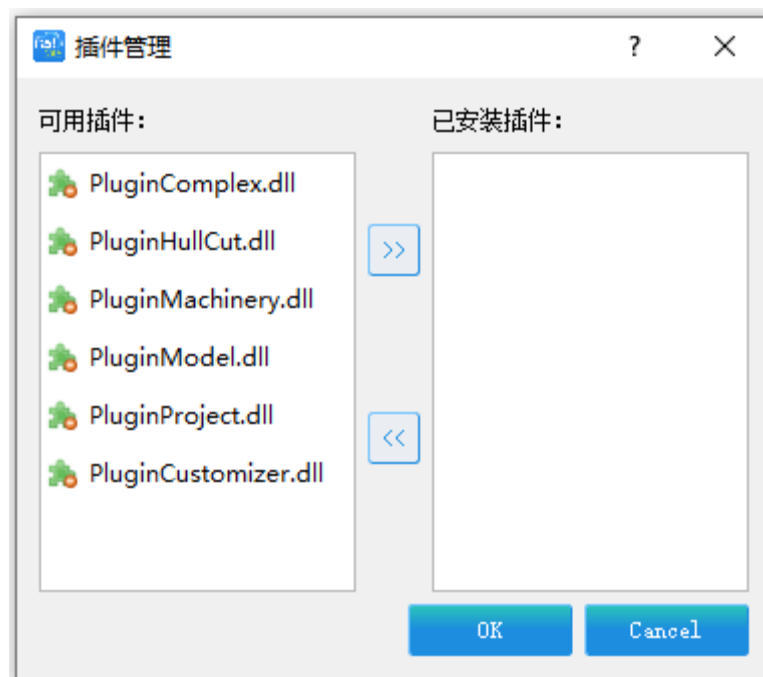
对于模型树的定制，本程序包含的参数身管形状极其参数、弹丸参数、火药参数、计算

控制参数等参数模块。

下面将详细的对定制过程进行说明。

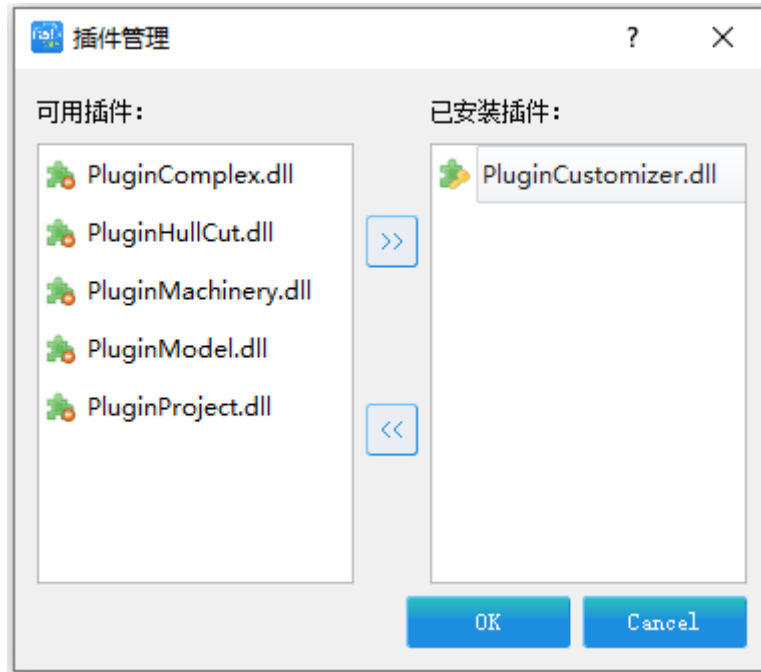
- 启动 FastCAE 2.0 版本软件加载 PluginCustomizer.dll 插件，开始进行软件常规项定制。

1. 点击【插件】 - 【插件管理】。



插件管理界面

2. 在可用插件列表中选中 PluginCustomizer.dll 插件，点击指向【已安装插件】方向剪头的按键。



添加定制插件

3.点击【OK】按键，这样 PluginCustomizer 插件将会被 FastCAE 软件装载。软件菜单栏上将会出现定制项。



插件添加成功

4.点击【定制】-【开始定制】，软件标题栏上名称后面会有“---定制中”字样，此时我们处于定制模式。



定制中

5.点击【帮助】-【关于】，软件将弹出软件基本信息设置界面，这里面我们按照下图输入我们想要填入的软件基本信息。然后点击【确定】按键，完成软件基本信息的录入。

软件基本参数

?
×

基本信息

中文名称 枪炮内弹道计算程序 *

英文名称 Interior Ballistic Analysis *

版本信息 1.0

单位名称 NUC

电子邮箱 libaojunqd@foxmail.com

单位网站 http://www.nuc.edu.cn/

图片信息

Logo (建议尺寸60X60)

欢迎页 (建议尺寸400X300)

确定

取消

软件基本信息

6.几何操作部分，我们选中特征建模功能。

几何 网格 求解 后处理 窗口

↶ 撤销 Ctrl+Z

↷ 重做 Ctrl+Y

☒ 特征建模

☐ 特征操作

☐ 草图

视图

选择

几何模块功能选择

7.网格操作部分，我们选中面网格划分、网格质量检查、创建组件功能。



网格模块功能选择

8.接下来在模型树中的材料中添加一个名称为“身管材料”的模型



添加材料

右键点击“材料”，选择添加材料，在弹出的对话框中输入所添加的材料的中英文名称。



材料名称设置

双击新生成的【身管材料】节点，添加一个浮点类型的参数

fast

身管材料

?

×

基本信息

中文名称

身管材料

英文名称

身管材料

图标

:/QUI/icon/material.png

上传图标

参数列表

添加

删除

编辑

清空全部

整型

浮点类型

布尔类型

字符串类型

枚举类型

表格类型

路径类型

名称	类型	默认值
----	----	-----

显示参数组

确定

取消

给材料添加浮点参数

fast

浮点类型参数

?

×

名称

屈服极限

单位

MPa

浮点精度

2

↑

↓

默认值

800.00

最小值

0

最大值

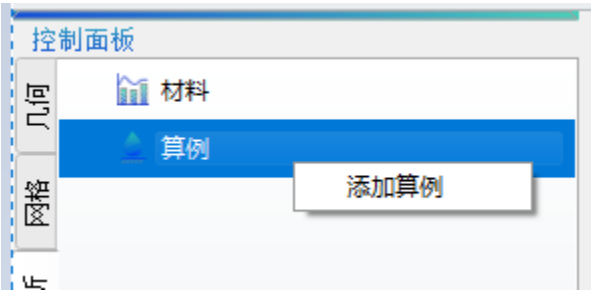
10000

确定

取消

浮点参数设置

9.鼠标左键选中算例节点， 点击鼠标右键， 点击弹出右键菜单的【添加算例】 菜单项。



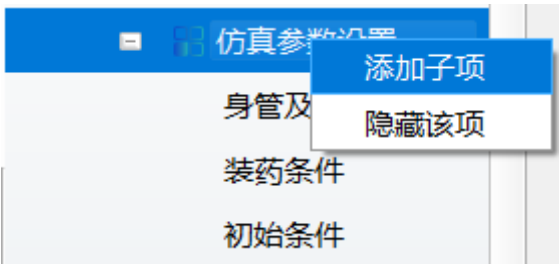
添加算例模型树

10.软件将弹出添加算例对话框， 我们按照下图添加入算例中文名称和英文名称。点击【确定】 按键完成算例名称信息的录入。



设置模型树名称

11.接下来进行参数的配置， 在【仿真参数设置】节点下面设置【身管及弹丸诸元】、【装药条件】、【初始条件】。



给仿真参数节点添加子节点

以下为三个参数节点模块分别包含的参数

身管构造及弹丸诸元表

名称	字母	类型	数值范围	默认值	单位
药室容积	V0	浮点型	0 1	2.07E-6	m3

弹丸质量	m	浮点型	0 1000	48.2E-3	Kg
------	---	-----	--------	---------	----

装药条件表

名称	字母	类型	数值范围	默认值	单位
火药力	f	浮点型	0 10000	980	KJ/kg
余容	a	浮点型	0 1	9.2E-4	
装药量	w	浮点型	0 100000	17	g
火药密度	Pp	浮点型	0 10000	1600	Kg/m3
热力系数	C	浮点型	0 1	0.22	
燃速系数	u1	浮点型	0 1	6.85E-8	
压力指数	n	浮点型	0 1	0.85	
弧厚	c1	浮点型	0 1	2E-4	m
形状特征量 1	cai	浮点型	0 1	0.8235	
形状特征量 2	r	浮点型	0 1	0.13207	
形状特征量 3	u	浮点型	-1 1	-0.0516	
形状特征量 4	p	浮点型	0 1	1.1518E-4	

初始条件表

名称	字母	类型	数值范围	默认值	单位
初始压强	p0	浮点型	0 10000	40	Mpa
空气压强	air	浮点型	0 1	0.1	Mpa
次要功系数 1	K1	浮点型	0 10	1.10	
次要功系数 2	b	浮点型	0 1	0.252	

以添加【身管及弹丸诸元】节点中的【药室容积】参数为例对添加参数方法进行说明：
 双击【身管及弹丸诸元】弹出该节点参数信息，点击【添加】按钮，选择浮点类型。

fast

身管及弹丸诸元

?

×

基本信息

中文名称

身管及弹丸诸元

英文名称

Barrel_pellet_parameters

图标

D:/FastCAE/ConfigFiles/icon//

上传图标

参数列表

添加

删除

编辑

清空全部

整型

浮点类型

布尔类型

字符串类型

枚举类型

表格类型

路径类型

名称	类型	默认值
药室容积	Double	0.00000207
弹丸质量	Double	0.0482

显示参数组

确定

取消

给浮点类型节点添加参数

根据表格中的参数信息对相应项进行填写，结果如下图


浮点类型参数
?
×

名称

药室容积

单位

m3

浮点精度

8

默认值

0.00000207

最小值

0

最大值

1

确定

取消

设置浮点类型参数


身管及弹丸诸元
?
×

基本信息

中文名称

身管及弹丸诸元

英文名称

Barrel_pellet_parameters

图标

D:/FastCAE/ConfigFiles/icon//

上传图标

参数列表

添加

删除

编辑

清空全部

序号	名称	类型	默认值
1	药室容积	Double	0.00000207
2	弹丸质量	Double	0.0482

显示参数组

确定

取消

节点参数列表

【身管及弹丸诸元】节点的参数设置完成后，点击【确定】即可保存本节点的参数设置信息。其他的浮点类型参数，按照本例以及参数图标填写即可。

12.在【装药条件】和【初始条件】分别有【形状特征量】以及【次要功系数】两个【参数组】，我们就【形状特征量】为例对如何添加【参数组】进行说明：双击【模型树】-【仿真参数设置】-【装药条件】，在弹出的对话框中点击【显示参数组】按钮。点解【参数组列表】下方的【添加】按钮，输入参数组名称“形状特征量”。点击【确定】后单击【形状特征量】，就可以在列表下方的【参数列表】添加参数了，我们将4个参数成员添加后效果如下图：

装药条件

?

×

基本信息

中文名称

装药条件

英文名称

GunpowderConditions

图标

D:/FastCAE/ConfigFiles/icon//

上传图标

参数列表

添加

删除

编辑

清空全部

序号	名称	类型	默认值
1	装药量	Double	17.00
2	火药力	Double	980.00
3	余容	Double	0.00092
4	密度	Double	1600.00
5	热力系数	Double	0.22
6	燃速系数	Double	0.000000...
7	压力指数	Double	0.85
8	弧厚	Double	0.0002

参数组列表

添加

删除

编辑

清空全部

序号	名称
1	形状特征量

添加

删除

编辑

清空全部

序号	名称	类型	默认值
1	cai	Double	0.8235
2	r	Double	0.13207
3	u	Double	-0.0516
4	p	Double	0.00011518

显示参数组

确定

取消

参数组列表

13.按照此例将所有节点中的参数按照参数表信息进行设置即可，效果如下：

控制面板

几何

网格

计算分析

后处理

身管材料

算例

内弹道计算

网格组件

仿真参数设置

身管及弹丸诸元

装药条件

初始条件

边界条件

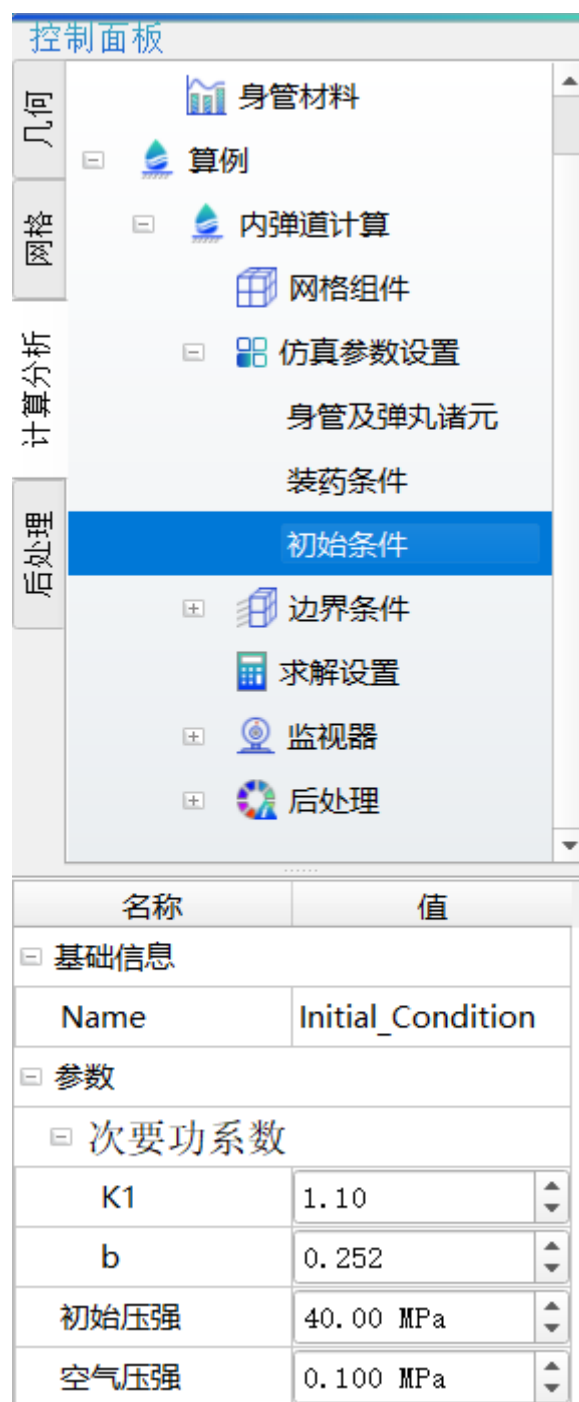
求解设置

监视器

后处理

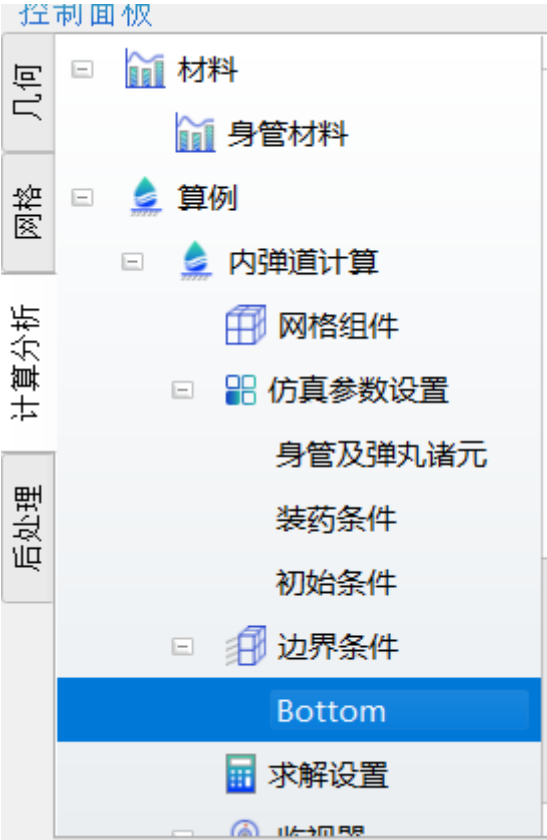
名称	值
基础信息	
Name	Barrel_pellet_pa...
参数	
药室容积	0.00000207 m3
弹丸质量	0.0482 Kg

【身管及弹丸诸元】节点参数



【初始条件】节点参数

14.在【边界条件】节点添加一个名称为【Bottom】的节点，无需添加多余参数。方法为右键单击【边界条件】按钮，选择【添加边界条件】，在弹出的对话框中输入“Bottom”，然后单击【确定】。



【边界条件】添加

15.在【求解设置】节点添加如下参数

计算控制参数

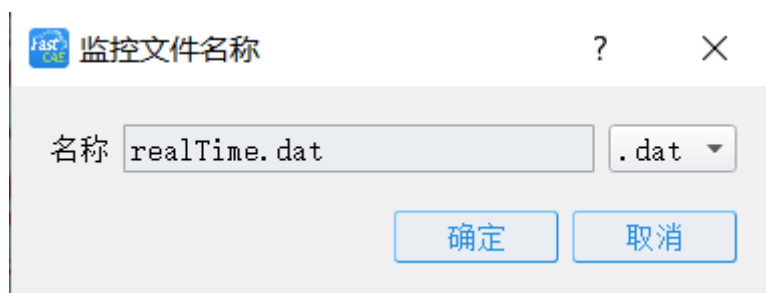
名称	字母	类型	数值范围	默认值	单位
时间步长	h	浮点型	0 1	0.01	s

预览效果如下图：



【求解设置】节点参数

16.在【监视器】添加求解过程中监控的曲线。右键单击【监视器】，选择【添加监控文件】，目前 FastCAE2.0 版本中，监控文件只支持后缀为【.dat】格式的文本文件。



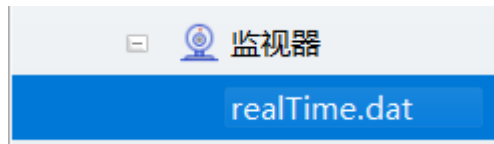
监控文件名称编写

我们根据 realTime.dat 文件的信息，对要监控的曲线信息进行填写，如下图，我们要监控的是一条横坐标为【t(s)】，纵坐标为【K】的曲线

1	t (s)	K
2	0	0.00494442
3	1.00513e-007	0.00570016
4	2.01027e-007	0.00655256
5	3.0154e-007	0.00751169
6	4.02054e-007	0.00858847
7	5.02567e-007	0.0097947
8	6.03081e-007	0.0111432
9	7.03594e-007	0.0126476
10	8.04108e-007	0.0143228
11	9.04621e-007	0.0161849

realTime.dat 文件格式

双击【realTime.dat】节点，点击【添加按钮】，添加一条名称为【火药燃烧情况】的曲线



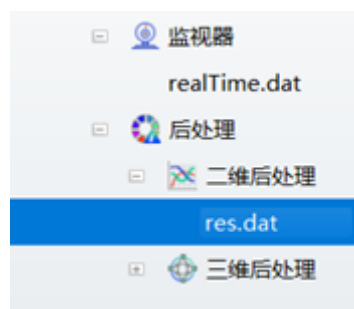
节点挂载成功



曲线信息情况

17.接下来进行后处理结果展示的配置。

右键单击【后处理】-【二维后处理】节点，选择【添加文件】，该案例中求解生成的后处理文件名为【res.dat】，因此在弹出的对话框中添加该文件名称，然后点击【确定】。

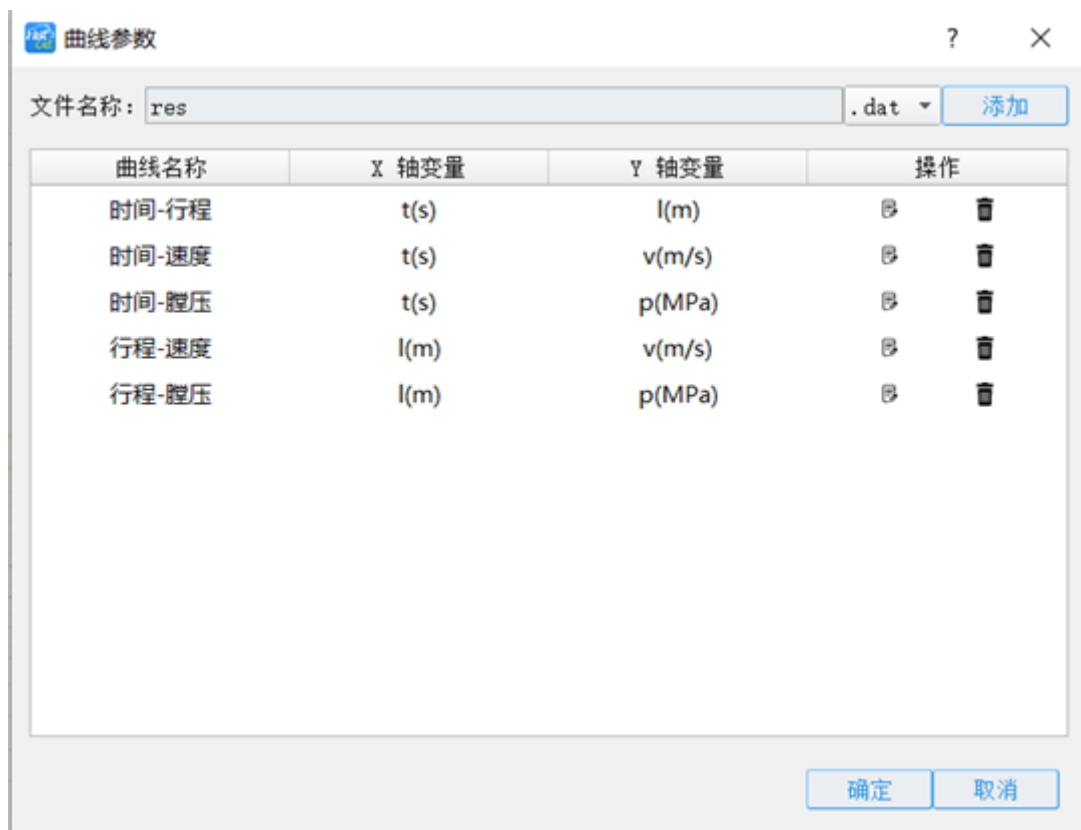


后处理二维曲线节点

1	t(s)	l(m)	v(m/s)	p(MPa)	K	Z
2	0	0	0	40	0.00494442	0.0059994
3	1.00513e-007	4.66892e-010	0.010598	46.114	0.00570016	0.00691557
4	2.01027e-007	2.0699e-009	0.0228044	53.0099	0.00655256	0.00794864
5	3.0154e-007	4.97967e-009	0.0368236	60.7694	0.00751169	0.00911074
6	4.02054e-007	9.38837e-009	0.0528811	69.4807	0.00858847	0.010415
7	5.02567e-007	1.55118e-008	0.0712255	79.2393	0.0097947	0.0118755
8	6.03081e-007	2.35919e-008	0.0921304	90.1486	0.0111432	0.0135075
9	7.03594e-007	3.38992e-008	0.115896	102.32	0.0126476	0.0153275

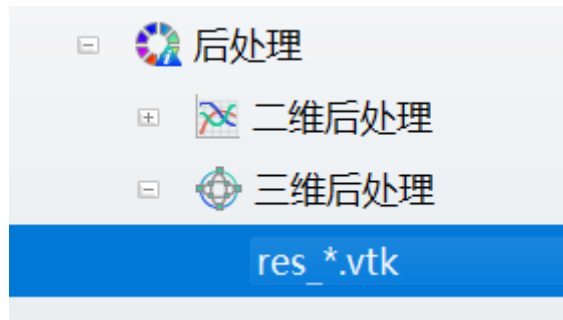
res.dat 文件格式

以上为 res.dat 的开头部分，我们根据想要查看的二维曲线的纵横坐标名称填写信息即可。方法为双击【后处理】 - 【二维后处理】 - 【res.dat】节点，然后点击【添加】按钮，在弹出的对话框中添加【曲线名称】为“时间-行程”，【X 轴】为“t(s)”，【Y】名称“l(m)”。点击【确定】完成该条曲线的添加。该文件中共有 5 条后处理曲线，参照说明，按下图添加曲线即可，完成后再次点击【确定】按钮。



五条曲线参数设置

18.下面添加三维后处理文件信息。方法为右键点击【后处理】 - 【三维后处理】，选择【添加文件】。本案例中三维后处理文件的名称格式为 res*.vtk (*代表含义为一组包含顺序编号的文件)。因此这里只添加一个文件名称为“res_*.vtk”的节点代表代表一组文件，添加完文件名称后，点击【确定】按钮，【res_*.vtk】节点生成。



Vtk 文件设置

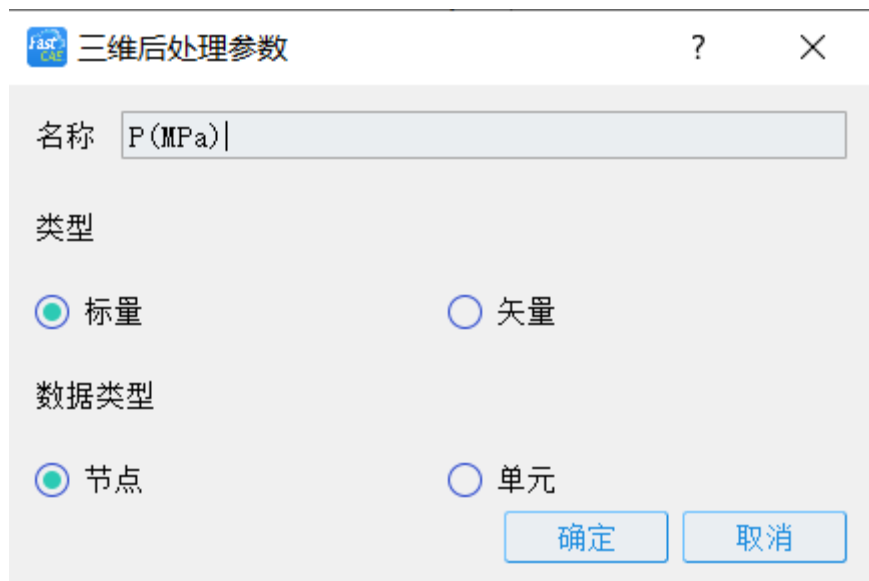
双击【res_*.vtk】节点，在弹出的窗口中，找到【序号 1】一行。

如下图每一行都有两个图标按钮，左侧图标代表【编辑】、右侧图标代表【删除】。



编辑/删除图标按钮

点击【编辑】按钮，【输入名称】输入“P(MPa)”，【类型】选择【标量】，【数据类型】选择【节点】。



Vtk 文件属性设置

完成后点击【确定】返回上一步。再点击【确定】保存并退出。至此，模型树配置完毕。

19.案例求解器设置，求解器设置不需要在 PluginCustomizer.dll 插件中，即定制模式下进行设置。用户点击菜单栏【定制】-【完成定制】，将 FastCAE 2.0 软件切换至常规模式，点击菜单栏【求解器】-【求解器管理】项，软件将弹出求解器管理界面。



求解器管理列表

20. 点击求解器管理界面的【添加】按键，软件将弹出添加求解器界面，按照下图所示添加求解器信息。需要注意的是这里的求解器路径要选择 InteriorBallisticAnalysis.exe 可执行文件所在的文件路径。点击界面中的【确认】按键完成求解器的添加。



求解器编辑界面

21.此时的求解器管理界面如下图所示，点击界面右上角的【关闭】按键，完成求解器的设置。



求解器信息填写完成

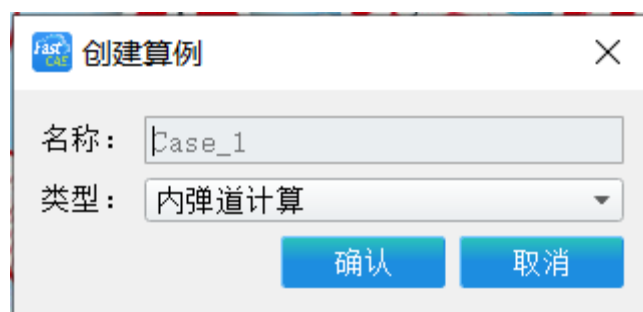
4.3 软件使用

1.此时我们已经完成了定制部分工作，我们将开始运行我们所做的程序进行求解工作。
鼠标左键点击【控制面板】-【计算分析】-【算例】节点，点击鼠标右键软件将弹出【算例】节点的右键菜单。



创建新的算例

2. 点击【算例】节点弹出的右键菜单中的【创建算例】菜单项，软件会弹出创建算例对话框。这里我们使用默认的名称，直接点击【确认】按钮完成算例的创建。如果有需要用户可以输入其他的算例名称。



编辑算例名称

3. 此时算例下的树形结构如下图所示。点击【算例】下的【仿真参数设置】节点，我们可以看到仿真参数设置节点下的定制的参数。用户可以根据需求，自行调整参数值。

控制面板

几何

网格

计算分析

后处理

材料

算例

Case_1

网格组件

仿真参数设置

身管及弹丸诸元

装药条件

初始条件

边界条件

求解设置

监视器

火药燃烧情况

名称	值
基础信息	
Name	Barrel_pellet_par...
ID	1
参数	
药室容积	0.00000207 m3
弹丸质量	0.0482 Kg

设置参数

4.点击【算例】下的【求解设置】节点，我们可以看到求解设置节点下的定制的参数。
用户可以根据需求，自行调整参数值。

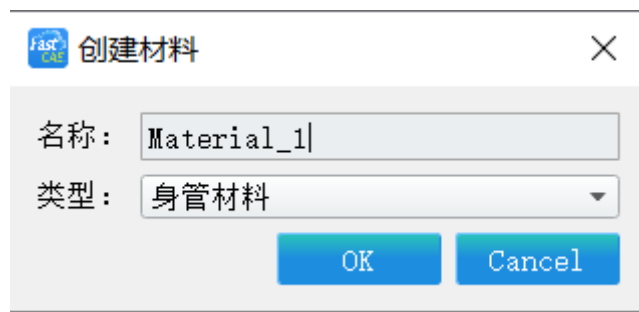


设置参数

5.创建材料。右键点击【材料】-【创建材料】节点，将弹出创建材料对话框，输入【材料名称】并选择【材料类型】即可。



创建材料

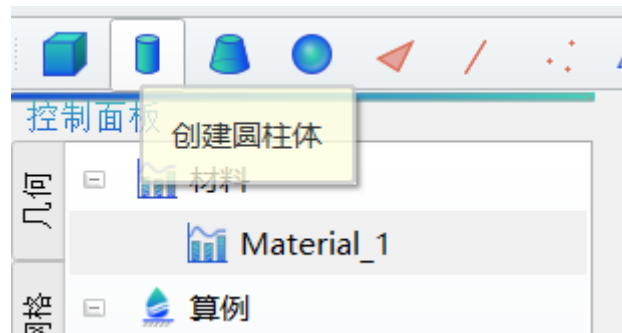


编写材料名称



创建材料节点挂载在模型树上

6.接下来进行几何建模。点击【控制面板】上方的【创建圆柱体】按钮。



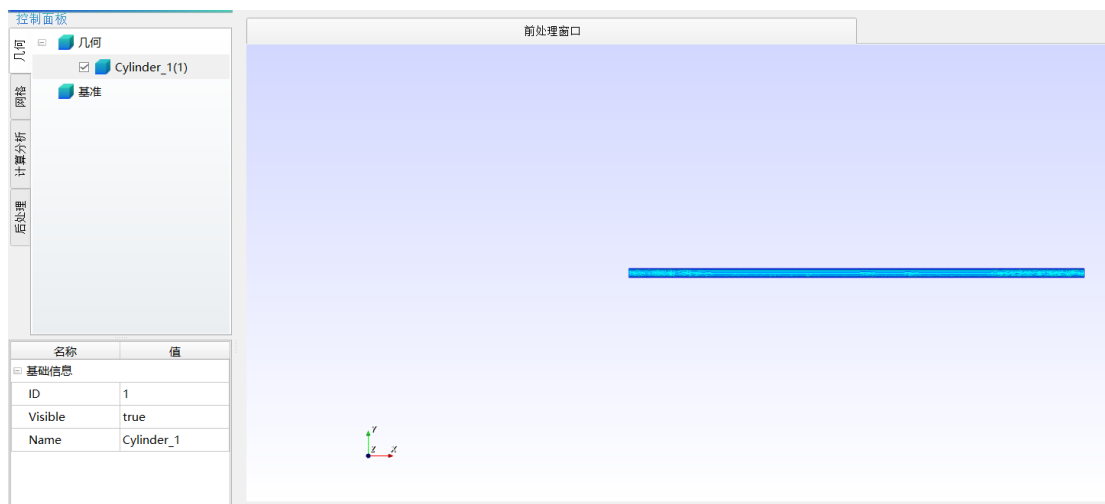
创建圆柱体

定义一个半径 6.35mm，长度 654.00，轴线为 X 轴的圆柱体



圆柱体参数设置

完成后效果如下图：



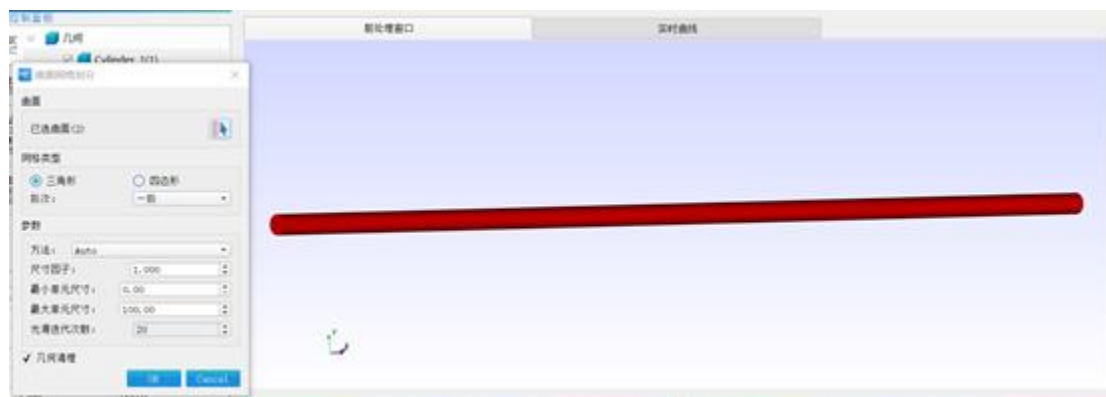
圆柱体创建后可视化

7.对生成的几何模型进行网格剖分，在【控制面板】-【几何】中检查刚刚生成的几何模型“Cylinder_1”是否已被勾选，然后点击【面网格剖分按钮】。



网格剖分参数设置

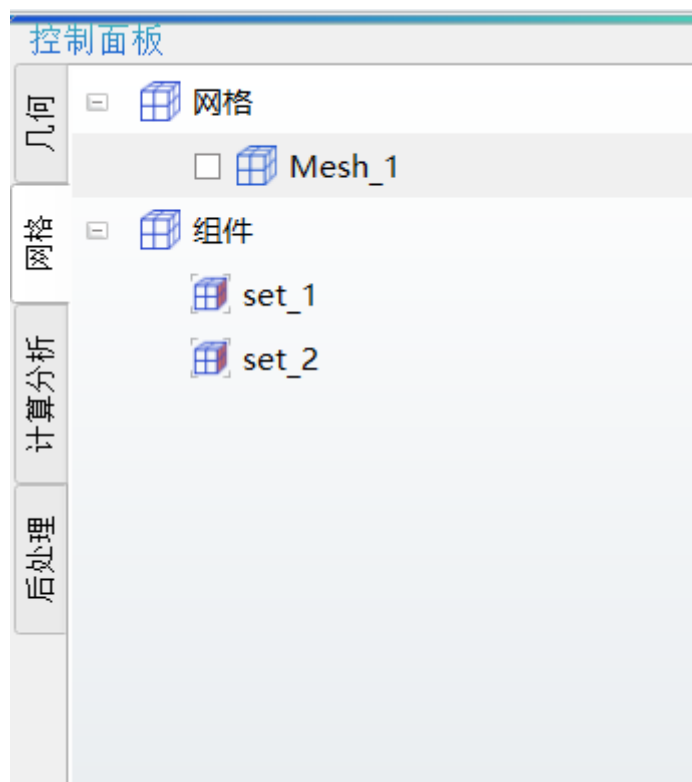
弹出的对话框中，所有选项默认即可，这里我们注意选择需要划分的曲面是圆柱的侧面和 x 坐标较小的底面，点击【选择曲面】按钮开始选择曲面，被标记为红色就代表曲面被选中，可以同时选中多个曲面。下图为选择两个曲面后的效果。选择完成后点击【OK】。



网格剖分面选择

网格生成时间较短，可以看到【网格】下已经生成名称为 Mesh_1 的网格。

8. 接下来进行组件的生。确保【几何】中的 Cylinder_1 文件被勾选，【几何】中 Mesh_1 网格没有被勾选。我们要生成一个底面以及圆柱侧面两个，首先点击【选择面】按钮，然后选择 x 坐标较小的底面，被标记为红色就代表被选中。然后点击【创建组件】按钮，在弹出对话框中，【名称】默认即可、【类型】选择为【单元】，完成后点击【确认】。同理，再次点击【选择面】按钮，选择圆柱侧面重复操作。可在【网格】-【组件】节点中看到已经生成的两个组件 set_1 和 set_2。



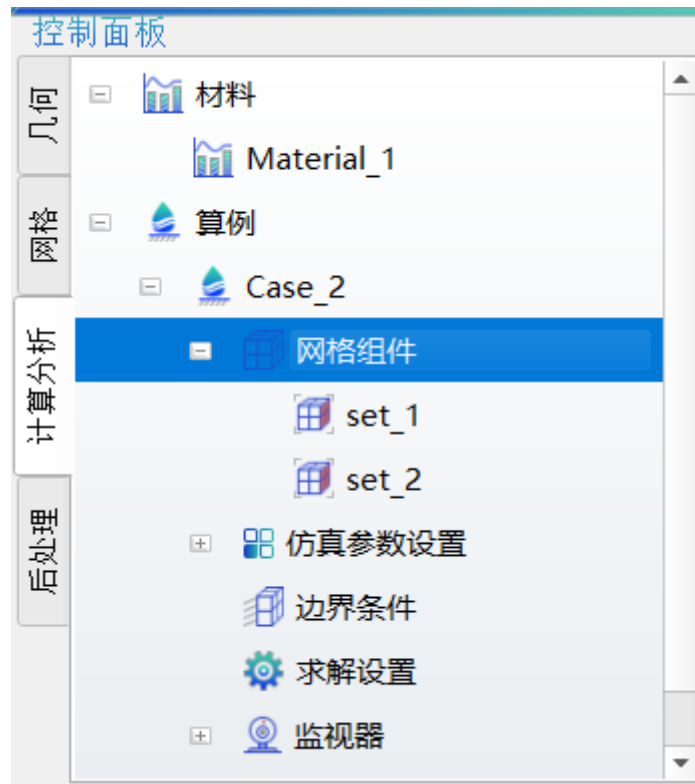
set_1 和 set_2 生成成功

9.这一步挂载网格组件，需要用到我们上一步生成的组件。右键单击算例模型树中的节点【网格组件】，选择【导入】，将【可选】列表中的 set_1 和 set_2 全部选中，点击指向【已选】列表按钮，set_1 和 set_2 移动到【已选】列表中



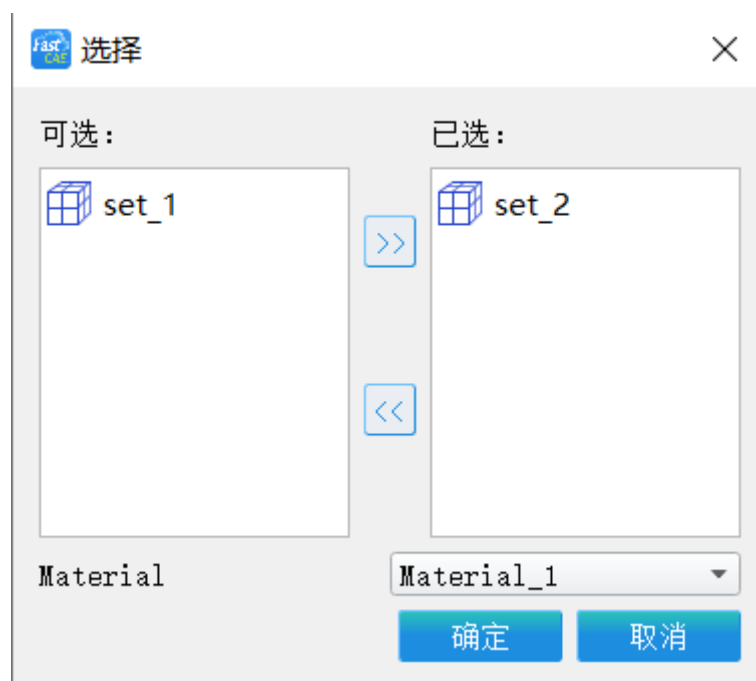
挂载组件

点击【确定】，两个组件被挂载到了【网格组件】节点下，如下图：



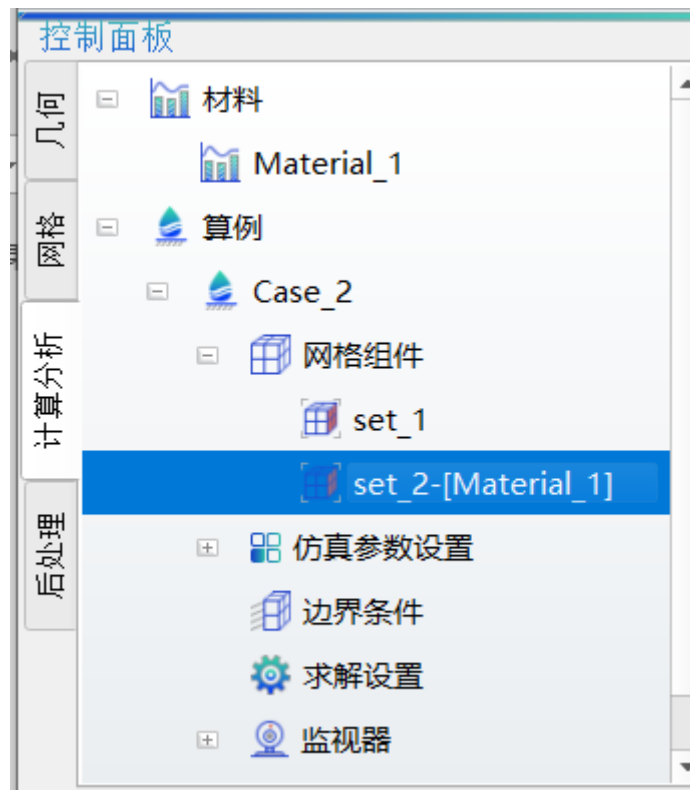
组件挂载成功

10.还需要为管壁 set_2（即圆柱侧壁指定材料），右键点击【网格组件】，选择【指定材料】。



set_2 绑定材料

选中【set_2】点击指向【已选】按钮移动到【已选】列表当中，【Material】默认选择为 Material_1,点击【确定】完成，如下图，set_2 和 Material_1 绑定：



材料绑定成功

11.下面添加边界条件，右键单击【边界条件】节点，选择【添加边界条件】。分别添加如下两种边界条件。



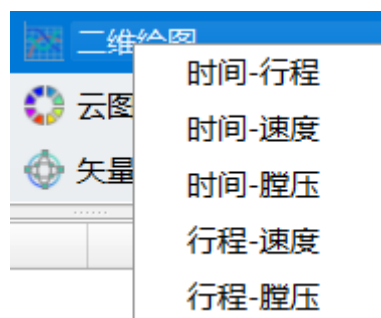
添加边界条件

12.至此我们的模型树参数、网格组件、边界条件等配置完成，效果如下图：

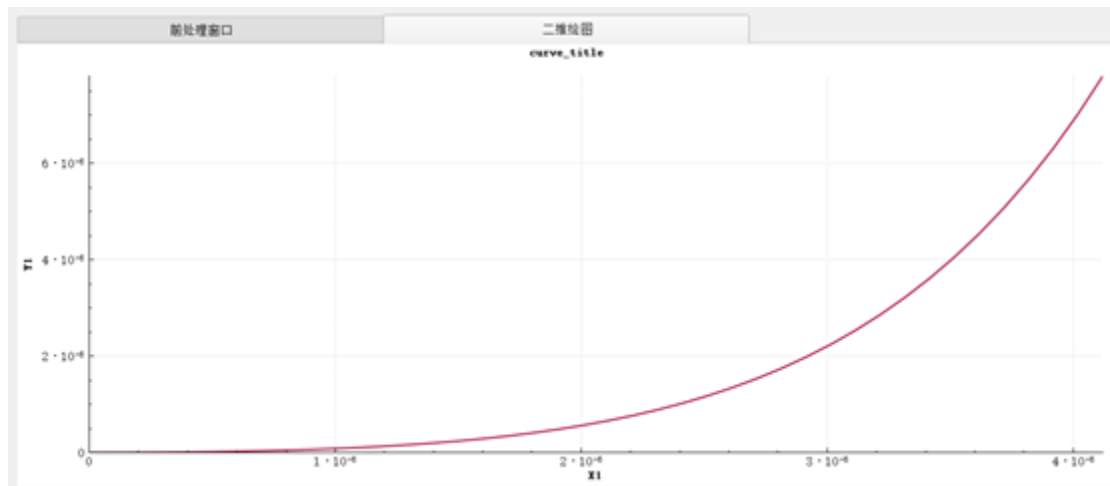


边界条件添加成功

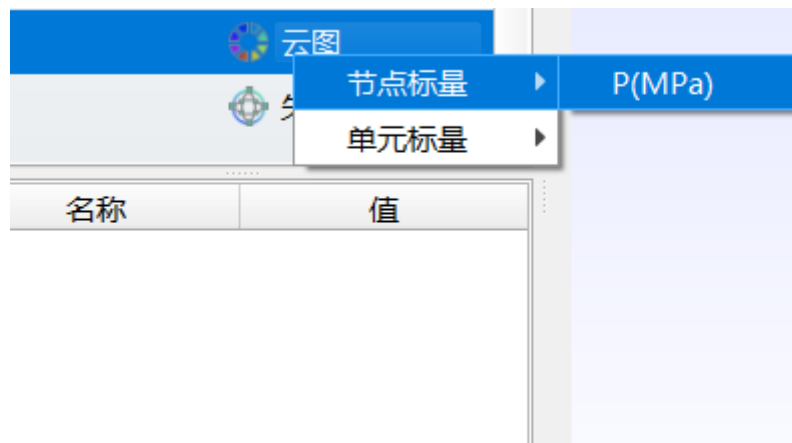
13. 点击工具栏的【求解】按键，软件将启动求解。在求解器完成后用户鼠标右键选择点击【后处理】节点下的【二维绘图】节点，在弹出的右键菜单中选择要查看的二维曲线。



后处理二维曲线



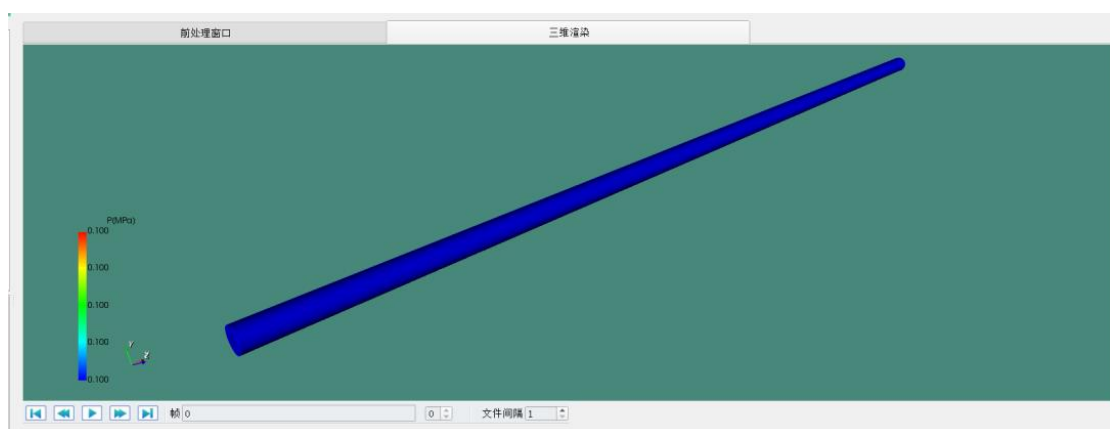
二维曲线可视化



三维可视化

14.完成上一步的操作后，软件后处理将访问我们算例下 Result/ res.dat 结果文件。

15.鼠标右键选择点击【后处理】节点下的【云图】-【节点标量】-P(MPa)节点，主窗口查看计算生成的 vtk 文件。



三维可视化

Fluent 集成示例

FastCAE-Fluent 集成示例（可视化集成与插件拓展集成）

一、概述

1.1 案例介绍

本案例基于 FastCAE 提供的插件拓展机制与可视化定制功能，针对石油行业中使用旋滤器进行油水分离的特定工况，开发了 Fluent 适配插件，实现了 FastCAE 与 Fluent 的数据交换，以及 Fluent 的计算驱动，形成了这对这一特定工况的仿真分析软件系统，该软件系统与 Fluent 无缝集成，是一套集前后处理与求解计算于一体的专业计算平台。

本案例名称为《Fluent 集成案例》，其中所有资料包含源码和相关文档均可以在本站“[下载](#)”菜单栏目内进行下载。

1.2 集成目标

- 通过 FastCAE2.0 集成框架完成一个典型的以 Fluent 软件为求解器的软件产品的集成工作。
- 通过本案例能够使用户快速了解使用 FastCAE 2.0 进行插件开发与定制开发方式相结合的开发过程，为 Fluent 软件集成与二次开发提供指导。

1.3 环境说明

- 开发环境
- Windows 10
- VS2013 Community(VS 2013 其他版本也可以)
- Qt 5.4.2-msvc-opengl-x64
- FastCAE 2.0
- Fluent 18.0

1.4 注意事项

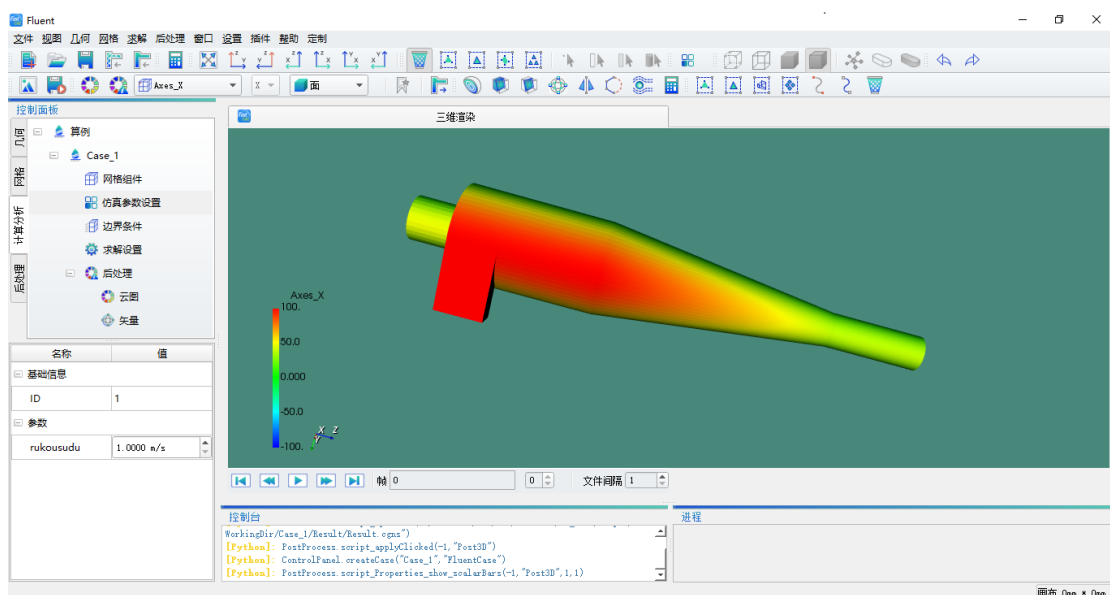
- 建议开展本案例学习之前，用户需要有一定 C++基础和 Fluent 使用经验。
- 由于 Fluent 导入 jou 脚本里文件路径不能有中文，所以脚本所使用的网格文件应放在不包含中文的路径中。在 FastCAE 使用过程中应尽量避免代码存放位置及文件存放位置中包括中文路径。

- Fluent 插件开发部分代码设置的边界条件及写出的网格边界条件代码仅适用于类似于本案例的，边界条件只使用到 Fluent 边界条件中 velocity-inlet, outflow, wall 这三种的案例。用户可以按照自己的需求，修改对应边界处的代码来满足具有其他边界要求的 Fluent 集成软件开发。

二、软件实现效果

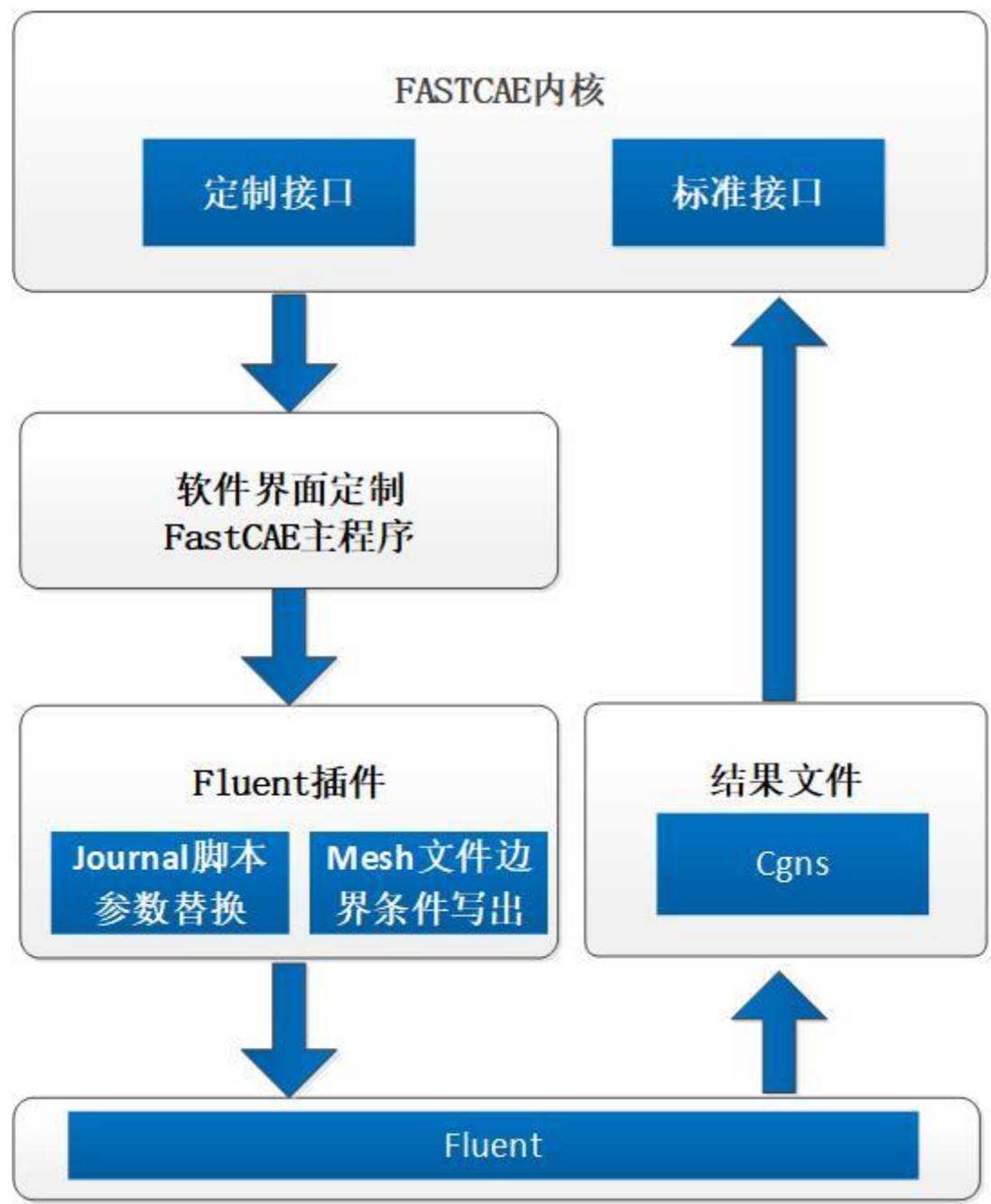
本案例最终形成了基于 Fluent, 针对特定对象与特定工况的软件系统, 该系统的 CFD 仿真计算由 Fluent 内核实现, 前后处理功能与流程通过 FastCAE 定制实现。前后处理的个性化流程定制通过 FastCAE 的可视化定制功能实现, 前后处理与 Fluent 的数据交换通过 Fluent 适配插件实现。这套针对于特定对象和特定工况的 CFD 软件固化了分析流程, 只需要极少的参数设置就能完成复杂的 CFD 计算, 在保证分析精度的前提下, 降低了学习成本, 提高了仿真分析的效率与可靠性。同时也为其他情境下的复杂 CFD 仿真流程简化提供了可行的解决方案。

在完成软件的开发工作后, 用户可以运行软件进行数值计算, 我们可以看到, 在主界面左侧为控制面板, 我们的工程树设置, 关联网格, 设置边界条件等都可以在工程树上进行操作。右侧主体部分为展示窗体区域, 前处理窗口, 后处理三维渲染窗口都在这个区域进行显示。在下面相邻的两个区域分别为控制台和进程窗口, 控制台用于显示用户的操作流程, 进程窗口则主要对求解状态进行显示及控制。完成的软件整体效果如下图所示。



三、原理说明

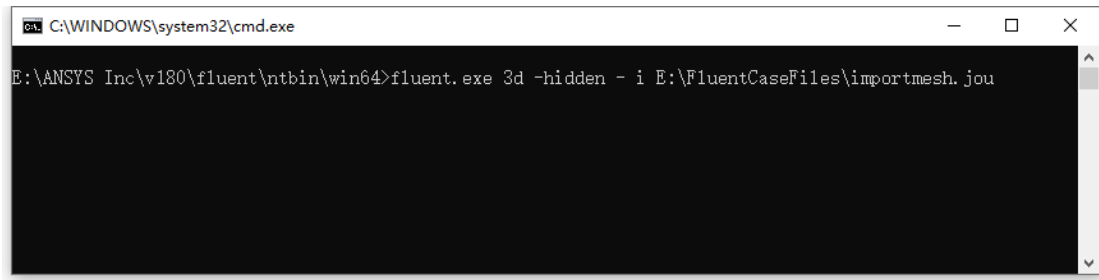
本案例的核心集成原理为：通过 FastCAE 平台软件定制界面，并生成 Fluent 运行所需的脚本文件，后台驱动 Fluent 软件进行科学计算并对计算结果进行展示。



案例原理图

3.1 Fluent 部分

- Fluent 软件支持通过命令行的方式使用 Jou 脚本进行启动运行。



命令下执行脚本文件

如上图，fluent.exe 就会在后台加载 importmesh.jou 脚本进行运算。

Fluent 命令行参数 -i journal 表示读取指定的脚本。-hidden 表示以最小化形式运行 fluent。3d 表示使用的网格文件是三维的。需要说明的是 -i journal 命令行参数时全平台适用的，而 -hidden 命令行参数只支持 windows 平台。

3.2 FastCAE 部分

根据 3.2FastCAE 部分内容，我们可以分析得出 **FastCAE 部分**需要调用 fluent.exe 通过传参的形式让 fluent.exe 在后台运行，即我们在 **FastCAE 部分**需要完成 journal 脚本内容的设置、显示、及脚本文件中参数的替换的工作。

其中，FastCAE 定制化插件部分及软件框架将提供软件参数的设置、网格的导入及展示、边界条件设置、后处理结果展示功能。这部分功能不需要用户重新开发，只需要进行配置就能够使用。而当前版本 FastCAE2.0 没有支持的功能：msh 文件边界条件写出功能、jou 脚本参数替换功能。这部分功能将由定制化脚本提供。

四、操作开发步骤

首先我们需要进行插件开发，将当前 FastCAE2.0 版本不支持的功能通过插件的形式进行开发，通过平台加载插件的形式完成这部分功能的支持。

由于我们开发的插件中具有 jou 脚本参数替换功能，这就需要在完成插件开发后，按照要求对我们计算需要的 jou 脚本进行修改进而形成 jou 脚本模板。

完成 jou 脚本模板边界后，我们需要按照我们软件的要求对软件的界面及功能进行定制。定制完成后，我们就可以开始我们所开发软件的使用旅程了！

4.1

4.1 Fluent 插件开发

- 插件通用部分代码修改。

我们的插件继承自插件基类 PluginBase, 因此基类的纯虚函数需要在我们我们的插件类中实现。这里我们把插件类名字命名为 PluginFluent。此时类所在的头文件 pluginFluent.h 内容如下:

```
namespace Plugins
{
    class _FLUENTPLUGINAPI FluentPlugin : public PluginBase
    {
        Q_OBJECT
    public:
        FluentPlugin(GUI::MainWindow* m);
        ~FluentPlugin();

        //加载插件
        bool install() override;

        //卸载插件
        bool uninstall() override;

    private:
        GUI::MainWindow* _mainwindow{};
    };
}

extern "C"
{
    void _FLUENTPLUGINAPI Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* plugs);
}
```

此时类所在的 CPP 文件 pluginFluent.cpp 内容如下:

```
void Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* ps)
{
```

```

        Plugins::PluginBase* p = new Plugins::FluentPlugin(m);
        ps->append(p);
    }
namespace Plugins
{
    FluentPlugin::FluentPlugin(GUI::MainWindow* m)
    {
        _mainwindow = m;
    }
    FluentPlugin::~~FluentPlugin()
    {
    }
    bool FluentPlugin::install()
    {
        return true;
    }
    bool FluentPlugin::uninstall()
    {
        return true;
    }
}

```

在基类 PluginBase 中有一个 Protected 成员变量 QString _describe，它是用于描述当前插件信息的字符串变量。这里我们在 FluentPlugin 类的构造函数中对其赋值。

```
_describe = "Fluent Function Plugin";
```

- 插件需要完成函数选择。

当前 FastCAE2.0 插件支持的输入输出扩展函数包括:

```

//注册写出的后缀与方法
static void RegisterInputFile(QString suffix, WRITEINPFILE fun);

```

```
//注册结果文件转换方法

static void RegisterOutputTransfer(QString name, TRANSFEROUTFILE fun);

//注册网格文件导入的方法

static void RegisterMeshImporter(QString suffix, IMPORTMESHFUN fun);

//注册网格导出的方法

static void RegisterMeshExporter(QString suffix, EXPORTMESHFUN fun);
```

需要注意的是 RegisterInputFile 函数注册完成后，当在求解器设置成我们所扩展的格式时，该函数会在求解器求解之前进行调用。而 RegisterOutputTransfer 函数注册完成后，会在完成求解器求解生成结果文件后调用。RegisterMeshImporter 和 RegisterMeshExporter 方法是为新格式的网格文件导入导出使用的函数。

根据我们的需求我们需要在**求解器求解之前**完成 Msh 格式网格文件的边界条件按照用户设置进行替换及 jou 脚本中参数替换。因此我们选择 RegisterInputFile 方法作为我们拓展的方法。因此我们在头文件中 C 函数部分添加：

```
//<MG 求解器求解之前的工作

bool _FLUENTPLUGINAPI BeforeCalcFunc(QString path, ModelData::ModelDataBase* d);

    在 CPP 文件中添加：

bool _FLUENTPLUGINAPI BeforeCalcFunc(QString path, ModelData::ModelDataBase* d)
{
    return true;
}
```

并在 BeforeCalcFunc 函数中添加相应的功能代码，具体代码可以参见本段落最终头文件和 CPP 文件全部内容部分。

- 插件功能开发。

插件最终头文件内容：

```
#ifndef _MODELPLUGIN_H_

#define _MODELPLUGIN_H_

#include "FluentPluginAPI.h"
```

```
#include "PluginManager/pluginBase.h"

#include "DataProperty/modelTreeItemType.h"

namespace GUI
{
    class MainWindow;
}

namespace MainWidget
{
    class PreWindow;
}

namespace ModelData
{
    class ModelDataBase;
}

namespace ProjectTree
{
    class ProjectTreeBase;
}

namespace Material
{
    class Material;
}

namespace Plugins
{
    class _FLUENTPLUGINAPI FluentPlugin : public PluginBase
    {
        Q_OBJECT
    public:
        FluentPlugin(GUI::MainWindow* m);

        ~FluentPlugin();
    };
}
```

```

//加载插件

bool install() override;

//卸载插件

bool uninstall() override;

private:

GUI::MainWindow* _mainwindow{};

};

}

extern "C"

{

    void _FLUENTPLUGINAPI Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>*
    plugs);

    ///<MG 求解器求解之前的工作

    bool _FLUENTPLUGINAPI BeforeCalcFunc(QString path, ModelData::ModelDataBase* d);

}

#endif

```

插件最终 CPP 文件内容：

```

#include "pluginFluent.h"

#include "mainWindow/mainWindow.h"

#include <QString>

#include "ModelData/modelDataFactory.h"

#include "ModelData/modelDataSingleton.h"

#include "ModelData/modelDataBase.h"

#include "MainWidgets/ProjectTreeFactory.h"

#include "IO/IOConfig.h"

#include "IO/TemplateReplacer.h"

#include "MeshData/meshSingleton.h"

#include "MeshData/meshSet.h"

#include "MeshData/meshKernal.h"

```

```

#include "BCBase/BCBase.h"

#include "ConfigOptions/ConfigOptions.h"

#include "ConfigOptions/SolverInfo.h"

#include <QDebug>

#include <QFile>

#include <QApplication>

#include <QDir>

void Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* ps)
{
    Plugins::PluginBase* p = new Plugins::FluentPlugin(m);
    ps->append(p);
}

namespace Plugins
{
    FluentPlugin::FluentPlugin(GUI::MainWindow* m)
    {
        _mainwindow = m;
        _describe = "Fluent Function Plugin";
    }

    FluentPlugin::~FluentPlugin()
    {
    }

    bool FluentPlugin::install()
    {
        IO::IOConfigure::RegisterInputFile("Fluent", BeforeCalcFunc);
        return true;
    }

    bool FluentPlugin::uninstall()
    {
        IO::IOConfigure::RemoveInputFile("Fluent");
    }
}

```

```

        return true;
    }
}

QString BCType2QString(BCBase::BCType type)
{
    if (type == BCBase::Inlet)
        return "velocity-inlet";
    else if (type == BCBase::Outlet)
        return "outflow";
    else if (type == BCBase::Wall)
        return "wall";
}

struct BCExchange
{
    QString bcName;
    BCBase::BCType toType;
};

bool ReplaceMeshFileBCType(const QString & fileName, QVector<BCExchange>
exchangInfos)
{
    QFile file(fileName);
    if (!file.exists()){ return false; }
    QString head = QString("(45 (");
    QString tail = QString("))\r\n");
    if (file.open(QIODevice::ReadWrite))
    {
        file.seek(file.size() - 1000);
        QString toContent;
        while (!file.atEnd())
        {

```



```

        QString lineContent = file.readLine();
        if (lineContent.size() < 6)
        {
            toContent += lineContent;
            continue;
        }
        if (lineContent.left(head.size()) == head && lineContent.right(tail.size()) == tail)
        {
            QStringList infos = lineContent.mid(5, lineContent.size() - 5 - 6).split(" ",
QString::SkipEmptyParts);
            if (infos.size() == 3)
            {
                QString id = infos[0], type = infos[1], name = infos[2];
                bool needReplace = false;
                for (BCExchange one : exchangInfos)
                {
                    if (name == one.bcName)
                    {
                        QString setName = "(45 (" + id + " " + name + " " +
BCType2QString(one.toType) + " " + name + ")))\r\n";
                        toContent += setName;
                        needReplace = true;
                    }
                }
                if (!needReplace)
                    toContent += lineContent;
            }
        }
        else
            toContent += lineContent;

```

```

    }

    file.seek(0);

    QString totalContent = file.read(file.size() - 1000) + toContent;

    file.seek(0);

    file.resize(totalContent.size());

    file.write(totalContent.toStdString().c_str());

    return true;

}

else

    return false;

}

bool BCOperation(ModelData::ModelDataBase* d, QString & meshFilePath)
{

    int nBC = d->getBCCount();

    if (nBC == 0){ return false; }

    QMap<QString, QVector<BCExchange>> bcExchangeInfos;

    for (int iBC = 0; iBC < nBC; ++iBC)

    {

        BCBase::BCBase * pBC = d->getBCAt(iBC);

        if (!pBC){ continue; }

        ///<MG 获取边界条件类型，及对应边界条件网格组件的名称

        BCBase::BCType bcType = pBC->getType();

        int meshSetID = pBC->getMeshSetID();

        MeshData::MeshSet * meshSet =

MeshData::MeshData::getInstance()->getMeshSetByID(meshSetID);

        if (!meshSet){ continue; }

        QString meshSetName = meshSet->getName();

        QList<int> meshKernelIDs = meshSet->getKernels();

        for (int KernelID : meshKernelIDs)

        {

```

```

MeshData::MeshKernal * pKernal =
MeshData::MeshData::getInstance()->getKernalByID(KernalID);

if (!pKernal){ continue; }

QString filePath = pKernal->getPath();

///<MG 判定类型，如果是 mesh 将原有名字为 meshSetName 的单元 改成
bcType 类型单元

if (filePath.contains(".msh"))
{
    BCExchange one;

    one.bcName = meshSetName;

    one.toType = bcType;

    if (bcExchangeInfos.contains(filePath))
    {
        bcExchangeInfos[filePath].push_back(one);
    }
    else
    {
        QVector<BCExchange> exchanges;

        exchanges.push_back(one);

        bcExchangeInfos.insert(filePath, exchanges);
    }
}

}

///<MG 进行替换

for (QString filePath : bcExchangeInfos.keys())

if (!ReplaceMeshFileBCType(filePath, bcExchangeInfos[filePath])){ return false; }

if (bcExchangeInfos.size() == 1)

    meshFilePath = bcExchangeInfos.keys()[0];

return true;

```

```

}

bool JournalOperation(QString path, ModelData::ModelDataBase* d, QString meshFilePah)
{
    QString templatePath = QApplication::applicationDirPath() + "../Template/template.jou";
    QFile file(templatePath);

    QString toPath = path + "/script.jou";
    if (file.exists(toPath)){ file.remove(toPath); }

    QString toOutputPath = path + "/Result/Result.cgns";
    file.copy(templatePath, toPath);

    ///<MG 向数据中添加参数

    DataProperty::ParameterBase * paraMshPath = d->getParameterByName("MshPath");
    if (!paraMshPath)
    {
        paraMshPath = d->appendParameter(DataProperty::Para_String);
        paraMshPath->setDescribe("MshPath");
    }

    paraMshPath->setValueFromString(meshFilePah);

    DataProperty::ParameterBase * paraOutputPath =
d->getParameterByName("OutputPath");
    if (!paraOutputPath)
    {
        paraOutputPath = d->appendParameter(DataProperty::Para_String);
        paraOutputPath->setDescribe("OutputPath");
    }

    paraOutputPath->setValueFromString(toOutputPath);

    ///<MG 对脚本中的值进行替换
    IO::TempalteReplacer replcer(d);
    replcer.appendFile(toPath);
    replcer.replace();

    ///<MG 删除添加的参数

```

```

d->removeParameter(paraMshPath);

d->removeParameter(paraOutputPath);

///

```

其中 BCOperation 函数用于将传入的 msh 文件按照当前 FastCAE2.0 中设置的边界条件进行重写。JournalOperation 函数用于 Jou 脚本中参数的替换。具有 C++ 基础的用户可自行研读这部分代码内容，可以发现 jou 脚本替换是通过替换脚本中**%参数名称%**为参数值来实现的。

4.2 脚本修改

Fluent 的 jou 脚本采用了 SScheme 语言，我们可以看到在我们使用的录制 jou 脚本中：

```

1 (cx-gui-do cx-activate-item."MenuBar*ReadSubMenu*Mesh...")
2 (cx-gui-do cx-set-file-dialog-entries."Select.File".('E:/FluentCaseFiles/0814.msh")."Mesh.Files. (*.msh;*.MSH)")
3 (cx-use-window-id.1)
4 (cx-set-camera-relative.'(-1.01367.1.08424.-0.414614).'(0.0.0).'(0.0506475.-0.0791588.-0.386639).0.0)
5 (cx-gui-do cx-activate-item."Ribbon*Frame1*Frame2 (Setting:Up-Domain)*Table1*Table3 (Mesh)*PushButton1 (Display)")
6 (cx-gui-do cx-activate-item."Mesh.Display*PanelButtons*PushButton1 (OK)")
7 (cx-gui-do cx-activate-item."Mesh.Display*PanelButtons*PushButton2 (Cancel)")
8
9 (cx-gui-do cx-activate-item."General*Table1*ButtonBox1 (Mesh)*PushButton3 (Check)")
10 (cx-gui-do cx-activate-item."General*Table1*ButtonBox1 (Mesh)*PushButton1 (Scale)")
11 (cx-gui-do cx-set-list-selections."Scale.Mesh*Table1*DropDownList3 (View.Length.Unit.In)".'(.2))
12 (cx-gui-do cx-activate-item."Scale.Mesh*Table1*DropDownList3 (View.Length.Unit.In)")
13 (cx-gui-do cx-set-list-selections."Scale.Mesh*Table1*Table2 (Scaling)*DropDownList2 (Mesh.Was.Created.In)".'(.3))
14 (cx-gui-do cx-activate-item."Scale.Mesh*Table1*Table2 (Scaling)*DropDownList2 (Mesh.Was.Created.In)")
15 (cx-gui-do cx-activate-item."Scale.Mesh*PanelButtons*PushButton1 (Close)")
16 (cx-gui-do cx-set-list-tree-selections."NavigationPane*List_Tree1".(list."Setup|Models"))
17 (cx-gui-do cx-set-list-tree-selections."NavigationPane*List_Tree1".(list."Setup|Models"))
18 (cx-gui-do cx-activate-item."NavigationPane*List_Tree1")
19 (cx-gui-do cx-activate-item."Models*Table1*PushButton2 (Edit)")
20 (cx-gui-do cx-set-toggle-button2."Multiphase.Model*Table1*ToggleBox1 (Model)*Mixture".#t)
21 (cx-gui-do cx-activate-item."Multiphase.Model*Table1*ToggleBox1 (Model)*Mixture")
22 (cx-gui-do cx-activate-item."Multiphase.Model*PanelButtons*PushButton1 (OK)")
23 (cx-gui-do cx-set-list-selections."Models*Table1*List1 (Models)".'(.2))
24 (cx-gui-do cx-activate-item."Models*Table1*List1 (Models)")
25 (cx-gui-do cx-activate-item."Models*Table1*PushButton2 (Edit)")
26 (cx-gui-do cx-set-toggle-button2."Viscous.Model*Table1*ToggleBox1 (Model)*k-omega(2 eqn)".#t)
27 (cx-gui-do cx-activate-item."Viscous.Model*Table1*ToggleBox1 (Model)*k-omega(2 eqn)")
28 (cx-gui-do cx-activate-item."Viscous.Model*PanelButtons*PushButton1 (OK)")
29 (cx-gui-do cx-set-list-tree-selections."NavigationPane*List_Tree1".(list."Setup|Materials"))
30 (cx-gui-do cx-set-list-tree-selections."NavigationPane*List_Tree1".(list."Setup|Materials"))
31 (cx-gui-do cx-activate-item."NavigationPane*List_Tree1")
32 (cx-gui-do cx-set-list-tree-selections."NavigationPane*List_Tree1".(list."Setup|Materials"))
33 (cx-gui-do cx-set-list-selections."Materials*Table1*List1 (Materials)".'(.0))

```

原始 jou 脚本文件

第 1 、 2 行表示打开 Fluent 菜单项 Read Mesh 读取“E:/FluentCaseFiles/0814.msh”网格文件。

第 26 、 27 行表示 Viscous Model 项勾选为 k-omega(2 eqn)。

由于我们需要与 FastCAE2.0 进行对接，将我们需要在 FastCAE2.0 中设置的参数在 jou 文件中进行替换，这里我们采用%参数名称%形式对 jou 脚本中的参数值进行替换，以第 26 、 27 行为例，我们要将第 26 、 27 行的值 k-omega(2 eqn)替换为%Viscous Model%，其中 Viscous Model 为该参数的名称，k-omega(2 eqn)为该参数当前的设置值。对导入网格的路径采用%MshPath%形式对 jou 脚本中的导入网格路径进行替换，对导出结果文件采用%OutputPath%形式对 jou 脚本中导出结果文件路径进行替换。

经过替换后的前 33 行内容如下图所示，整个替换前的脚本和替换后的脚本可以通过下载获取。用户也可以根据需求录制自己使用的脚本，并将结果文件和网格文件以及想要在 FastCAE2.0 集成软件中进行设置的参数进行替换，完成自己需要的 jou 脚本文件。

```

1 (cx-gui-do-cx-activate-item."MenuBar*ReadSubMenu*Mesh...")
2 (cx-gui-do-cx-set-file-dialog-entries."Select.File".'%MshPath%').Mesh.Files.(*.msh*.MSH*)
3 (cx-use-window-id.1)
4 (cx-set-camera-relative.'(-1.01367.1.08424.-0.414614)'.(0.0.0)'.(0.0506475.-0.0791588.-0.386639).0.0)
5 (cx-gui-do-cx-activate-item."Ribbon*Frame1*Frame2(Setting.Up.Domain)*Table1*Table3(Mesh)*PushButton1(Display)")
6 (cx-gui-do-cx-activate-item."Mesh.Display*PanelButtons*PushButton1(OK)")
7 (cx-gui-do-cx-activate-item."Mesh.Display*PanelButtons*PushButton2(Cancel)")
8
9 (cx-gui-do-cx-activate-item."General*Table1*ButtonBox1(Mesh)*PushButton3(Check)")
10 (cx-gui-do-cx-activate-item."General*Table1*ButtonBox1(Mesh)*PushButton1(Scale)")
11 (cx-gui-do-cx-set-list-selections."Scale.Mesh*Table1*DropDownList3(View.Length.Unit.In)".'(.2)')
12 (cx-gui-do-cx-activate-item."Scale.Mesh*Table1*DropDownList3(View.Length.Unit.In)")
13 (cx-gui-do-cx-set-list-selections."Scale.Mesh*Table1*Table2(Scaling)*DropDownList2(Mesh.Was.Created.In)".'(.3)')
14 (cx-gui-do-cx-activate-item."Scale.Mesh*Table1*Table2(Scaling)*DropDownList2(Mesh.Was.Created.In)")
15 (cx-gui-do-cx-activate-item."Scale.Mesh*PanelButtons*PushButton1(Close)")
16 (cx-gui-do-cx-set-list-tree-selections."NavigationPane*List_Tree1.(list."Setup|Models)")
17 (cx-gui-do-cx-set-list-tree-selections."NavigationPane*List_Tree1.(list."Setup|Models)")
18 (cx-gui-do-cx-activate-item."NavigationPane*List_Tree1")
19 (cx-gui-do-cx-activate-item."Models*Table1*PushButton2(Edit)")
20 (cx-gui-do-cx-set-toggle-button2."Multiphase.Model*Table1*ToggleBox1(Model)*Mixture".#t)
21 (cx-gui-do-cx-activate-item."Multiphase.Model*Table1*ToggleBox1(Model)*Mixture")
22 (cx-gui-do-cx-activate-item."Multiphase.Model*PanelButtons*PushButton1(OK)")
23 (cx-gui-do-cx-set-list-selections."Models*Table1*List1(Models)".'(.2)')
24 (cx-gui-do-cx-activate-item."Models*Table1*List1(Models)")
25 (cx-gui-do-cx-activate-item."Models*Table1*PushButton2(Edit)")
26 (cx-gui-do-cx-set-toggle-button2."Viscous.Model*Table1*ToggleBox1(Model)*%Viscous.Model%".#t)
27 (cx-gui-do-cx-activate-item."Viscous.Model*Table1*ToggleBox1(Model)*%Viscous.Model%")
28 (cx-gui-do-cx-activate-item."Viscous.Model*PanelButtons*PushButton1(OK)")
29 (cx-gui-do-cx-set-list-tree-selections."NavigationPane*List_Tree1.(list."Setup|Materials)")
30 (cx-gui-do-cx-set-list-tree-selections."NavigationPane*List_Tree1.(list."Setup|Materials)")
31 (cx-gui-do-cx-activate-item."NavigationPane*List_Tree1")
32 (cx-gui-do-cx-set-list-tree-selections."NavigationPane*List_Tree1.(list."Setup|Materials)")
33 (cx-gui-do-cx-set-list-selections."Materials*Table1*List1(Materials)".'(.0)')
34 (cx-gui-do-cx-activate-item."Materials*Table1*List1(Materials)")

```

替换后的 jou 脚本文件

在完成脚本修改后，我们找到 FastCAE 2.0 应用所在路径的上级目录下创建名为 Template 的文件夹，此时 Template 文件夹所在的文件夹如下图所示。

名称	修改日期	类型	大小
 bin	2020/2/12 15:29	文件夹	
 bin_d	2020/2/12 15:14	文件夹	
 ConfigFiles	2020/2/12 15:19	文件夹	
 Install	2020/2/13 14:13	文件夹	
 python37	2020/2/6 11:11	文件夹	
 Template	2020/2/13 14:35	文件夹	
 WorkingDir	2020/2/7 9:57	文件夹	

案例文件目录结构

我们将替换完的脚本文件命名为 template.jou，并将其拷贝到 Template 文件夹下。从 Fluent 插件开发中的源码部分我们可以看出，每次求解的时候我们会将 Template 文件夹下的 template.jou 文件拷贝到算例所在文件夹，重命名为 script.jou，并将脚本文件中的需要替换的参数通过程序完成替换。

4.3 软件定制

在软件定制中，我们先进行通用项的定制，包括：软件名称、公司信息、网格功能、几

何功能，及模型树等。

然后进行求解器需要参数的定制，我们将按照需要替换的参数来进行软件参数的定制，这里我们需要替换的参数有：

Viscous Model、Oil Droplet Diameter、Velocity Magnitude、Volume Fraction、Flow Rate Weighting(Up)、Flow Rate Weighting(In)、Iterations、Absolute Criteria。其中按照类型将前六个参数归类为仿真参数，将后两个参数归类为求解参数。

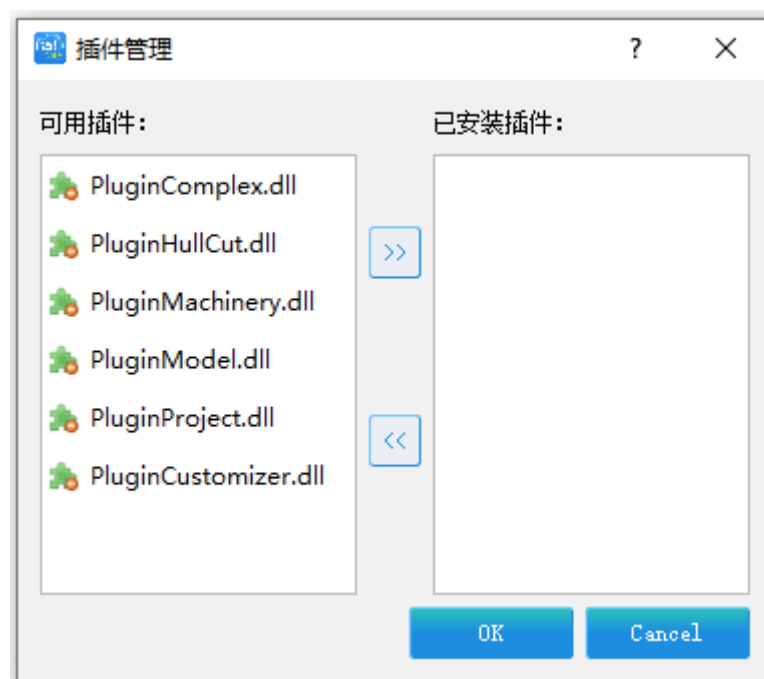
在完成求解器需要的参数定制后，我们需要对后处理文件进行设置。

在完成后处理文件设置后，我们需要对求解器进行设置。其中一些设置工作可以穿插进行，上面列举的定制流程是相对通用的一个定制流程，用户可根据自身操作习惯完成所需的定制工作。

下面将详细的对定制过程进行说明。

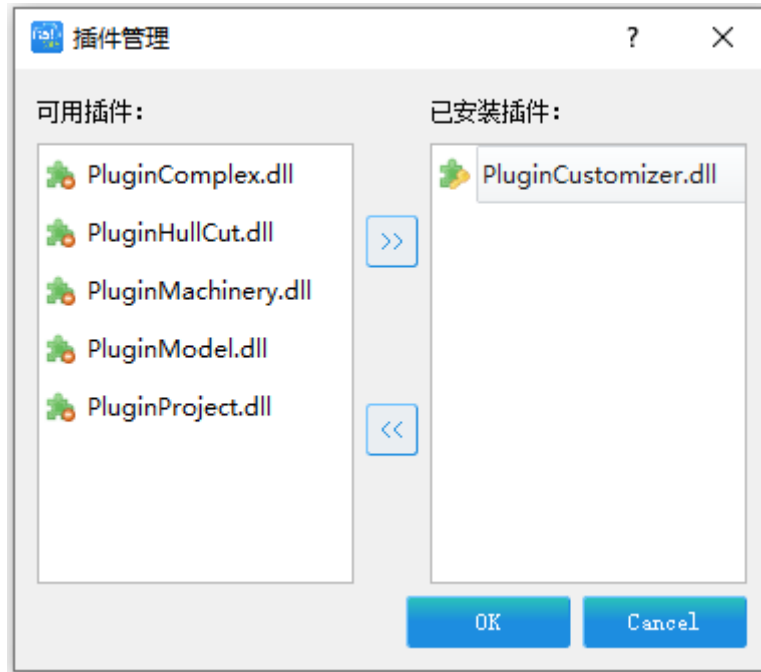
- 启动 FastCAE 2.0 版本软件加载 PluginCustomizer.dll 插件，开始进行软件常规项定制。

1.点击【插件】-【插件管理】。



插件管理

2.在可用插件列表选中 PluginCustomizer.dll 插件，点击按键。



加载插件

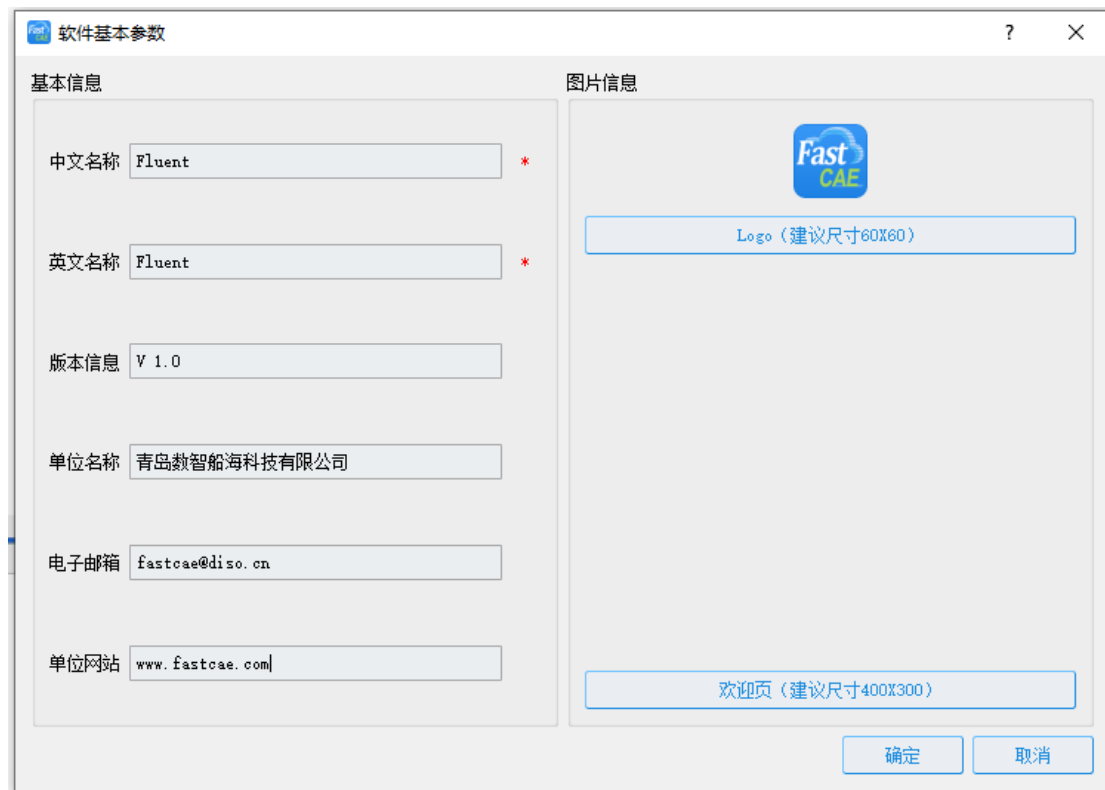
3. 点击按键，这样 PluginCustomizer 插件将会被 FastCAE 软件装载。软件菜单栏上将会出现定制菜单项。



4. 点击【定制】-【开始定制】，软件标题栏上名称后面会有“---定制中”字样，表示此时我们处于定制模式。

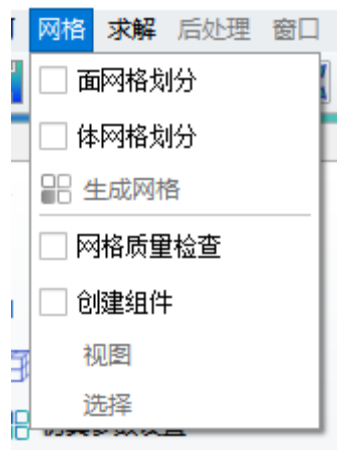


5. 点击【帮助】-【关于】，软件将弹出软件基本参数设置界面，这里面我们按照下图输入我们想要填入的软件基本信息。然后点击按键，完成软件基本信息的录入。



软件基本参数设置

6. 由于我们 Fluent 案例不需要网格划分、网格质量检查和创建组件功能，为保证最终软件最简化，我们定制去掉这部分功能。点击【网格】，在菜单中将【面网格划分】、【体网格划分】、【网格质量检查】、【创建网格组件】菜单前面的勾去掉。

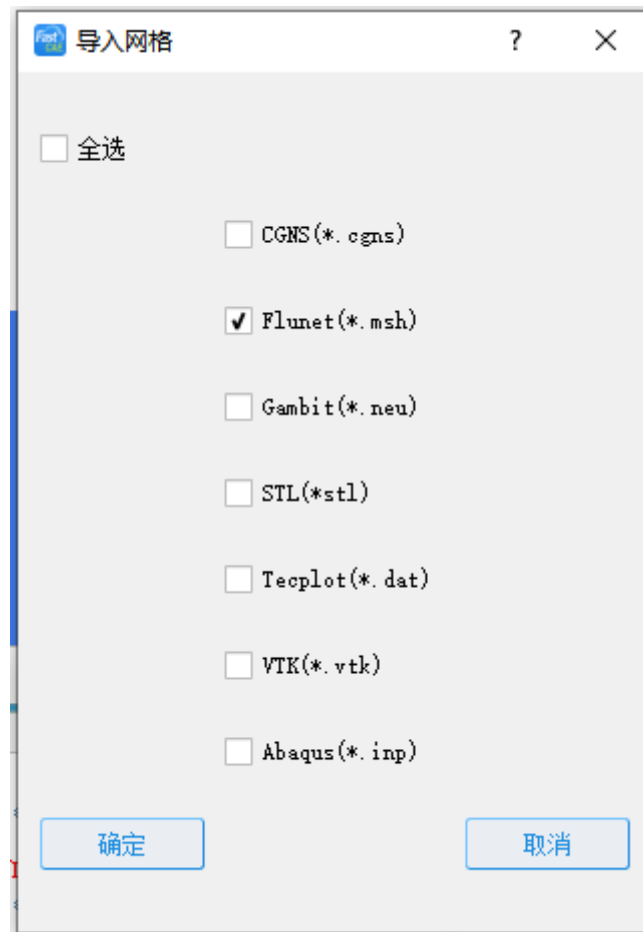


网格功能设置

7. 我们的 Fluent 案例导入的网格类型为.msh 文件，为保证最终软件最简化，我们定制软件只需要支持.msh 格式网格文件。点击【文件】-【导入网格】，软件将弹出软件导入网格项配置界面，在界面中我们只保留【Fluent(*.msh)】项，将【CGNS(*.cgns)】、【Gambit(*.neu)】、【STL(*.stl)】、【Tecplot(*.dat)】、【VTK(*.vtk)】、【Abaqus(*.inp)】项的勾取消掉。然后点击

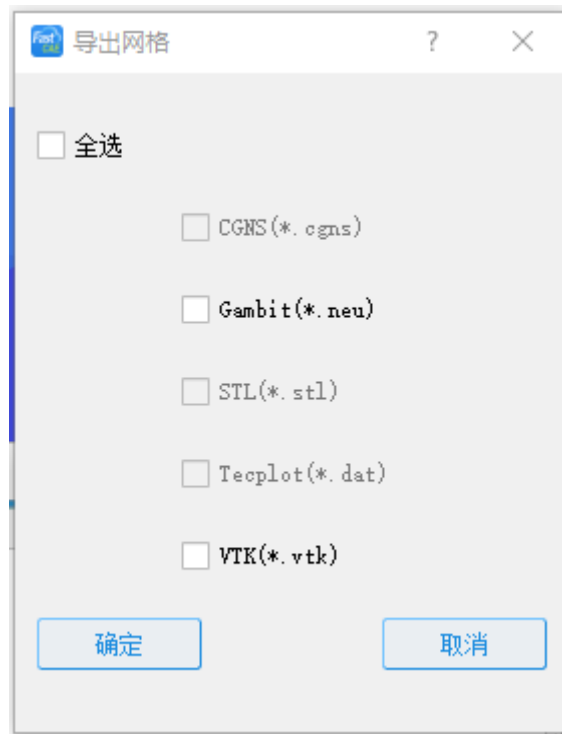
确定

按键完成导入网格功能更的定制。



导入网格格式设置

8.我们的案例不需要导出网格功能，为保证最终软件最简化，我们定制将该部分功能去掉。点击【文件】-【导出网格】，软件将弹出导出网格功能定制界面，我们将全选前面的勾选的勾去掉，确保勾选项处于反选状态，点击按键，完成导出网格功能的定制。



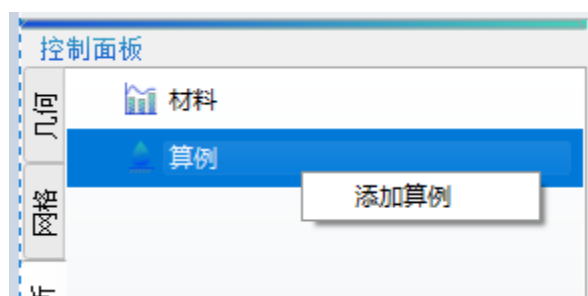
导出网格格式设置

9. 下面我们将进行模型树的定制，将控制面板切换到计算分析分页。



计算分析页

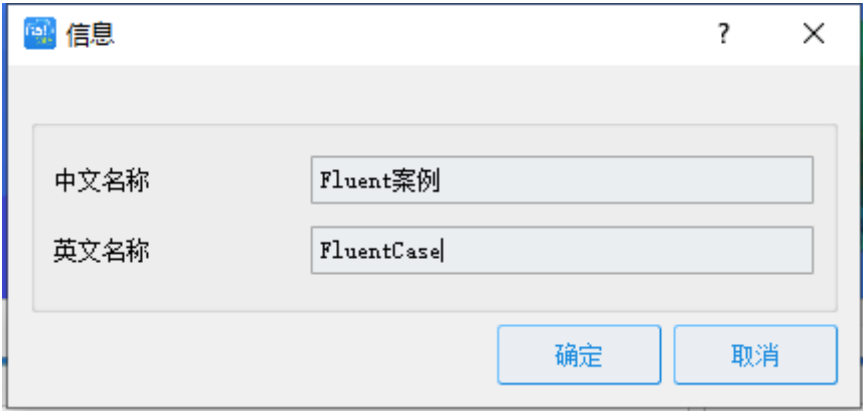
10.鼠标左键选中算例节点，点击鼠标右键，点击弹出右键菜单的【添加算例】菜单项。



添加算例

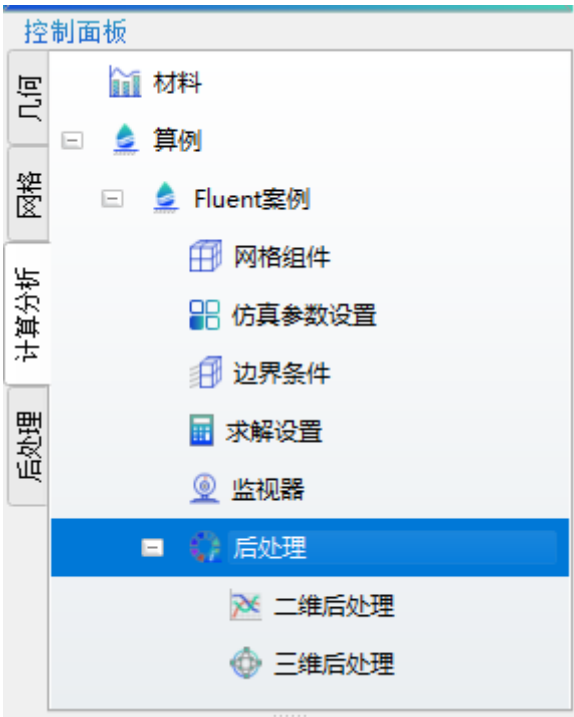
11.软件将弹出添加算例对话框，我们按照下图填写算例的中文名称和英文名称。点击

按键完成算例名称信息的录入。



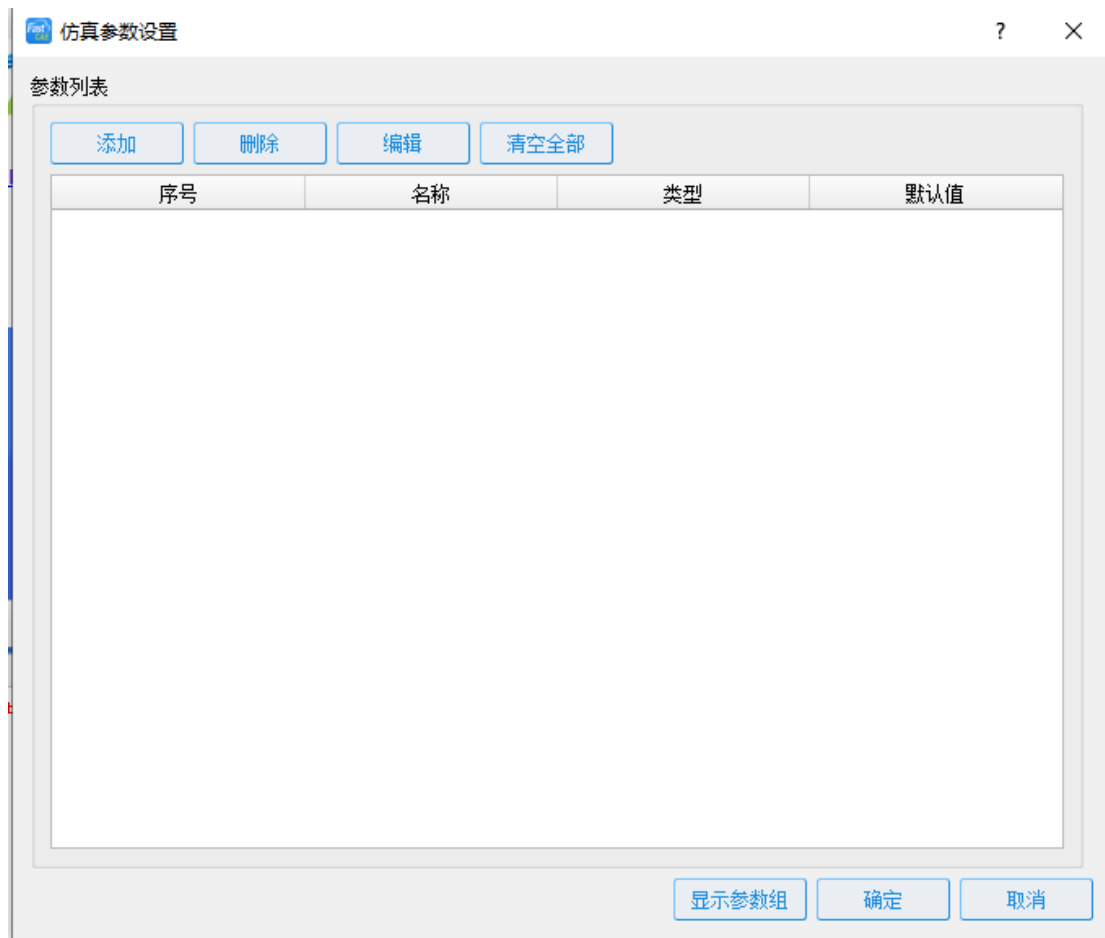
添加算例信息

12.此时，【控制面板】 - 【算例】节点下面将会有一颗用于定制的算例树。



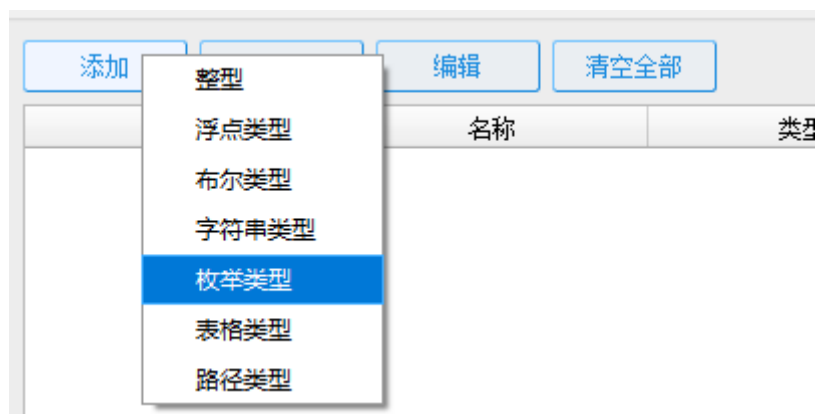
算例树

13.鼠标左键双击算例树下的【仿真参数设置】节点，软件将弹出仿真参数设置界面。



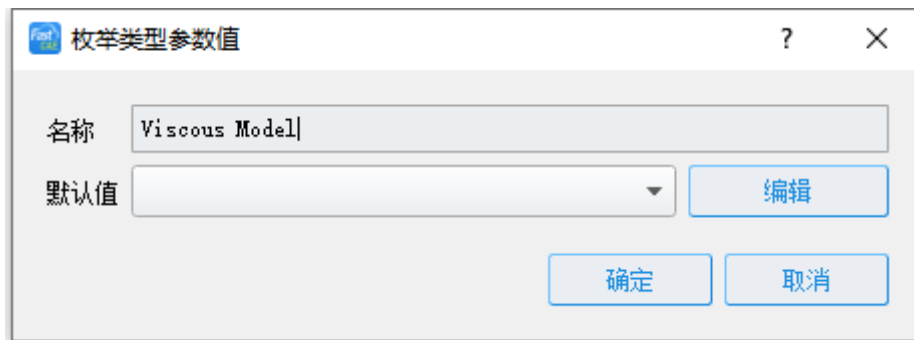
仿真参数设置

14.鼠标左键点击仿真参数设置界面中的添加按钮，弹出添加参数类型选择的菜单。



添加枚举参数

15.在弹出的添加参数类型选择菜单中，鼠标左键点击【枚举类型】菜单项，软件将弹出添加枚举参数对话框。在枚举参数对话框名称处填入 Viscous Model，即该枚举项的名称。



设定枚举参数名称

16. 点击添加枚举类型参数对话框中的按键，软件将弹出枚举值编辑对话框。



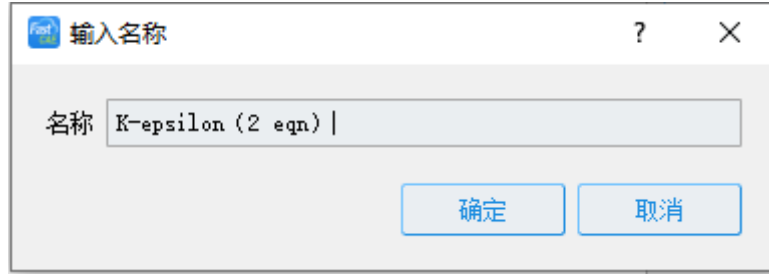
编辑枚举参数值

17. 点击枚举值编辑对话框中的按键，软件将弹出添加枚举单个值的对话框，在对话框中填入如下图的参数后点击按键，完成枚举项【K-omega(2 eqn)】项的添加。**注意：涉及到中英文具有差异的符号进行输入的时候，要使用英文符号进行输入。这里的括号如果使用中文括号，Fluent 将不识别。**



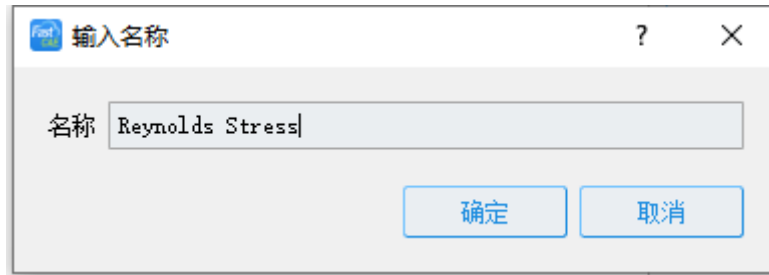
添加枚举项 1

18. 点击枚举值编辑对话框中的按键，软件将弹出添加枚举单个值的对话框，在对话框中填入如下图的枚举参数值后点击按键，完成枚举项【K-epsilon(2 eqn)】项的添加。



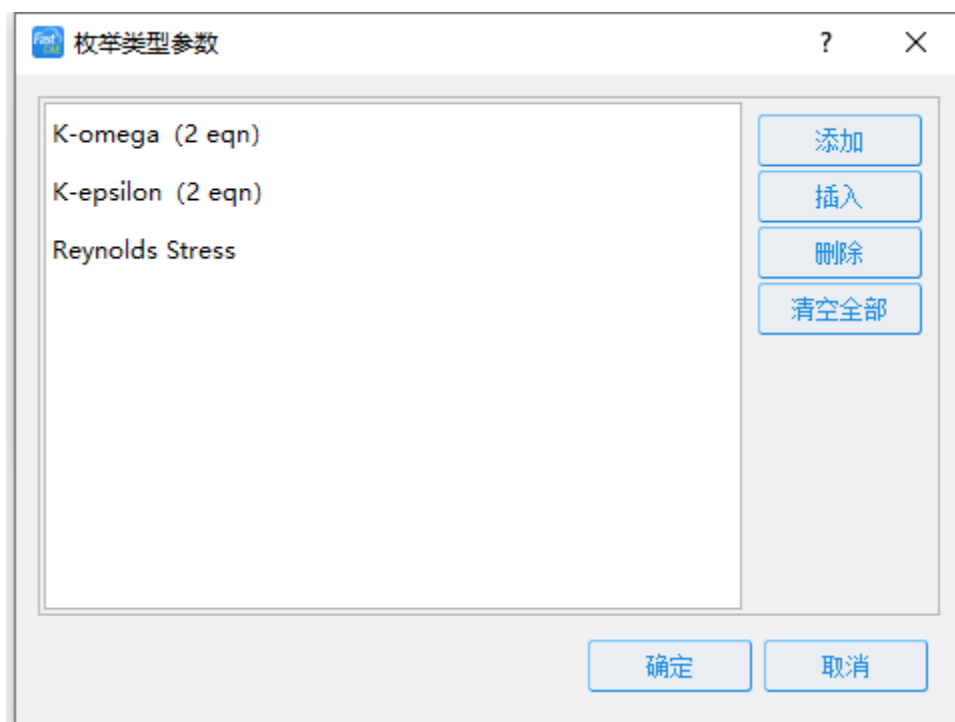
添加枚举项 2

19. 点击枚举值编辑对话框中的按键，软件将弹出添加枚举单个值的对话框，在对话框中填入如下图的枚举参数值后点击按键，完成枚举项【Reynolds Stress】项的添加。



添加枚举项 3

20. 此时枚举值编辑对话框的界面如下图所示。点击界面中的按键，完成枚举项的添加工作。



添加枚举参数效果

21.此时的枚举参数对话框如下图所示，击界面中的按键，完成枚举参数的添加工作。



设置默认枚举值

22.鼠标左键点击仿真参数设置界面中的添加按键，弹出添加参数类型选择的菜单。点击菜单中的【浮点参数】菜单项，软件将弹出添加浮点参数界面。按照下图所示内容在添加浮点参数界面添加信息，点击



按键，完成 Oil Droplet Diameter 浮点参数的添加。

The screenshot shows a dialog box titled '浮点类型参数' (Floating Point Parameter Type) with a 'Fast' logo. It contains the following fields:

名称	单位	浮点精度	默认值	最小值	最大值
Oil Droplet Diameter	M	6	0.0005	0.00001	0.999

At the bottom right, there are two buttons: '确定' (OK) and '取消' (Cancel).

添加浮点参数 1

23.鼠标左键点击仿真参数设置界面中的添加按键，弹出添加参数类型选择的菜单。点击菜单中的【浮点参数】菜单项，软件将弹出添加浮点参数界面。按照下图所示内容在添加浮点参数界面添加信息，点击按键，完成 Velocity Magnitude 浮点参数的添加。

The screenshot shows a dialog box titled '浮点类型参数' (Floating Point Parameter Type) with a 'Fast' logo. It contains the following fields:

名称	单位	浮点精度	默认值	最小值	最大值
Velocity Magnitude	M/S	1	10	0.1	1000

At the bottom right, there are two buttons: '确定' (OK) and '取消' (Cancel).

添加浮点参数 2

24.鼠标左键点击仿真参数设置界面中的添加按键，弹出添加参数类型选择的菜单。点击菜单中的【浮点参数】菜单项，软件将弹出添加浮点参数界面。按照下图所示内容在添加浮点参数界面添加信息，点击按键，完成 Volume Fraction 浮点参数的添加。

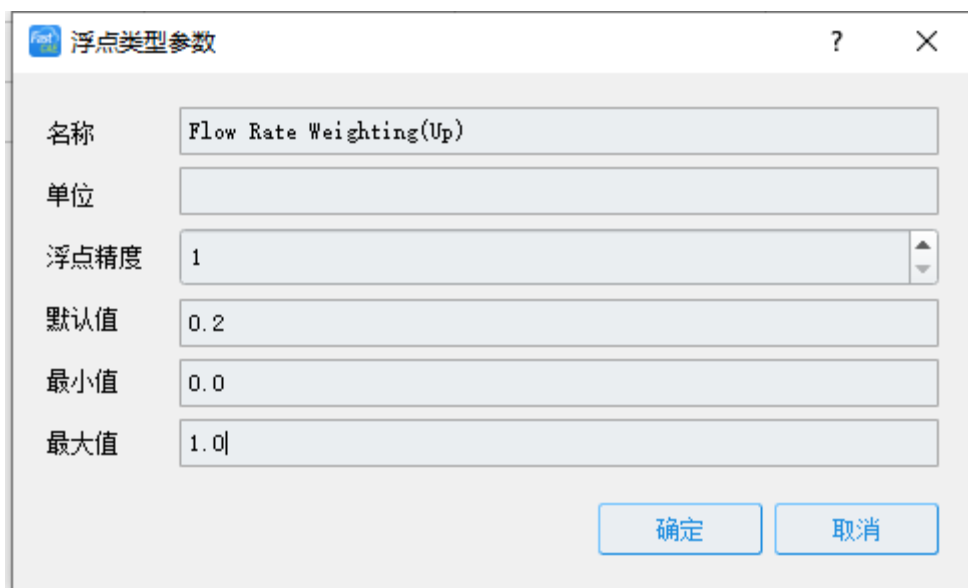


名称	Volume Fraction
单位	
浮点精度	2
默认值	0.02
最小值	0.0
最大值	1.0

确定 取消

添加浮点参数 3

25.鼠标左键点击仿真参数设置界面中的添加按键，弹出添加参数类型选择的菜单。点击菜单中的【浮点参数】菜单项，软件将弹出添加浮点参数界面。按照下图所示内容在添加浮点参数界面添加信息，点击按键，完成 Flow Rate Weighting(Up)浮点参数的添加。



名称	Flow Rate Weighting(Up)
单位	
浮点精度	1
默认值	0.2
最小值	0.0
最大值	1.0

确定 取消

添加浮点参数 4

26.鼠标左键点击仿真参数设置界面中的添加按键，弹出添加参数类型选择的菜单。点击菜单中的【浮点参数】菜单项，软件将弹出添加浮点参数界面。按照下图所示内容在添加浮点参数界面添加信息，点击按键，完成 Flow Rate Weighting(In)浮点参数的添加。

浮点类型参数

名称: Flow Rate Weighting(In)

单位:

浮点精度: 1

默认值: 0.8

最小值: 0.0

最大值: 1.0

确定 取消

添加浮点参数 5

27.此时我们仿真参数设置界面如下图所示，点击按键后，完成仿真参数的设置。

仿真参数设置

参数列表

添加 删除 编辑 清空全部

序号	名称	类型	默认值
1	Viscous Model	Selectable	K-omega (2 eqn)
2	Oil Droplet Diameter	Double	0.000500
3	Velocity Magnitude	Double	10.0
4	Volume Fraction	Double	0.02
5	Flow Rate Weighting(Up)	Double	0.2
6	Flow Rate Weighting(In)	Double	0.8

显示参数组 确定 取消

仿真参数设置完成

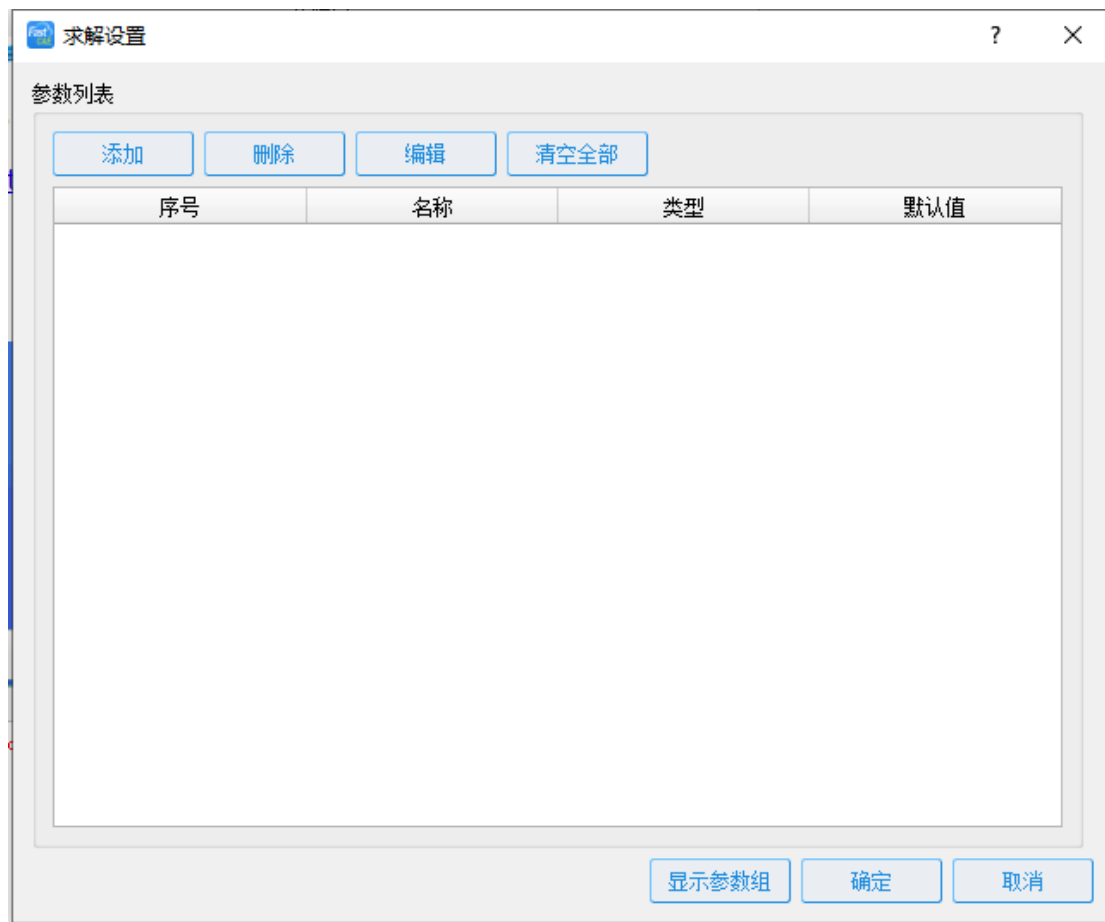
28.鼠标左键双击【控制面板】-【算例】节点下的【边界条件】节点，软件将弹出边界

条件编辑对话框，由于我们的 Fluent 算例只使用到【入口】、【出口】、【壁面】三种边界，因此我们按照下图所示对软件边界条件进行勾选定制。然后点击按键，完成边界条件定制。



边界条件设置

29.鼠标左键双击【控制面板】-【算例】节点下的【求解设置】节点，软件将弹出求解参数设置界面。



求解参数设置

30.鼠标左键点击求解参数设置界面中的添加按键，弹出添加参数类型选择的菜单。



添加整形求解参数

31.点击菜单中的【整型参数】菜单项，软件将弹出添加整形参数界面。按照下图所示内容在添加整形参数界面添加信息，点击按键，完成 Iterations 整形参数的添加。

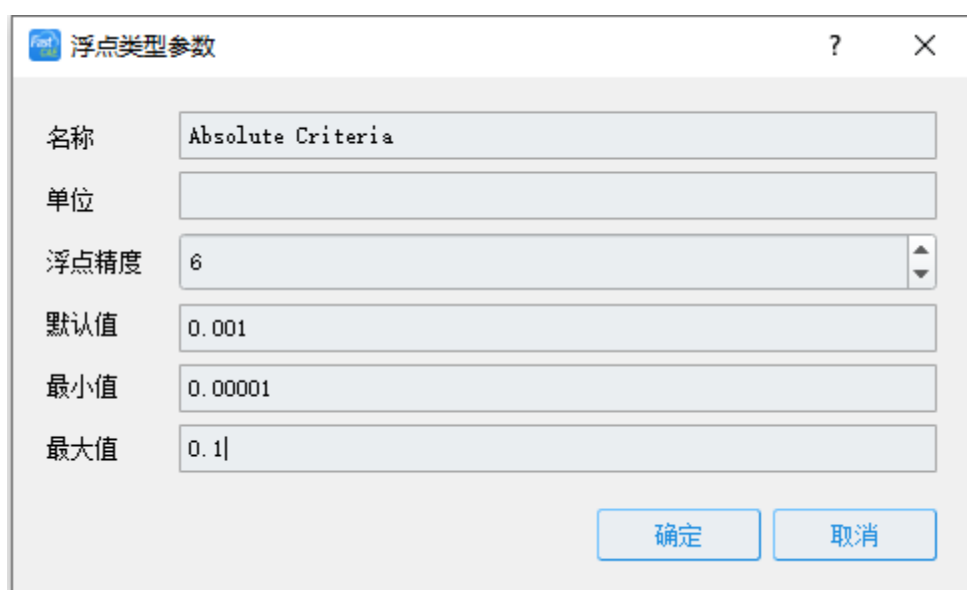


名称	Iterations
单位	
默认值	1000
最小值	1
最大值	99999

确定 取消

设置整形求解参数信息

32.鼠标左键点击求解参数设置界面中的添加按钮，弹出添加参数类型选择的菜单。点击菜单中的【浮点参数】菜单项，软件将弹出添加浮点参数界面。按照下图所示内容在添加浮点参数界面添加信息，点击按钮，完成 Absolute Criteria 浮点参数的添加。



名称	Absolute Criteria
单位	
浮点精度	6
默认值	0.001
最小值	0.00001
最大值	0.1

确定 取消

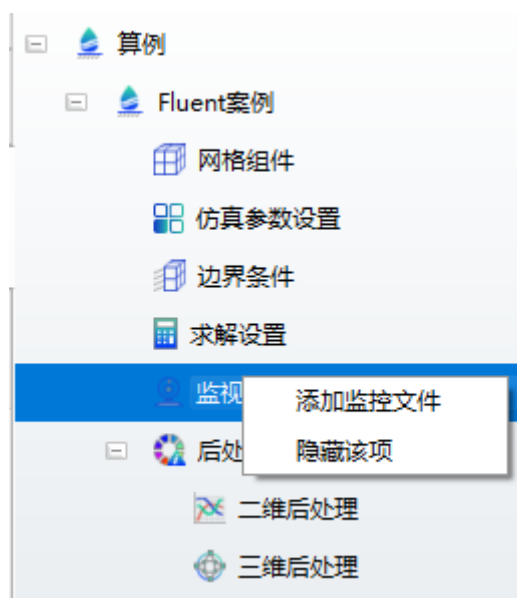
设置浮点参数信息

33.此时求解参数设置界面如下图所示，点击界面中的按钮，完成求解参数的设置。



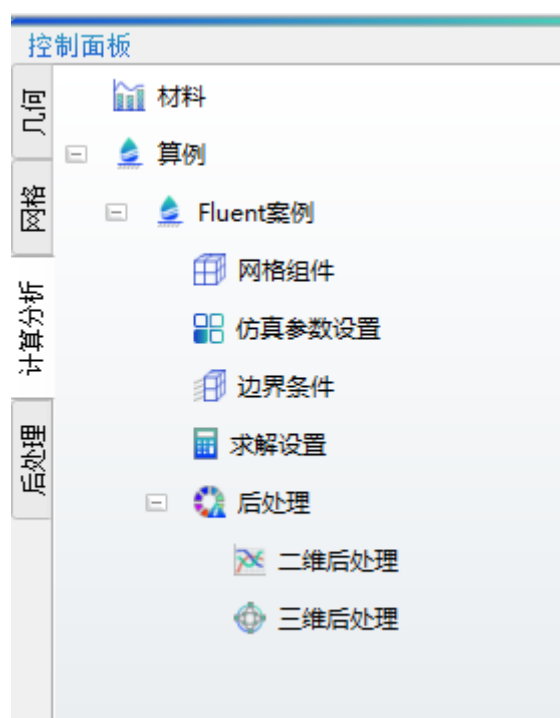
求解信息设置完成

34. Fluent 案例中不要求解过程中进行曲线的绘制。鼠标右键点击【控制面板】-【算例】节点下面的监视器节点，软件弹出【监视器】节点的右键菜单。



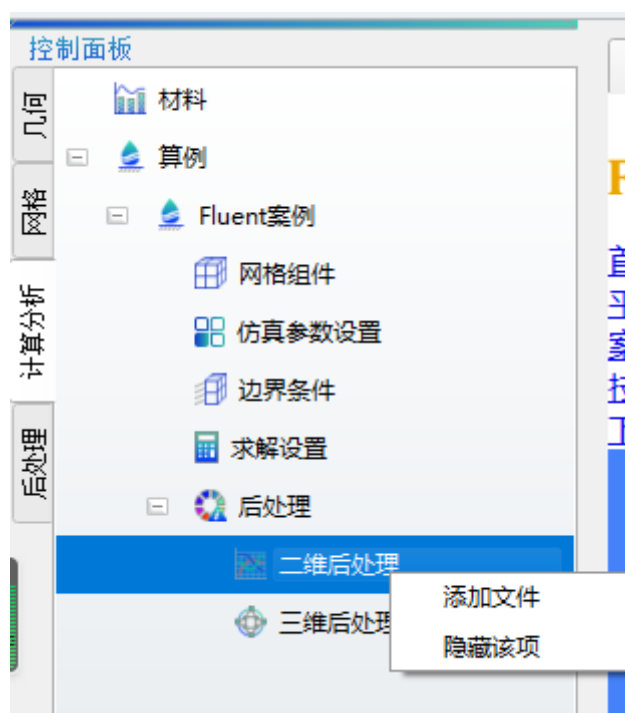
添加监控文件

35.点击【监视器】节点右键菜单中的【隐藏该项】菜单项，将【监视器】节点隐藏。此时的【控制面板】-【算例】树如下图所示。



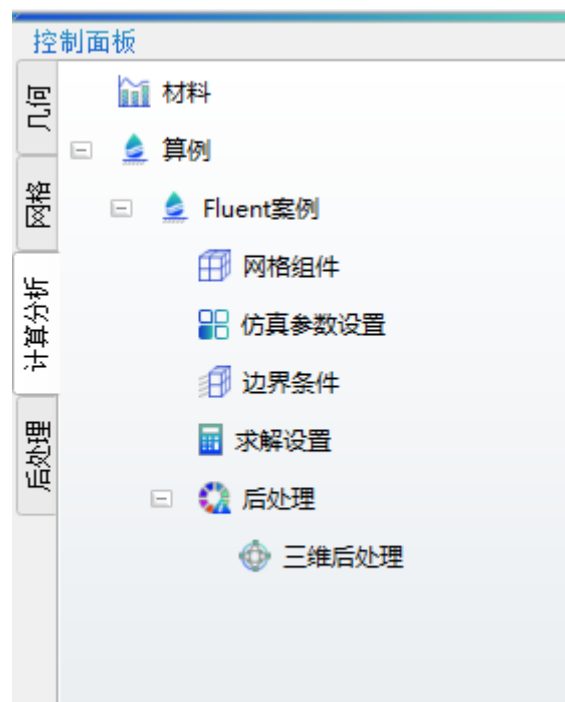
隐藏节点

36.Fluent 案例中不需要进行二维后处理。鼠标右键点击【控制面板】-【算例】节点下面的【二维后处理】节点，软件弹出【二维后处理】节点的右键菜单。



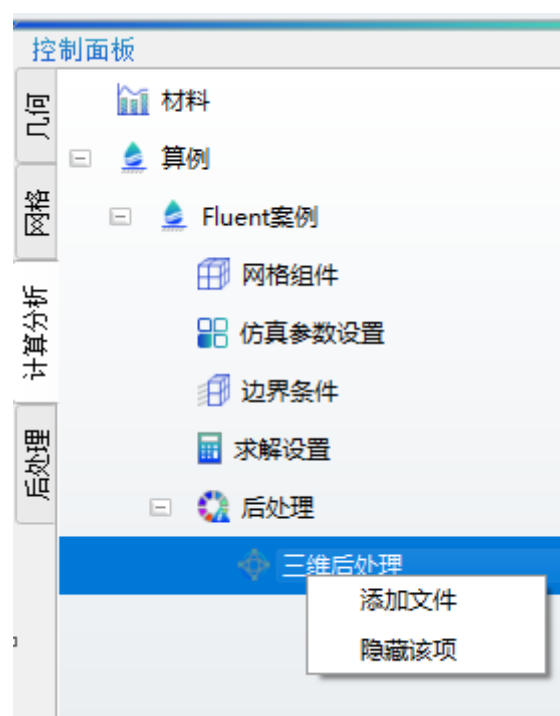
隐藏二维后处理节点

37.点击【二维后处理】节点右键菜单中的【隐藏该项】菜单项，将【二维后处理】节点隐藏。此时的【控制面板】-【算例】树如下图所示。



隐藏节点后的模型树

38.Fluent 案例具有三维后处理功能，因此我们需要对三维后处理进行设置。鼠标左键选中【控制面板】-【算例】下的【三维后处理】节点，点击鼠标右键，软件将弹出【三维后处理】节点右键菜单。



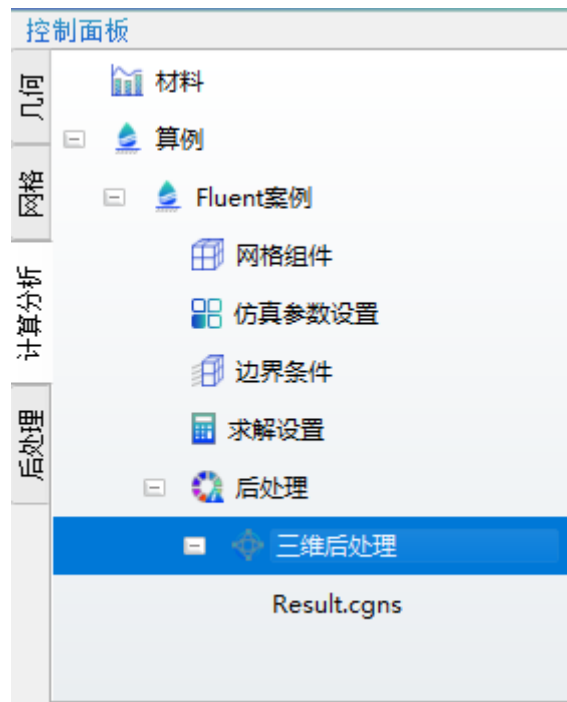
三维后处理添加文件

39.点击【三维后处理】节点右键菜单中的【添加文件】菜单项，软件将弹出添加三维后处理文件名称的对话框。在对话框中输入进行三维后处理的文件的名称，如下图所示。点击按钮完成名为 Result 的三维后处理文件的添加。下拉框中的格式我们选择.cgns。



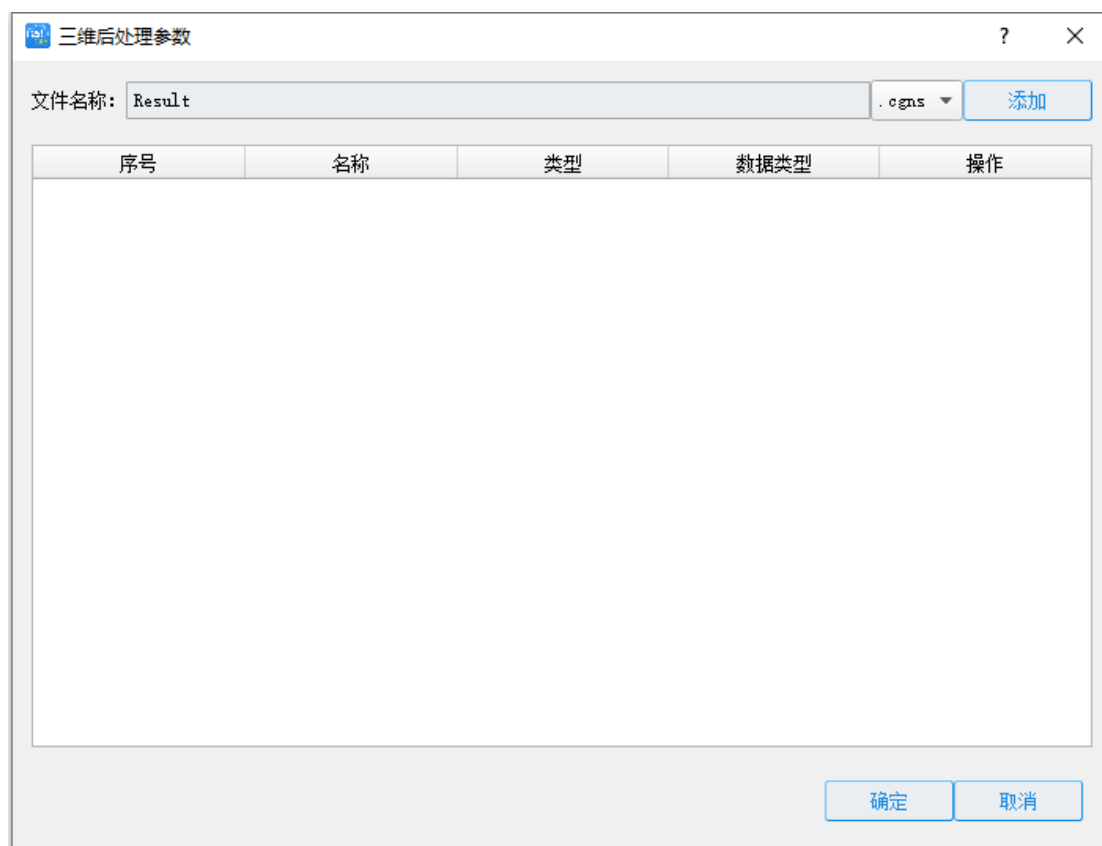
设置三维后处理文件名称

40.此时【控制面板】-【算例】的整个算例树如下图所示。



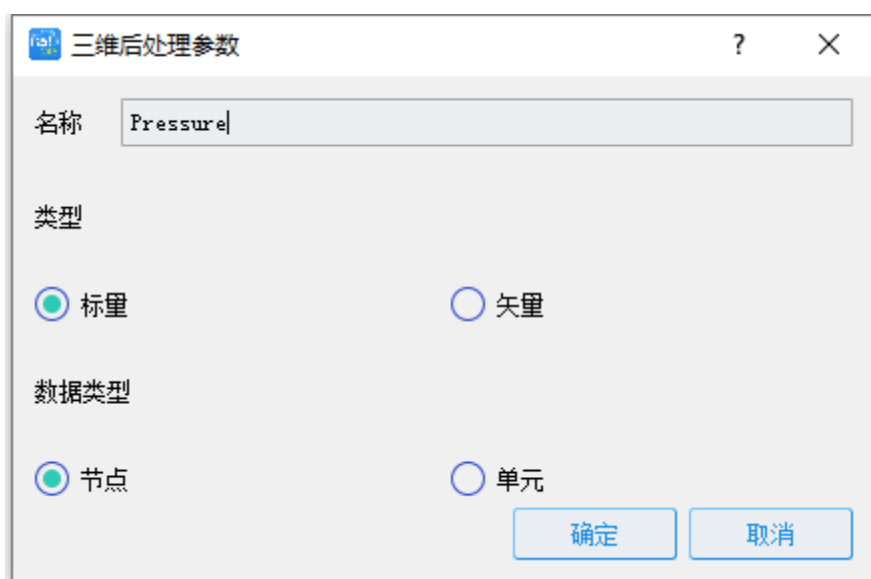
设置三维后处理文件名称后的模型树

41.鼠标左键双击刚添加的【Result.cgns】节点，软件将弹出三维后处理【Result.cgns】节点绘图设置对话框。



三维后处理文件信息设定

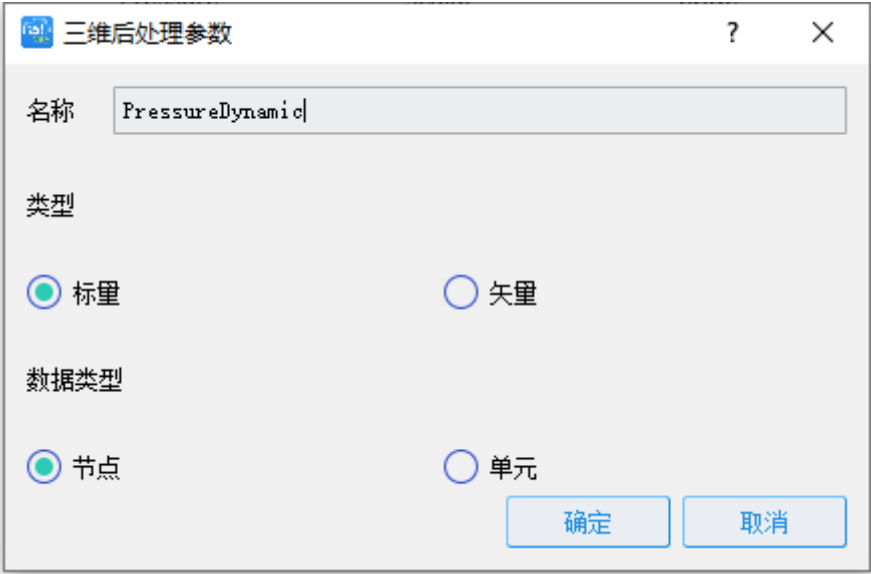
42. 由于 Result.cgns 文件中存有位于节点上的名为 Pressure 的标量值，因此我们点击【Result.cgns】节点上的按键，并按照下图填写勾选，完成后点击界面中的按键完成 Result.cgns 文件中位于节点上的名为 Pressure 的标量信息的添加。



添加 Pressure 节点标量三维后处理参数

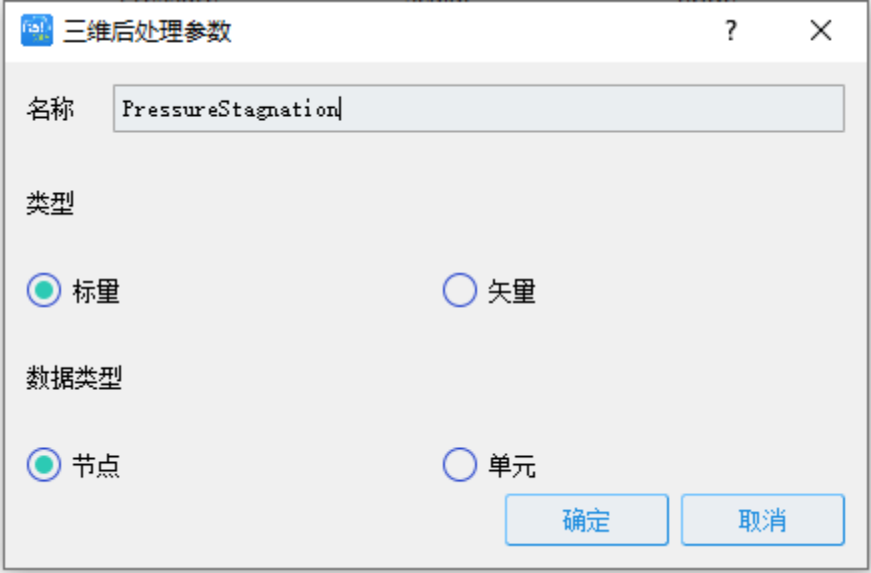
43.由于 Result.cgns 文件中存有位于节点上的名为 PressureDynamic 的标量值, 因此我

们点击【Result.cgns】节点上的按键，并按照下图填写勾选后点击界面中的按键完成 Result.cgns 文件中位于节点上的名为 PressureDynamic 的标量信息的添加。



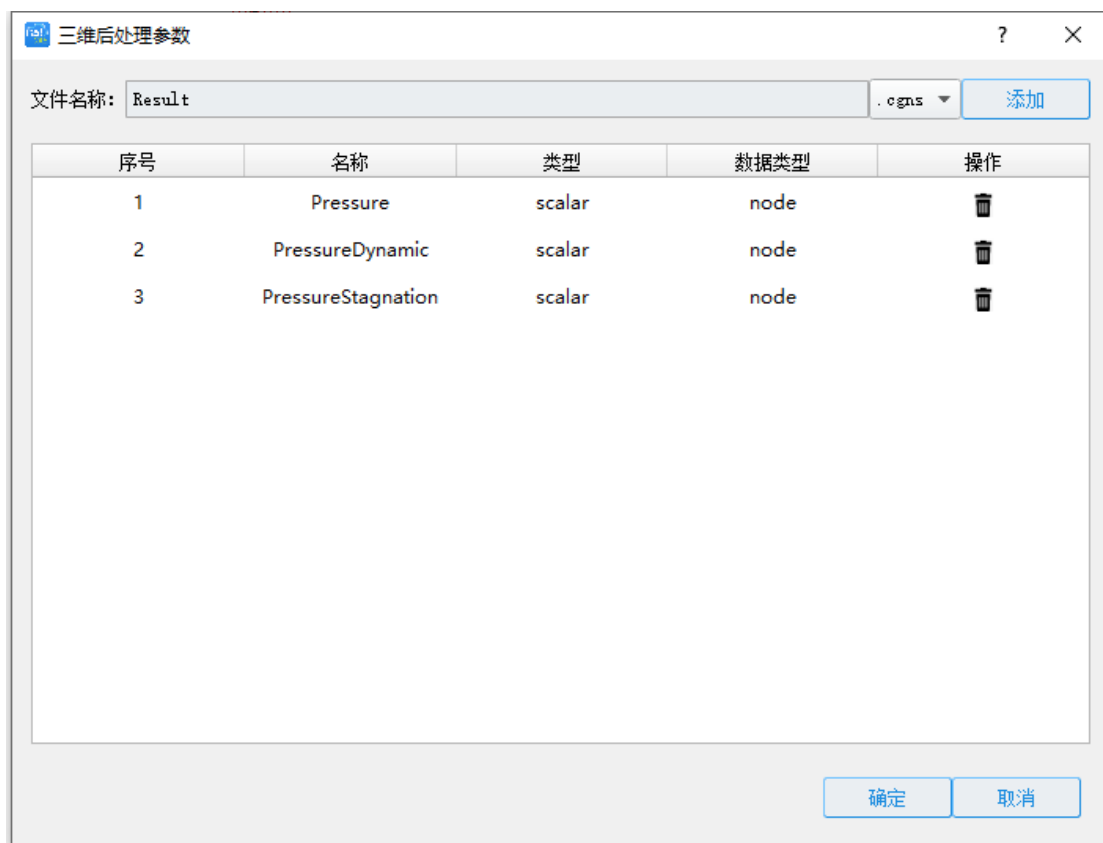
添加 PressureDynamic 节点标量三维后处理参数

44.由于 Result.cgns 文件中存有位于节点上的名为 PressureStagnation 的标量值，因此我们点击【Result.cgns】节点上的按键，并按照下图填写勾选，完成后点击界面中的按键完成 Result.cgns 文件中位于节点上的名为 PressureStagnation 的标量信息的添加。



添加 PressureStagnation 节点标量三维后处理参数

45.此时，三维后处理【Result.cgns】节点绘图设置对话框如下图所示。我们点击界面中的按键完成三维后处理【Result.cgns】节点的绘图信息设置。



三维参数后处理信息设置完成

46. Fluent 案例求解器设置，求解器设置不需要在 PluginCustomizer.dll 插件中，即定制模式下进行设置。用户点击菜单栏【定制】-【完成定制】，将 FastCAE 2.0 软件切换至常规模式，点击菜单栏【求解器】-【求解器管理】项，软件将弹出求解器管理界面。



求解器管理

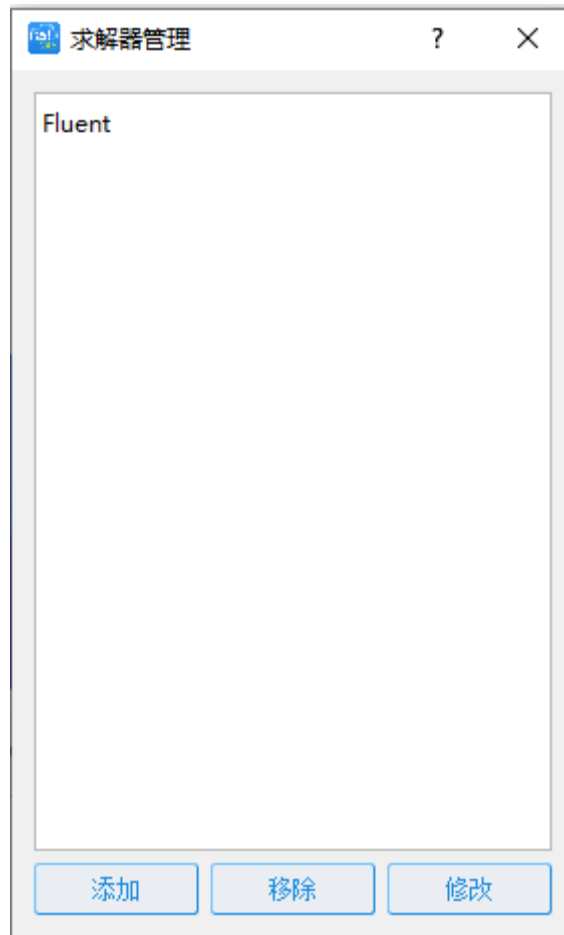
47. 点击求解器管理界面的按键，软件将弹出添加求解器界面，按照下图所示添加求解器信息。需要注意的是这里的求解器路径要选择则 Fluent.exe 可执行文件所在的文件路径。点击界面中的按键完成求解器的添加。



编辑求解器

需要注意的是启动参数中的%modelpath%代表用户创建算例后算例路径。在我们开发的插件中会把替换完的jou脚本取名为script.jou并拷贝到算例路径中，所以启动参数处如此填写。注意：如果我们需要Fluent启动后再后台运行，则启动参数处我们可以添加-hidden选项进行运行，此处没有-hidden是为了方便我们进行调试。添加-hidden后启动参数为3d -hidden -i %modelpath%/script.jou。

48.此时的求解器管理界面如下图所示，点击界面右上角的按钮，完成求解器的设置。



添加求解器完成

4.4 软件使用

1.此时我们已经完成了定制部分工作，我们将开始运行我们所做的程序进行求解工作。
鼠标左键点击【控制面板】-【计算分析】-【算例】节点，点击鼠标右键软件将弹出【算例】节点的右键菜单。



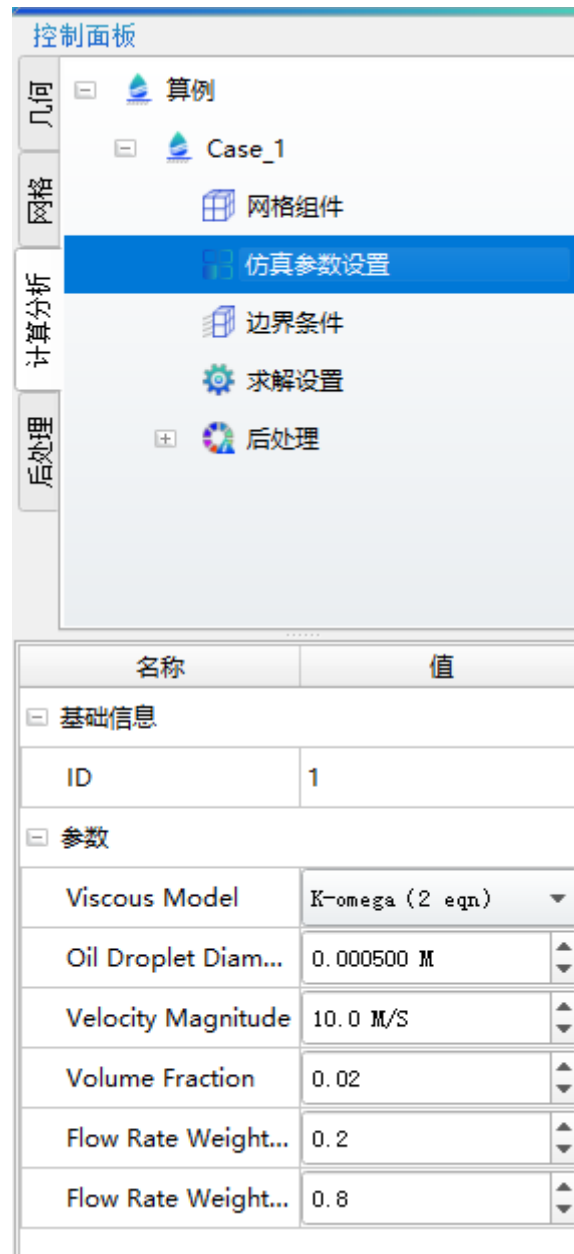
创建算例

2. 点击【算例】节点弹出的右键菜单中的【创建算例】菜单项，软件会弹出创建算例对话框。这里我们使用默认的名称，直接点击按钮完成算例的创建。如果有需要用户可以输入其他的算例名称。



填写算例名称

3. 此时算例下的树形结构如下图所示。点击【算例】下的【仿真参数设置】节点，我们可以看到仿真参数设置节点下的定制的参数及相应默认的参数值。用户可以根据需求，自行调整参数值。



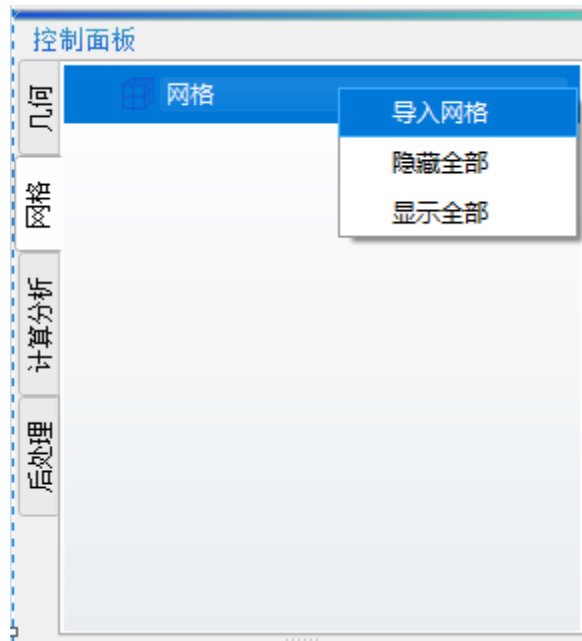
控制面板界面

4. 点击【算例】下的【求解设置】节点，我们可以看到求解设置节点下的定制的参数及相应默认的参数值。用户可以根据需求，自行调整参数值。



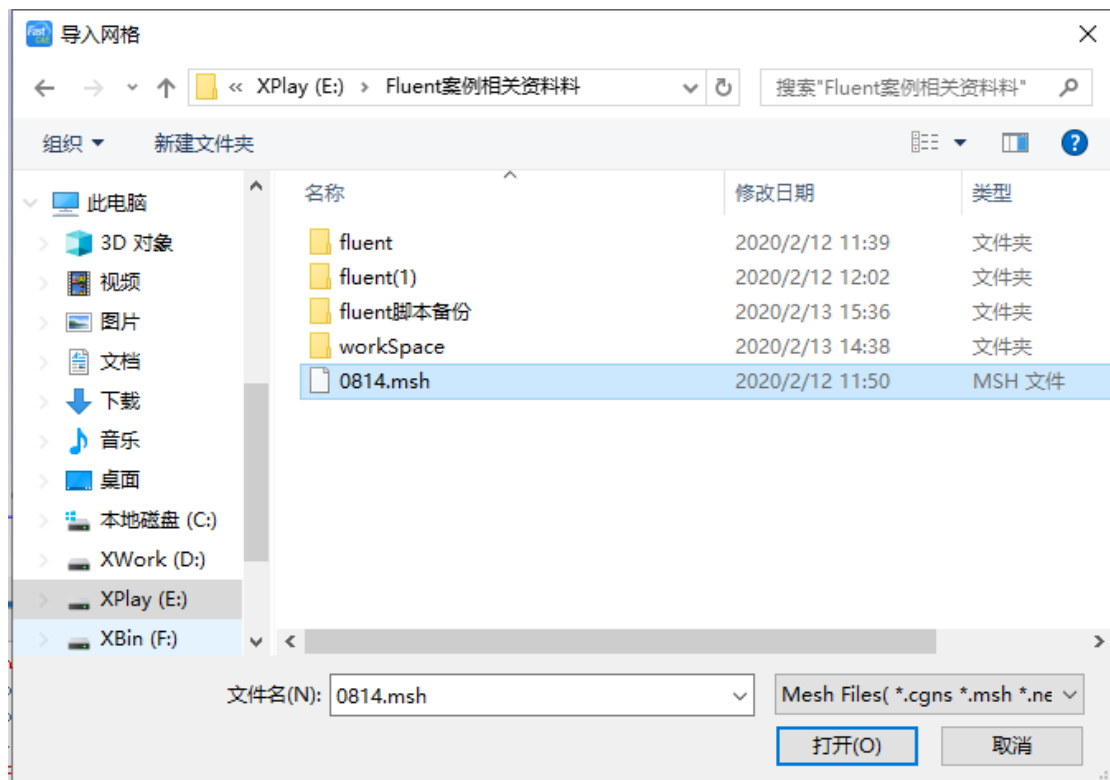
求解设置参数及其默认值

5.导入网格。点击【控制面板】-【网格】节点，将控制面板切换到网格界面，鼠标左键点击【网格】节点，点击鼠标右键，软件将弹出【网格】节点右键菜单。



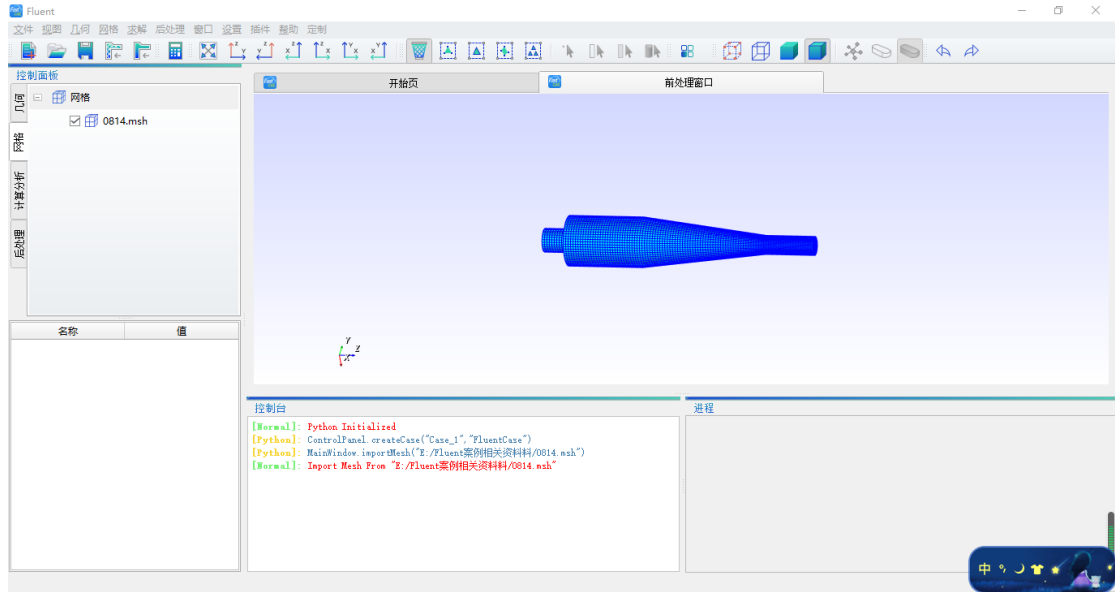
导入网格

6. 点击【网格】节点右键菜单中的【导入网格】菜单项，软件将弹出导入网格对话框。这里我们选择我们案例需要的 0814.msh 后，点击按钮，软件将加载 0814.msh 文件。



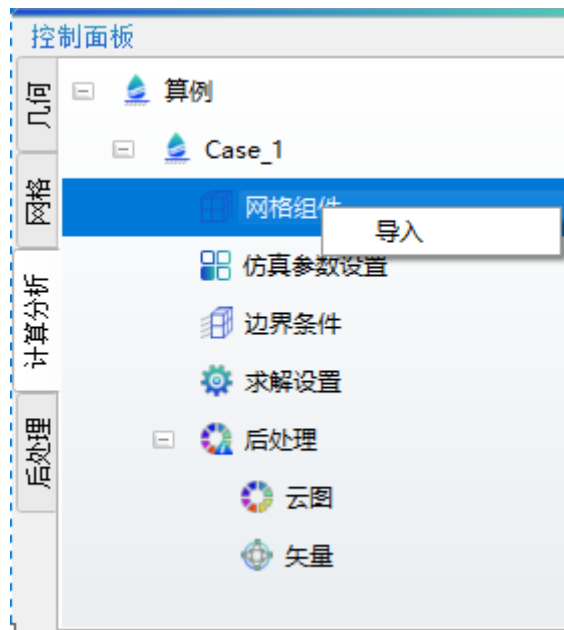
选择网格文件

7. 此时主界面中将在 0814.msh 文件，如下图所示。



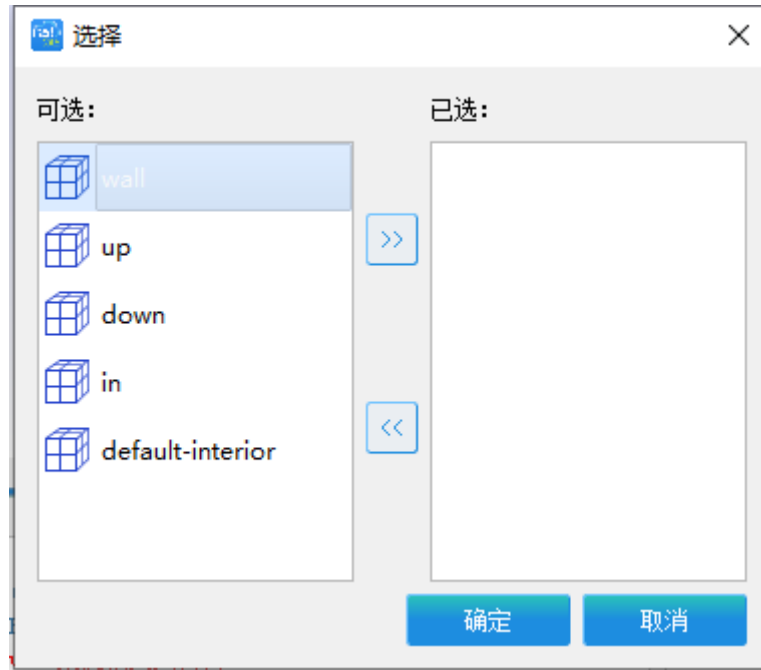
导入网格完成

8. 点击【控制面板】-【计算分析】将控制面板切换回算例所在分页，鼠标左键选中【算例】节点下的【网格组件】节点，点击鼠标右键，软件弹出【网格组件】节点右键菜单。



导入网格组件

9. 点击【网格组件】节点弹出右键菜单中的【导入】菜单项。软件弹出网格组件导入界面。



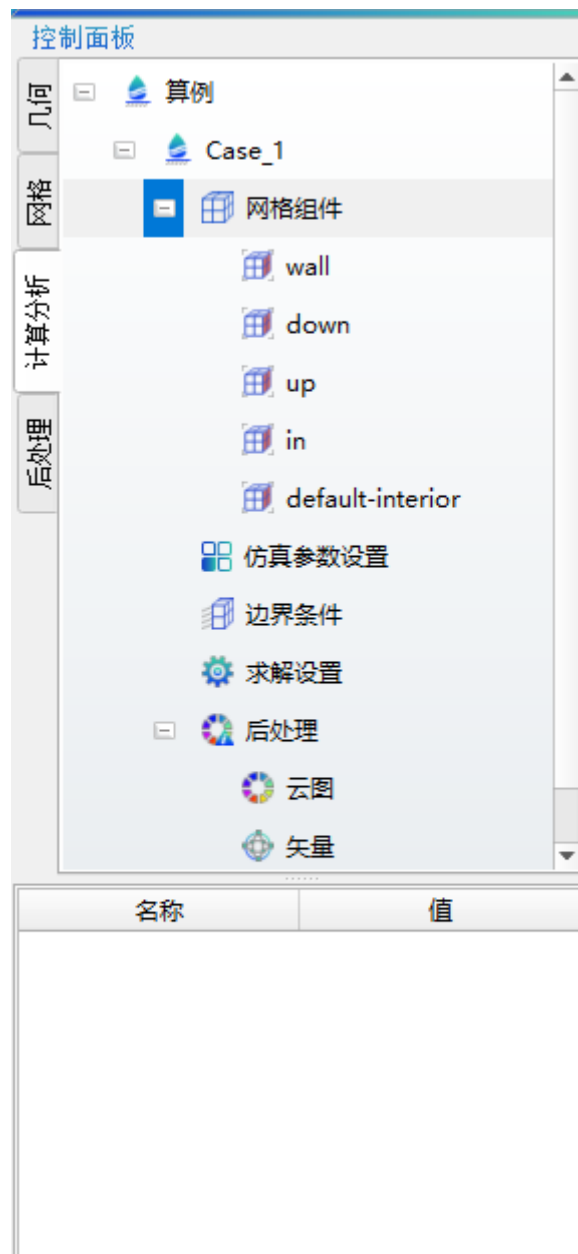
网格组件选择界面

10.在网格组件导入窗体界面中，我们使用鼠标左键将可选组件【wall】、【up】、【down】、【in】、【default-interior】组件全部选中后点击按钮，这样全部组件都会在右侧已选列表中，点击按钮完成网格组件的导入。



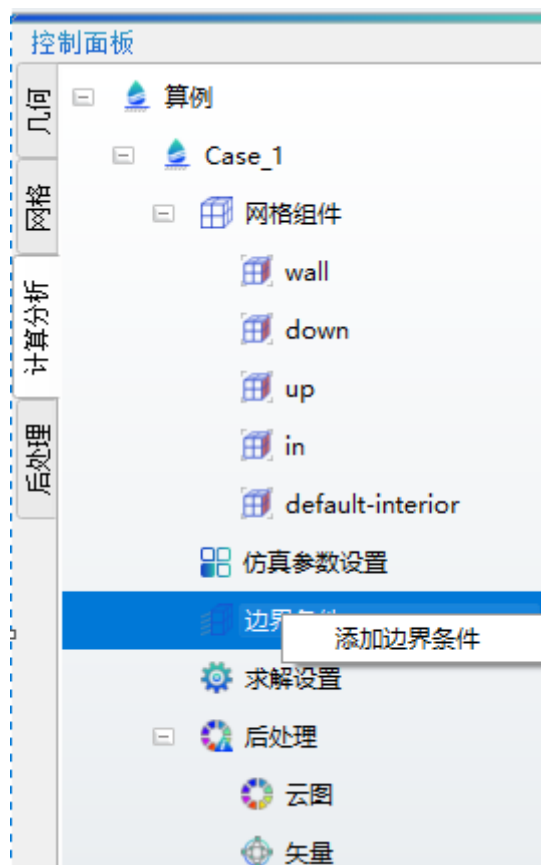
网格组件选择完成界面

11.此时的控制面板如下图所示。



导入网格组件后的控制面板

12.下面我们需要为不同的网格组件设置不同的边界条件，鼠标左键点击【算例】节点下面的【边界条件】节点使其处于选中状态，点击鼠标右键，软件将弹出【边界条件】右键菜单。



添加边界条件

13. 点击【边界条件】节点右键菜单中的【添加边界条件】菜单项，软件将弹出边界条件设置对话框。在对话框中，我们在网格组件选择为【wall】网格组件，类型选择为【Wall】壁面类型，点击按钮，这样就完成了【wall】网格组件的边界条件设定。



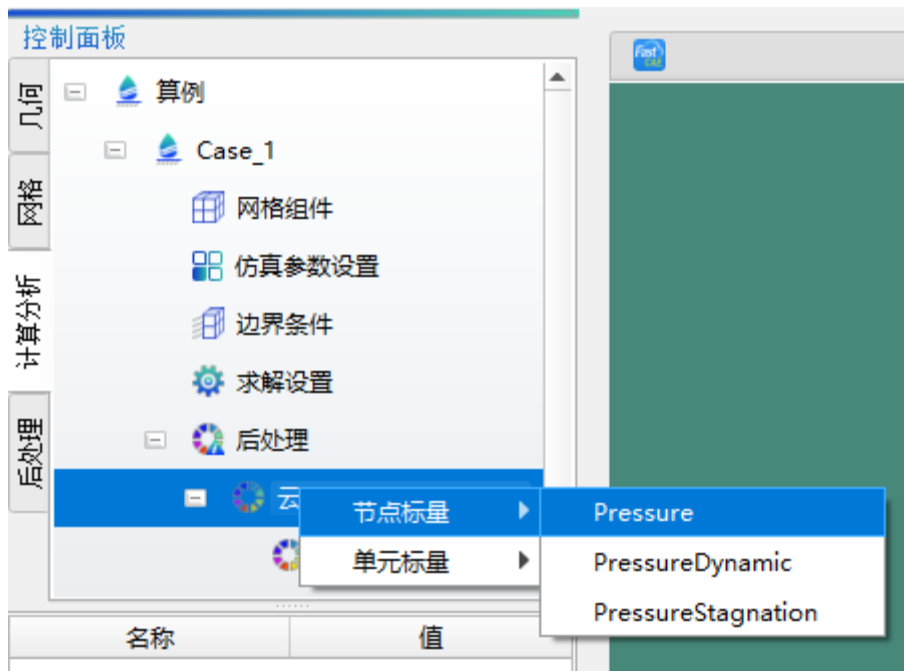
添加壁面边界条件

14. 按照同样方法将【down】网格组件设置为【Outlet】出口类型，将【up】网格组件设置为【Outlet】出口类型，将【in】网格组件设置为【Inlet】入口类型，此时【控制面板】-【计算分析】中的算例模型树如下图所示。



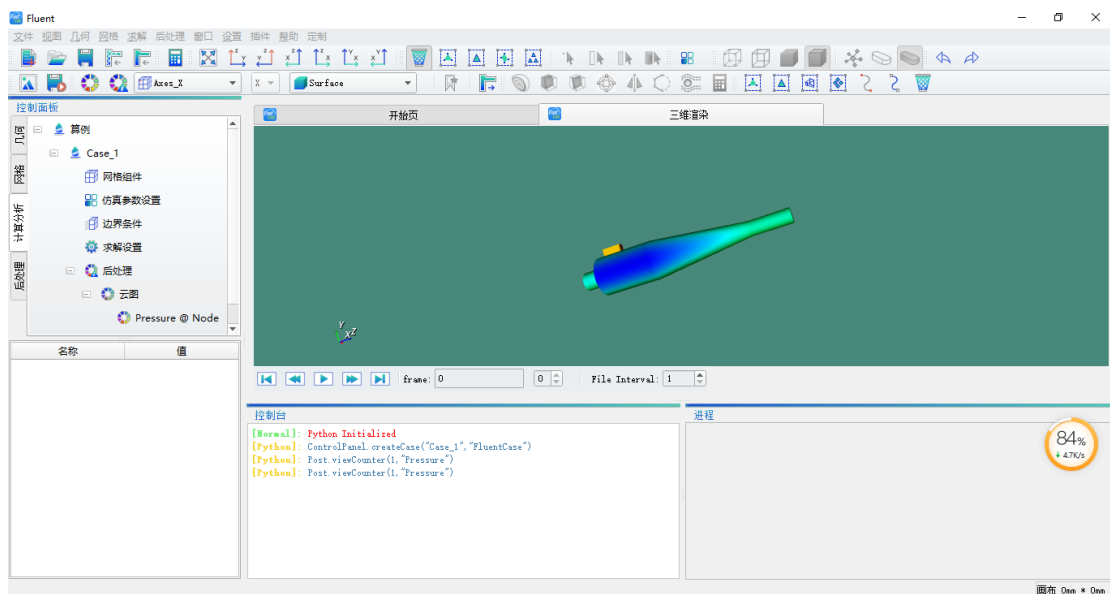
添加完边界条件的控制面板

15. 点击工具栏的按钮，软件将启动求解。在求解器完成后用户鼠标右键选择点击【后处理】节点下的【云图】节点，在弹出的右键菜单中选择【节点标量】 - 【Pressure】。



后处理压力云图操作

16.完成上一步的操作后，软件后处理将访问我们算例下 Result/Result.cgns 结果文件，并使用 Pressure 标量进行云图绘制。结果如下图所示。



软件后处理效果

OpenFoam 集成示例

FastCAE-OpenFoam 集成示例（可视化集成与插件拓展集成）

一、概述

1.1 案例介绍

本案例结合 FastCAE 提供的可视化定制功能与插件拓展机制，封装了 simpleFoam 求解器，形成了一款针对不可压缩流体稳态仿真问题的软件，本软件具有友好的交互界面，与求解器无缝对接，形成集前后处理于一体的针对特定问题的专用 CAE 仿真软件。

本案例名称为 OpenFoam 集成案例，其中所有资料包含源码和相关文档均可以在本站“[下载](#)”菜单栏目内进行下载。

1.2 集成目标

- 通过 FastCAE2.0 集成框架完成一个典型的以 OpenFoam 软件为求解器的软件产品集成工作，实现专业的用户界面定制，便捷求解操作。
- 通过本案例的开发，能够使用户快速掌握 FastCAE2.0 插件开发与定制开发相结合的开发过程，为 OpenFoam 软件集成与二次开发提供指导。

1.3 环境说明

开发环境

- Ubuntu16.04
- QtCreator3.4.1
- Linux 下 qt-opensource-linux-x64-5.4.2.run
- FastCAE 2.0
- OpenFoam-v1706

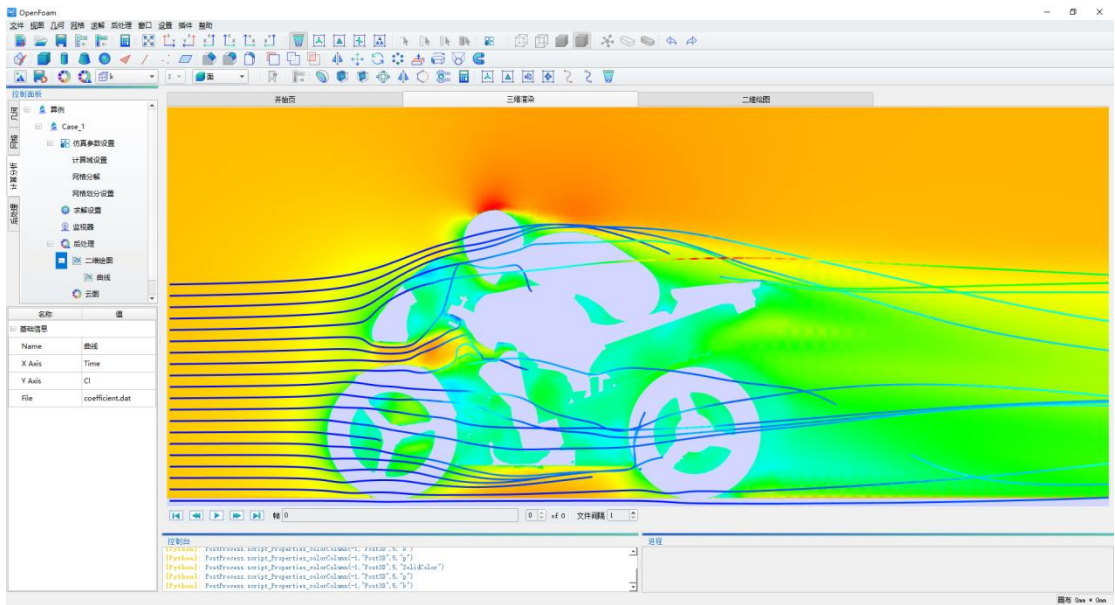
1.4 注意事项

- 对于 FastCAE 集成开发用户，建议开展本案例学习之前，要有一定 C++ 基础和 OpenFoam 使用经验。
- 应尽量避免在使用 FastCAE 过程中代码存放位置及文件存放位置中包括中文路径。
- 参考 4.1 把字典文件存放到指定文件夹
- 本案例使用的几何文件取自 \$FOAM_TUTORIALS/resources/geometry/motorBike.obj.gz,案例中没有做导入操作，在 AllRun.sh 脚本中会复制到工作目录下用于网格划分。

二、软件实现效果

本案例最终形成了基于 OpenFoam，使用 simpleFoam 求解器计算摩托车和骑手周围的稳态气流的软件系统，该系统仿真计算由 OpenFoam 内核实现，后处理功能与界面参数设置通过 FastCAE 的可视化定制功能实现，OpenFoam 字典参数设置通过 OpenFoam 适配插件实现。

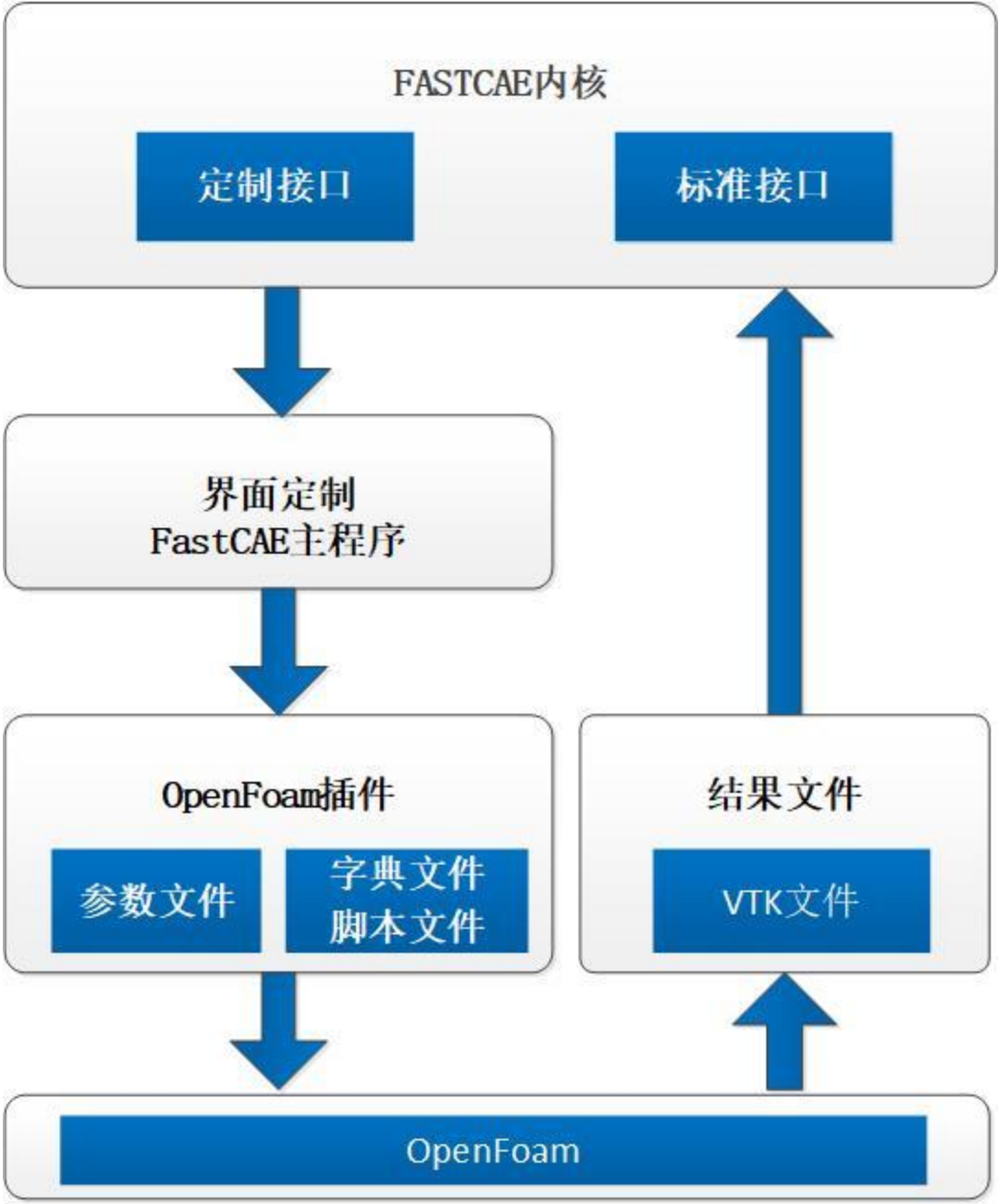
在完成软件的开发工作后，用户可以运行软件进行数值计算，我们可以看到，在主界面左侧为控制面板，我们的工程树设置，关联网格，设置边界条件等都可以在工程树上进行操作。右侧主体部分为展示窗体区域，前处理窗口，后处理三维渲染窗口都在这个区域进行显示。在下面相邻的两个区域分别为控制台和进程窗口，控制台用于显示用户的操作流程，进程窗口则主要对求解状态进行显示及控制。完成的软件整体效果如下图所示。



OpenFoam 案例集成软件效果

三、原理说明

本案例的实现过程是通过 FastCAE 平台实现软件界面定制, 并通过插件生成 OpenFoam 运行所需的参数文件，后台驱动 OpenFoam 软件进行科学计算并对计算结果进行展示。



案例原理图

3.1OpenFoam 部分

3.1.1OpenFoam 常用指令

OpenFoam 指令

blockMesh	-dict	计算域网格划分
surfaceFeatureExtract	-dict	提取特征边
snappyHexMesh	-dict -overwrite	六面体切割器

decomposePar	-dict	网格自动分割
simpleFoam	-dict	SIMPLE 算法稳态求解
mpirun	-np -parallel	并行计算
reconstructPar		网格重组

3.1.2OpenFoam 脚本文件说明

本案例中，我们采用脚本文件把相关指令放到脚本中执行计算

AllRun.sh

```
#!/bin/sh

cd ${0%/*} || exit 1 # Run from this directory

# Source tutorial run functions
. $WM_PROJECT_DIR/bin/tools/RunFunctions

#clean log file
./Allclean

# Alternative decomposeParDict name:
decompDict="-decomposeParDict system/decomposeParDict.6"

## Standard decomposeParDict name:

# unset decompDict

# copy motorbike surface from resources directory
\cp $FOAM_TUTORIALS/resources/geometry/motorBike.obj.gz constant/triSurface/

runApplication surfaceFeatureExtract

runApplication blockMesh

runApplication decomposePar $decompDict

# Using distributedTriSurfaceMesh?

if foamDictionary -entry geometry -value system/snappyHexMeshDict | \
grep -q distributedTriSurfaceMesh
then
runParallel $decompDict surfaceRedistributePar motorBike.obj independent
fi

runParallel $decompDict snappyHexMesh -overwrite
```



```
#- For non-parallel running: - set the initial fields
```

```
# restore0Dir
```

```
#- For parallel running: set the initial fields
```

```
restore0Dir -processor
```

```
runParallel $decompDict patchSummary
```

```
runParallel $decompDict potentialFoam
```

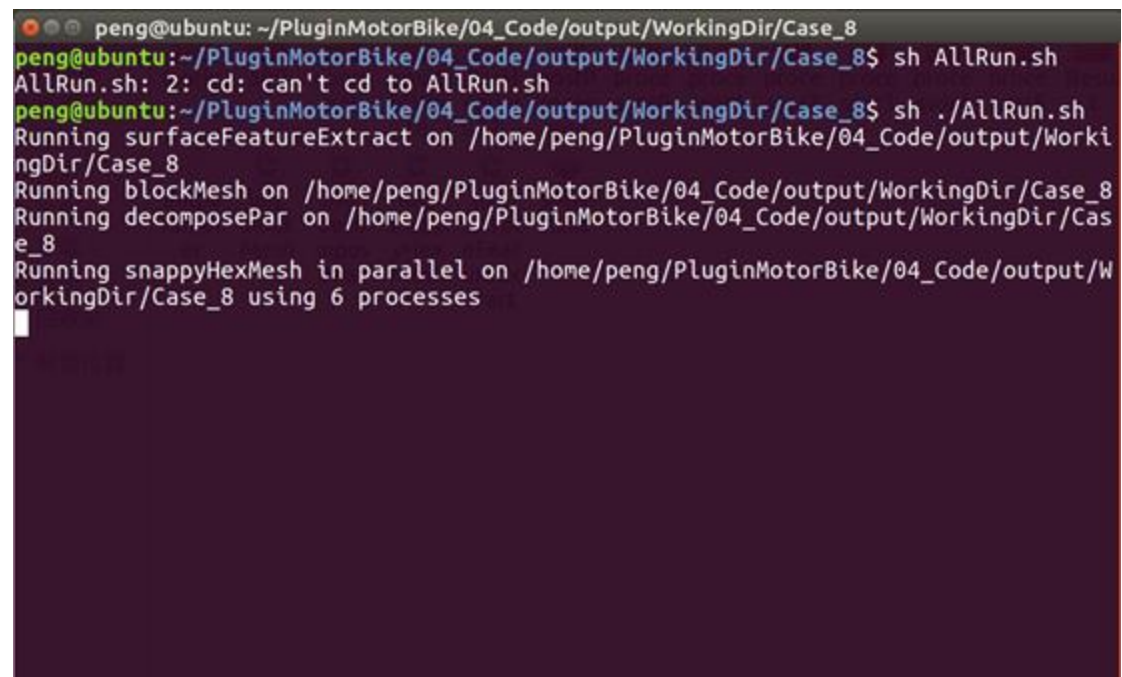
```
runParallel $decompDict $(getApplication)
```

```
runApplication reconstructParMesh -constant
```

```
runApplication reconstructPar -latestTime
```

```
#-----
```

脚本文件



```
peng@ubuntu: ~/PluginMotorBike/04_Code/output/WorkingDir/Case_8
peng@ubuntu:~/PluginMotorBike/04_Code/output/WorkingDir/Case_8$ sh AllRun.sh
AllRun.sh: 2: cd: can't cd to AllRun.sh
peng@ubuntu:~/PluginMotorBike/04_Code/output/WorkingDir/Case_8$ sh ./AllRun.sh
Running surfaceFeatureExtract on /home/peng/PluginMotorBike/04_Code/output/WorkingDir/Case_8
Running blockMesh on /home/peng/PluginMotorBike/04_Code/output/WorkingDir/Case_8
Running decomposePar on /home/peng/PluginMotorBike/04_Code/output/WorkingDir/Case_8
Running snappyHexMesh in parallel on /home/peng/PluginMotorBike/04_Code/output/WorkingDir/Case_8 using 6 processes
```

命令行下运行脚本

如上图，sh 脚本会执行 OpenFoam 相关指令进行运算

3.2FastCAE 部分

根据 3.0 原理图，我们可以分析得出 **FastCAE** 需要调用脚本文件让 OpenFoam 在后台运行，即我们在 **FastCAE 部分**需要在求解过程中完成 initparameter 文件的生成、字典文件及脚本文件复制到算例目录下。

其中，FastCAE **定制化插件**部分及**软件框架**将提供软件参数的设置、网格的导入及展示、后处理结果展示功能。这部分功能不需要用户重新开发，只需要进行配置就能够使用。

对于用户需要新增加参数的情况，需要在 FastCAE 框架中通过定制化插件配置好新增加的参数，同时在字典文件中进行参数的关联，可以参考 4.1 和 4.3 部分

3.3 算例变量

变量说明

变量类型	变量名称	类型	默认值
计算域设置	顶点坐标	表格型	(8 行 3 列) -5,-4,0 15,-4,0 15,4,0 -5,4,0 -5,-5,8 15,-4,8 15,4,8 -5,4,8
	各方向网格数量	表格型	(1 行 3 列) 20,8,8
网格分解	分解域的总数	整形	6
	各方向分解数量	表格形	(1 行 3 列) 6,1,1
	分解方法	字符串枚举	hierarchical ptscotch
网格划分设置	是否切割网格	Bool	true
	是否进行网格贴合	Bool	ture
	添加网格边界层	Bool	true
	最小表面细化等级	整形	5
	最大表面细化等级	整形	6

	边界层添加迭代数	整形	50
求解设置	求解器名称	字符串	simpleFoam
	文件生成间隔	整形	100
	开始步	整形	0
	结束步	整形	500

四、操作/开发步骤

本案例开发包括四个步骤：字典操作、脚本文件生成、软件界面定制、插件开发。

- 字典操作：从 OpenFoam 指定路径下获取字典文件添加参数
- 脚本文件生成：生成 AllRun.sh 脚本文件
- 软件界面定制：实现界面相关参数设置
- 插件开发：实现求解输入文件生成及求解后结果文件复制

4.1 字典操作

通过分析 OpenFoam 命令得知，OpenFoam 执行指令需要通过读取字典文件完成，字典文件引用 initparameter 参数文件实现参数的设置，字典文件需要用户根据算例需求提前设置好，放到指定目录下，字典文件如下图所示：

```

1  /*-----*- C++ -*/
2  =====
3  \ \ \ \ \ F i e l d      | OpenFOAM: The Open Source CFD Toolbox
4  \ \ \ \ \ O peration    | Version: plus
5  \ \ \ \ \ A nd          | Web: www.OpenFOAM.com
6  \ \ \ \ \ M anipulation |
7  /*-----*/
8  FoamFile
9  {
10     version      2.0;
11     format       ascii;
12     class        dictionary;
13     object       decomposeParDict;
14 }
15 #include "../initparameter"
16 // *****
17
18 numberOfSubdomains $core;
19
20 method            hierarchical;
21 // method         ptscotch;
22
23 simpleCoeffs
24 {
25     n              ($nx $ny $nz);
26     delta          0.001;
27 }
28
29 hierarchicalCoeffs
30 {
31     n              (3 2 1);
32     delta          0.001;
33     order          xyz;
34 }
35
36 manualCoeffs
37 {
38     dataFile       "cellDecomposition";
39 }
40
41
42 // *****

```

关联输入参数

对应initparameter文件中的参数

字典关联参数文件

Initparameter 参数文件由 FastCAE 调用求解器前生成，具体内容如下图所示：

```
initparameter
1 starttime .....0;
2 endtime .....500;
3 application .....simpleFoam;
4 writeInterval .....100;
5 xgridnumber .....20;
6 ygridnumber .....8;
7 zgridnumber .....8;
8 x1 .....-5;
9 y1 .....-4;
10 z1 .....0;
11 x2 .....15;
12 y2 .....-4;
13 z2 .....0;
14 x3 .....15;
15 y3 .....4;
16 z3 .....0;
17 x4 .....-5;
18 y4 .....4;
19 z4 .....0;
20 x5 .....-5;
21 y5 .....-5;
22 z5 .....8;
23 x6 .....15;
24 y6 .....-4;
25 z6 .....8;
26 x7 .....15;
27 y7 .....4;
28 z7 .....8;
29 x8 .....-5;
30 y8 .....4;
31 z8 .....8;
32 nx .....6;
33 ny .....1;
34 nz .....1;
35 core .....6;
36 decomposemethod .....hierarchical;
37 castellatedMesh .....true;
38 snap .....true;
39 addLayers .....true;
40 minlevel .....5;
41 maxlevel .....6;
42 nLayerIter .....50;
```

initparameter 文件

4.1.1 复制字典文件

- 在 FastCAE 2.0 应用所在路径的上级目录下创建 solver/dictionary 文件夹，此算例中 dictionary 全路径为 PluginMotorBike/04_Code/Output/solver/dictionary。

FastCAE2.0 的全路径为 PluginMotorBike/04_Code/Output/bin

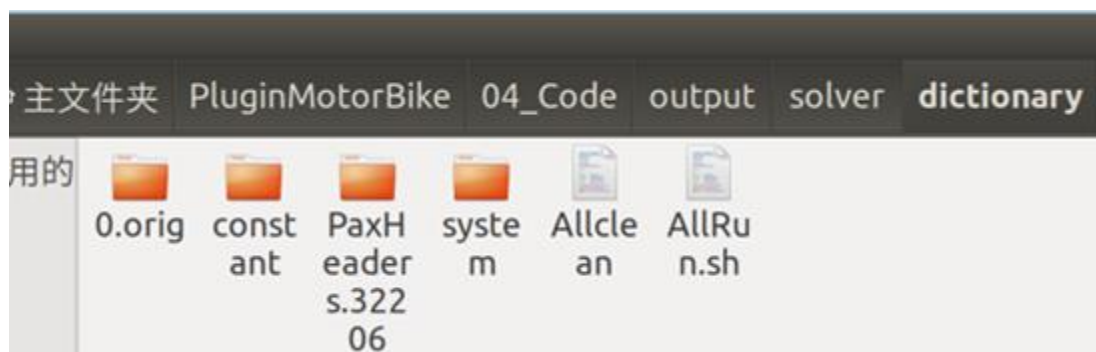
- 将\$FOAM_TUTORIALS \incompressible\simpleFoam\motorBike 文件夹中所有文件复制到 dictionary 文件夹下（也可以使用我们提供配置好的字典文件），Linux 下命令如下：

```
cp -r $FOAM_TUTORIALS\incompressible\simpleFoam\motorBike .
```

1

Shell

- linux 下需要将 Allclean, AllRun.sh 赋予运行权限，如下图所示：



字典文件路径

AllRun.sh 驱动求解计算的脚本文件

Allclean 清除算例日志文件

Linux 更改权限命令如下：

```
chmod 777 AllRun.sh
```

```
chmod 777 Allclean
```

4.1.2 字典文件关联参数

OpenFoam 可通过修改字典文件实现相关指令参数的修改，在字典文件中包含 initparameter 参数文件，用于参数的修改，可用文本编辑工具 (notepad++) 修改字典文件，本例中字典文件路径 PluginMotorBike/04_Code/output/solver/dictionary（用的是我们提供的配置好的字典文件可以不做此操作）

字典中的参数采用 \$parameter 格式指定，parameter 为参数名称，此名称要与 initparameter 文件中的变量名称一致。本案例中需要修改 blockMeshDict、

decomposeParDict.6、controlDict、snappyHexMeshDict 字典文件，修改过程如下：

- 修改 blockMeshDict 字典文件

文件所在路径 /PluginMotorBike/04_Code/output/solver/

dictionary/system/blockMeshDict

```
blockMeshDict
1  /*-----*--C++--*-----*\
2  |=====|
3  |\\|Field|OpenFOAM: The Open Source CFD Toolbox|
4  |\\|Operation|Version: plus|
5  |\\|And|Web: www.OpenFOAM.com|
6  |\\|Manipulation|
7  \*-----*\
8  FoamFile
9  {
10  ....version.....2.0;
11  ....format.....ascii;
12  ....class.....dictionary;
13  ....object.....blockMeshDict;
14  }
15  #include "../initparameter"
16  //*****
17
18  convertToMeters 1;
19
20  vertices
21  (
22  .... ($x1 $y1 $z1)
23  .... ($x2 $y2 $z2)
24  .... ($x3 $y3 $z3)
25  .... ($x4 $y4 $z4)
26  .... ($x5 $y5 $z5)
27  .... ($x6 $y6 $z6)
28  .... ($x7 $y7 $z7)
29  .... ($x8 $y8 $z8)
30  );
31
32  blocks
33  (
34  .... hex (0 1 2 3 4 5 6 7) ($xgridnumber $ygridnumber $zgridnumber) simpleGrading (1 1 1)
35  );
36
37  edges
38  (
39  );
40
41  boundary
42  (
43  .... frontAndBack
44  .... {
45  .... type patch;
46  .... faces
```

blockMeshDict 字典

对于"vertices"，现在有 8 个顶点，则有 8 行，分别为：

(\$x1 \$y1 \$z1)

(\$x2 \$y2 \$z2)

(\$x3 \$y3 \$z3)

(\$x4 \$y4 \$z4)

(\$x5 \$y5 \$z5)

(\$x6 \$y6 \$z6)

(\$x7 \$y7 \$z7)

(\$x8 \$y8 \$z8)

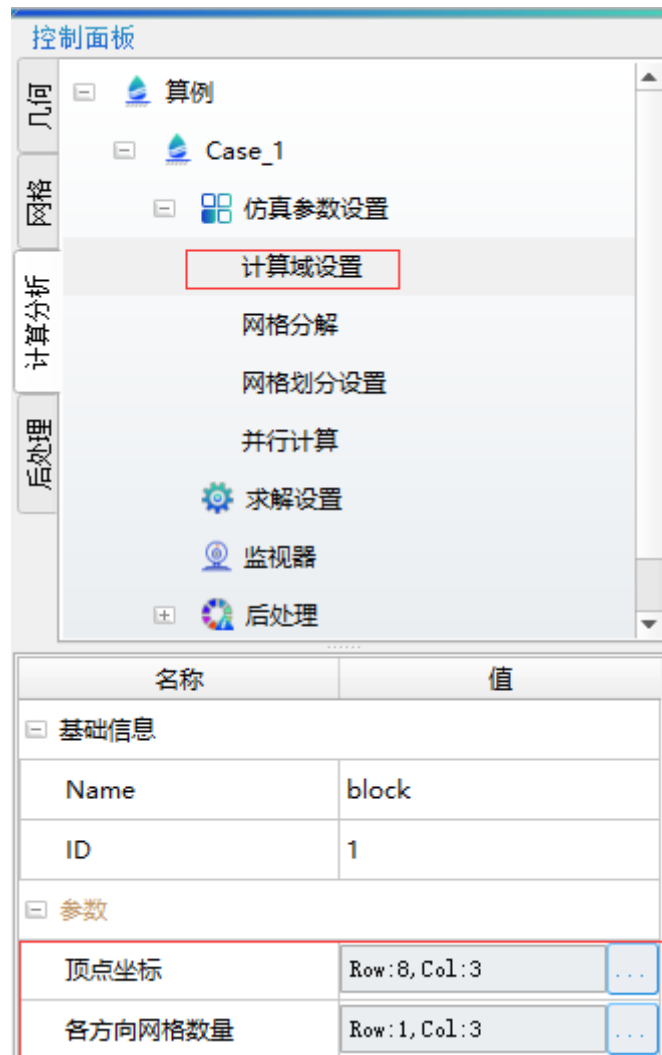
如有新增顶点，则可以按顺序增加顶点坐标，如\$x9 \$y9 \$z9 等。

对于 blocks, \$xgridnumber \$ygridnumber \$zgridnumber 用于设置各方向网格数量，如下图所示：

```
blocks
(
  ...hex (0.1.2.3.4.5.6.7) ($xgridnumber-$ygridnumber-$zgridnumber) simpleGrading (1.1.1)
);
```

各方向上网格数量

界面上计算域的设置，如下图所示：



计算域设置

- 修改 decomposeParDict.6 字典文件

文件所在路径：System/decomposeParDict.6


```

1  /*-----*-- C++ --*-----
2  |=====|
3  |\\...../ F i e l d .....| OpenFOAM: The Open Source CFD
4  |\\...../ O p e r a t i o n .....| Version:  plus
5  |\\...../ A n d .....| Web:  www.OpenFOAM.com
6  |\\...../ M a n i p u l a t i o n .....|
7  \*-----*
8  FoamFile
9  {
10  .. version ..... 2.0;
11  .. format ..... ascii;
12  .. class ..... dictionary;
13  .. object ..... decomposeParDict;
14  }
15  #include "../initparameter"
16  // *****
17
18  numberOfSubdomains $score;
19
20  method ..... hierarchical;
21  // method ..... ptscotch;
22
23  simpleCoeffs
24  {
25  .. n ..... ($nx $ny $nz);
26  .. delta ..... 0.001;
27  }
28
29  hierarchicalCoeffs
30  {
31  .. n ..... (3 2 1);
32  .. delta ..... 0.001;
33  .. order ..... xyz;
34  }
35
36  manualCoeffs
37  {
38  .. dataFile ..... "cellDecomposition";
39  }
40

```

decomposeParDict 字典

\$nx,\$ny,\$nz 用于关联各方向分解数量,如下图所示:

```

simpleCoeffs
{
  .. n ..... ($nx $ny $nz);
  .. delta ..... 0.001;
}

```

各方向分解数量

\$score 用于关联计算核数,如下图所示:

```

numberOfSubdomains $score;

```

计算核数设置

注：nx,ny,nz 相乘之和要与\$score 参数相等，否则网格划分过程中会出错。

界面上网格分解参数，如下图所示：

控制面板

几何

网格

计算分析

后处理

[-] 算例

[-] Case_1

[-] 仿真参数设置

计算域设置

网格分解

网格划分设置

求解设置

监视器

[+] 后处理

名称	值
[-] 基础信息	
Name	DecomposeParDict
ID	1
[-] 参数	
分解域的总数	6
各方向分解数量	Row:1, Col:3
分解方法	hierarchical

网格分解参数

3、修改 controlDict 字典文件

所在路径/PluginMotorBike/04_Code/output/solver/ dictionary/system/controlDict

```

1 /*-----*.C++.*-----
2 | .=====| .
3 | \\ / Field | OpenFOAM: The Open Sc
4 | \\ / Operation | Version: plus
5 | \\ / And | Web: www.OpenFOA
6 | \\ / Manipulation |
7 \*-----
8 FoamFile
9 {
10     version 2.0;
11     format ascii;
12     class dictionary;
13     object controlDict;
14 }
15 //*****
16 #include "../initparameter"
17 application $application;
18
19 startFrom latestTime;
20
21 startTime $starttime;
22
23 stopAt endTime;
24
25 endTime $endtime;
26
27 deltaT 1;
28
29 writeControl timeStep;
30
31 writeInterval $writeInterval;
32
33 purgeWrite 0;
34
35 writeFormat binary;
36
37 writePrecision 6;
38
39 writeCompression off;
40
41 timeFormat general;
42
43 timePrecision 6;
44
45 runTimeModifiable true;
46

```

controlDict 字典

用于关联求解设置参数，如下图所示：

求解器名称	simpleFoam
文件生成间隔	100
开始步	0
结束步	500
启用并行	☐

界面求解器参数

\$application 关联求解器名称

\$writeInterval 关联文件生成间隔

\$starttime 关联开始时间

\$endtime 用于关联结束时间

4、修改 snappyHexMeshDict 字典文件

所 在 路 径 /PluginMotorBike/04_Code/output/solver/

dictionary/system/snappyHexMeshDict


```

275 .....maxThicknessToMedialRatio:0.3;
276
277 .....//Angle used to pick up medial axis points
278 .....//Note: changed(corrected) w.r.t 17x! 90 degrees corresponds to 130 in 17
279 .....minMedianAxisAngle:90;
280
281
282 .....//Create buffer region for new layer terminations
283 .....nBufferCellsNoExtrude:0;
284
285
286 .....//Overall max number of layer addition iterations. The mesher will exit
287 .....//if it reaches this number of iterations; possibly with an illegal
288 .....//mesh.
289 .....nLayerIter:$nLayerIter;
290 }
291

```

边界层参数

界面求解设置参数，如下图所示：

名称	值
[-] 基础信息	
Name	SnappyHexMeshSetti...
ID	1
[-] 参数	
是否切割网格	☒
是否进行网格贴合	☒
添加网格边界层	☒
最小表面细化等级	5
最大表面细化等级	6
边界层添加迭代数	50

界面 SnappyHexMesh 参数

\$castellatedMesh 关联是否切割网格

\$snap 是否进行网格贴合

\$addLayers 添加网格边界层

\$minlevel 最小表面细化等级

\$maxlevel 最大表面细化等级

\$nLayerIter 边界层添加迭代数

4.2 脚本文件

例中用到的脚本文件为 Allclean 和 AllRun.sh 文件，AllRun.sh 用于启动求解计算，Allclean 为清空历史日志文件及网格文件等，要在求解计算前先调用 Allclean 脚本。

AllRun.sh 和 Allclean 文件可通过下载获取，也可以在此基础上自行扩展。

本案例中脚本文件存放路径为 PluginMotorBick/04_Code/Output/solver/dictionaray

AllRun.sh 脚本文件内容如下所示：

```
#!/bin/sh

cd ${0%/*} || exit 1 # Run from this directory

# Source tutorial run functions
. $WM_PROJECT_DIR/bin/tools/RunFunctions

#clean log file
./Allclean

# Alternative decomposeParDict name:
decompDict="-decomposeParDict system/decomposeParDict.6"

## Standard decomposeParDict name:
# unset decompDict

# copy motorbike surface from resources directory
\cp $FOAM_TUTORIALS/resources/geometry/motorBike.obj.gz constant/triSurface/

runApplication surfaceFeatureExtract

runApplication blockMesh

runApplication decomposePar $decompDict

# Using distributedTriSurfaceMesh?
if foamDictionary -entry geometry -value system/snappyHexMeshDict | \
grep -q distributedTriSurfaceMesh
then
runParallel $decompDict surfaceRedistributePar motorBike.obj independent
fi

runParallel $decompDict snappyHexMesh -overwrite

#- For non-parallel running: - set the initial fields
```

```
# restore0Dir

#- For parallel running: set the initial fields

restore0Dir -processor

runParallel $decompDict patchSummary

runParallel $decompDict potentialFoam

runParallel $decompDict $(getApplication)

runApplication reconstructParMesh -constant

runApplication reconstructPar -latestTime

#-----
```

主要脚本文件描述如下：

- `decompDict="-decomposeParDict system/decomposeParDict.6"`
用于设置网格分割需要的字典文件参数
- `\cp $FOAM_TUTORIALS/resources/geometry/motorBike.obj.gz constant/triSurface/`
从 OpenFoam 安装目录下复制网格文件到字典目录 `constant/triSurface/`
提取特征边
- `runApplication surfaceFeatureExtract`
划分计算域网格
- `runApplication blockMesh`
网格自动分割
- `runApplication decomposePar`
六面体网格切割
- `runParallel $decompDict snappyHexMesh -overwrite`
- 调用求解器求解计算
- `runParallel $decompDict $(getApplication)`
网格重组
- `runApplication reconstructPar -latestTime`

4.3 软件界面定制

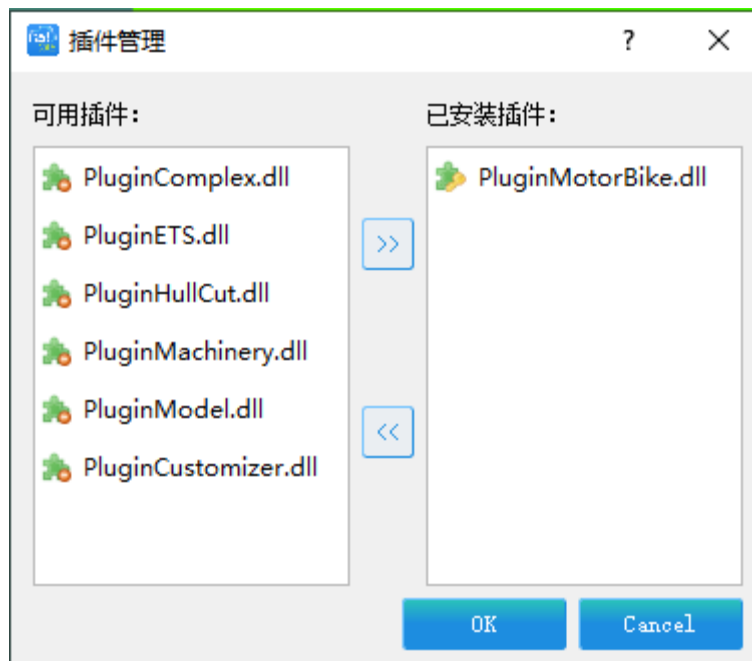
使用 FASTCAE 定制插件完成，OpenFoam 案例界面定制，定制内容如下表所示：

软件定制内容

编号	定制步骤说明
1	加载定制化插件
2	添加软件基本信息
3	网格相关设置
4	网格导出设置
5	算例设置
6	添加计算域设置
7	网格分解设置
8	网格划分设置
9	添加求解设置
10	后处理二维曲线设置
11	求解器管理

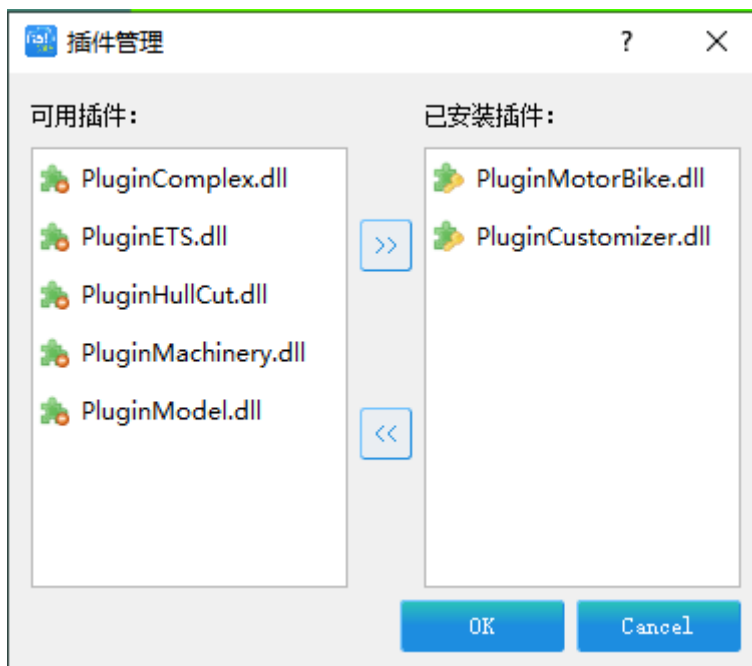
4.3.1 加载定制化插件

点击【插件】-【插件管理】，启动 FastCAE 2.0 版本软件加载 PluginCustomizer.dll 插件，开始进行软件常规项定制。



插件管理

在可用插件列表选中 PluginCustomizer.dll 插件，点击按键。



加载插件

点击按键，这样 PluginCustomizer 插件将会被 FastCAE 软件装载。软件菜单栏上将会出现定制项。



点击【定制】-【开始定制】，软件标题栏上名称后面会有“---定制中”字样，此时我们处于定制模式。



4.3.2 添加软件基本信息

点击【帮助】-【关于】，软件将弹出软件基本信息设置界面，这里面我们按照下图输入我们想要填入的软件基本信息。然后点击

确定

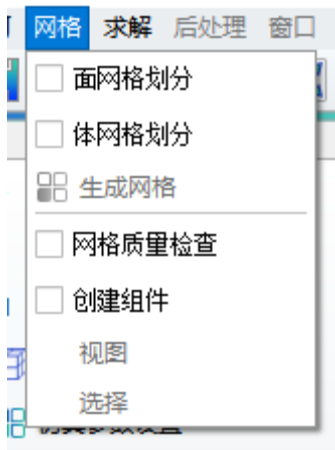
按键，完成软件基本信息的录入。

The image shows a dialog box titled '软件基本参数' (Software Basic Parameters). It is divided into two main sections: '基本信息' (Basic Information) on the left and '图片信息' (Image Information) on the right. The '基本信息' section contains several text input fields: '中文名称' (Chinese Name) with 'OpenFoam' and a red asterisk, '英文名称' (English Name) with 'OpenFoam' and a red asterisk, '版本信息' (Version Information) with 'V1.0', '单位名称' (Unit Name) with '青岛数智船海科技有限公司', '电子邮箱' (Email) with 'fastcae@diso.cn', and '单位网站' (Unit Website) with 'www.fastcae.com'. The '图片信息' section contains two image placeholder boxes: 'Logo (建议尺寸60X60)' and '欢迎页 (建议尺寸400X300)'. At the bottom right of the dialog are '确定' (OK) and '取消' (Cancel) buttons.

基本参数

4.3.3 网格相关设置

由于我们 OpenFoam 案例不需要网格剖分、网格质量检查和创建组件功能，为保证最终软件最简化，我们定制去掉这部分功能。点击【网格】，在菜单中将【面网格划分】、【体网格划分】、【网格质量检查】、【创建网格组件】菜单前面的勾去掉。

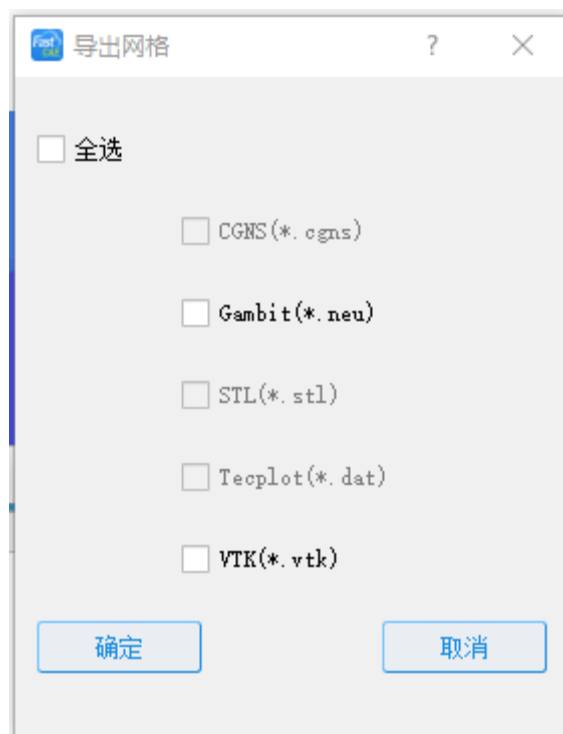


网格相关

我们的 OpenFoam 案例从 OpenFoam 安装目录下读取网格文件，所以我们去掉网格文件导入。

4.3.4 网格导出设置

我们的案例不需要导出网格功能，为保证最终软件最简化，我们定制将该部分功能去掉。点击【文件】-【导出网格】，软件将弹出到处网格功能定制界面，我们将全选前面的勾选的勾去掉，确保勾选项处于反选状态，点击按键，完成导出网格功能的定制，如下图所示：



网格导出设置

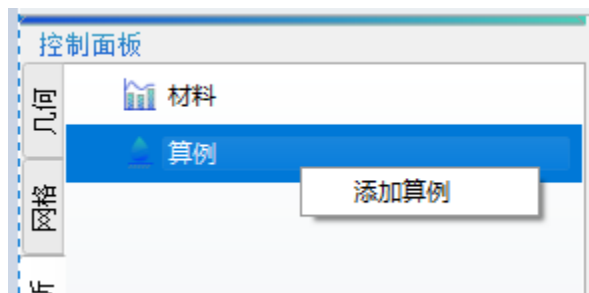
4.3.5 算例设置

下面我们进行模型树的定制，将控制面板切换到计算分析分页。



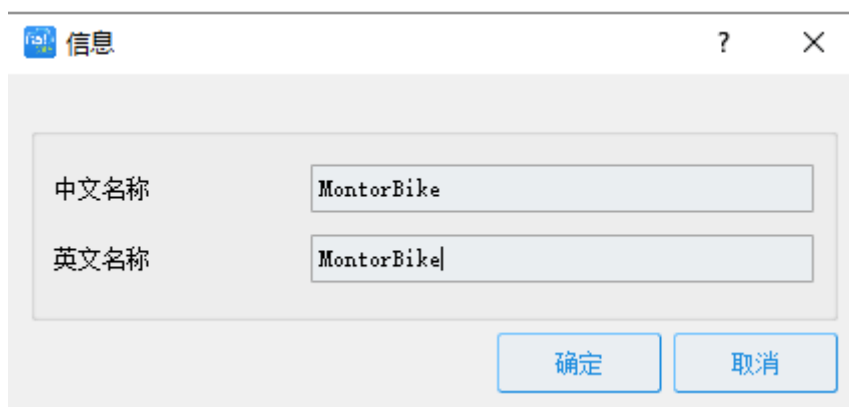
算例节点

鼠标左键选中算例节点，点击鼠标右键，点击弹出右键菜单的【添加算例】菜单项。

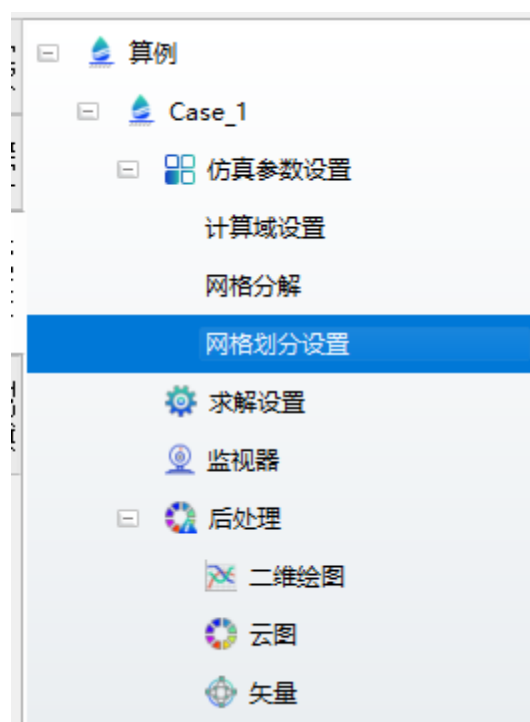


添加算例

软件将弹出添加算例对话框，我们按照下图添加入算例中文名称和英文名称。点击按钮完成算例名称信息的录入。



此时，【控制面板】-【算例】节点下面将会有有一个用于定制的算例树。



网格划分设置节点

4.3.6 添加计算域设置

鼠标左键单击算例树下的【仿真参数设置】节点，右键弹出【添加子项】，添加“计算域设置”节点。

1. 双击“计算域设置”节点，添加“顶点坐标”和“各方向网格数量”表格参数。

(用户配置参数名称要与文档中给出的变量名称一致，因为生成参数文件时会用到此名称去获取对应的参数值)

FastCAE

	x	y	z
1	-5	-4	0
2	15	-4	0
3	15	4	0
4	-5	4	0
5	-5	-5	8
6	15	-4	8
7	15	4	8
8	-5	4	8

Export Import OK Apply Cancel

项点坐标

表格类型参数

名称 各方向网格数量

行数 1

列数 3

默认值 刷新表格

x	y	z
20	8	8

确定 取消

各方向网格数量

计算域设置

?

×

基本信息

中文名称

计算域设置

英文名称

block

图标

上传图标

参数列表

添加

删除

编辑

清空全部

序号	名称	类型	默认值
1	顶点坐标	Table	8x3
2	各方向网格数量	Table	1x3

显示参数组

确定

取消

计算域设置

4.3.7 网格分解设置

鼠标左键单击算例树下的【仿真参数设置】节点，右键弹出【添加子项】，添加“网格分解设置”

双击“网格分解”节点，添加“分解域的总数”、“各方向分解数量”、“分解方法”。

整型参数

?

×

名称

分解域的总数

单位

默认值

6

最小值

-1000000

最大值

1000000

确定

取消

分解域的总数

表格类型参数

?

×

名称

各方向分解数量

行数

1

列数

3

默认值

刷新表格

x	y	z
6	1	1

确定

取消

各方向分解数量

枚举类型参数值

?

×

名称

分解方法

默认值

hierarchical

编辑

确定

取消

分解方法



添加分解方法枚举值

可以添加 hierarchical 和 ptscotch 两种分解方法，默认用 hierarchical 方法。

4.3.8 网格划分设置

鼠标左键单击算例树下的【仿真参数设置】节点，右键弹出【添加子项】，添加“网格划分设置”

右键点击“网格划分设置”节点，添加“是否切割网格”、“是否进行网格贴合”、“添加网格边界层”、“最小表面细化等级”、“最大表面细化等级”、“边界层添加迭代数”

网格划分设置

?

×

基本信息

中文名称

网格划分设置

英文名称

SnappyHexMeshSetting

图标

上传图标

参数列表

添加

删除

编辑

清空全部

序号	名称	类型	默认值
1	是否切割网格	Bool	true
2	是否进行网格贴合	Bool	true
3	添加网格边界层	Bool	true
4	最小表面细化等级	Int	5
5	最大表面细化等级	Int	6
6	边界层添加迭代数	Int	50

显示参数组

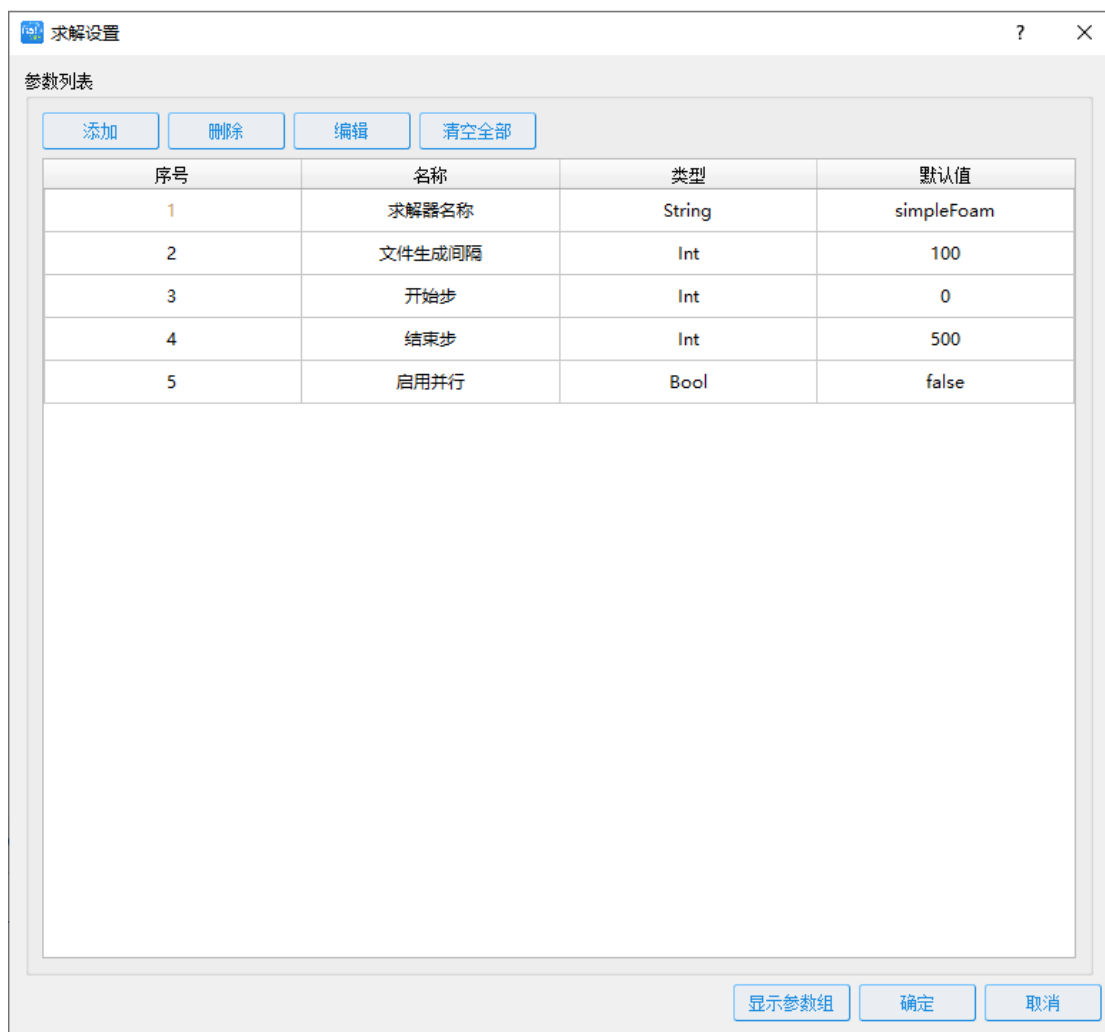
确定

取消

网格划分设置

4.3.9 添加求解设置

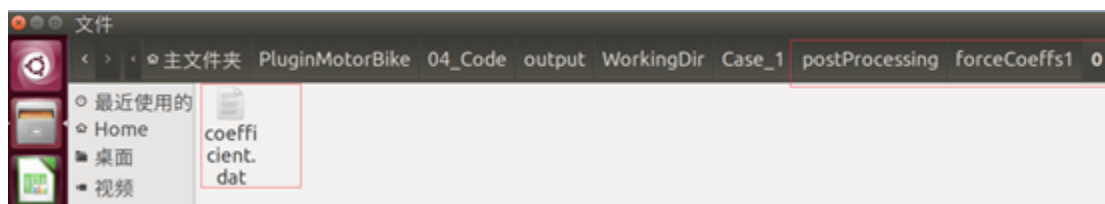
鼠标左键双击【控制面板】-【算例】节点下的【求解设置】节点，软件将弹出求解参数设置界面,添加“求解器名称”，“文件生成间隔”，“开始步”，“结束步”，“启用并行”等参数。



求解参数设置

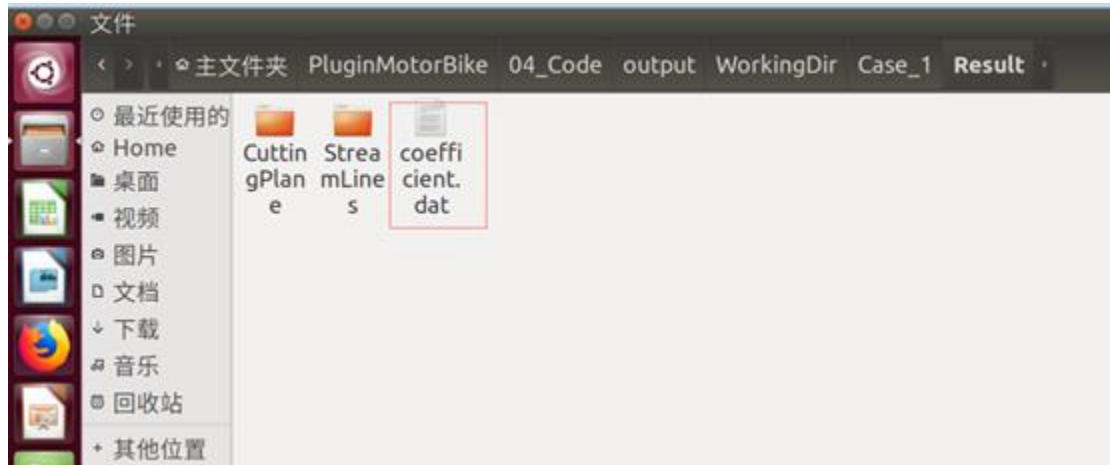
4.3.10 后处理二维曲线设置

本算例求解完成后，二维文件默认保存在 postProcessing 目录下不符合 FastCAE 后处理文件路径读取规则，如下图所示：



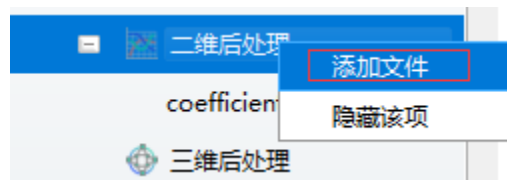
求解完成后存放路径

求解完成后，需通过转换函数将结果文件复制算例目录下的 Result/ coefficient.dat 文件（已在插件中完成此功能）。如下图所示：



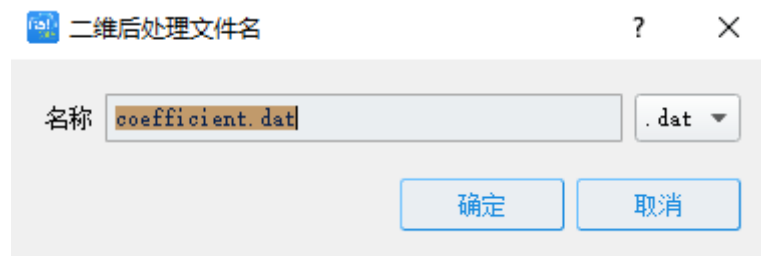
二维曲线文件转换后存放路径

在“二维后处理”节点上，按右键弹出“添加文件”对话框，如下图所示：



添加文件

单击“添加文件”弹出如下对话框，如下图所示：



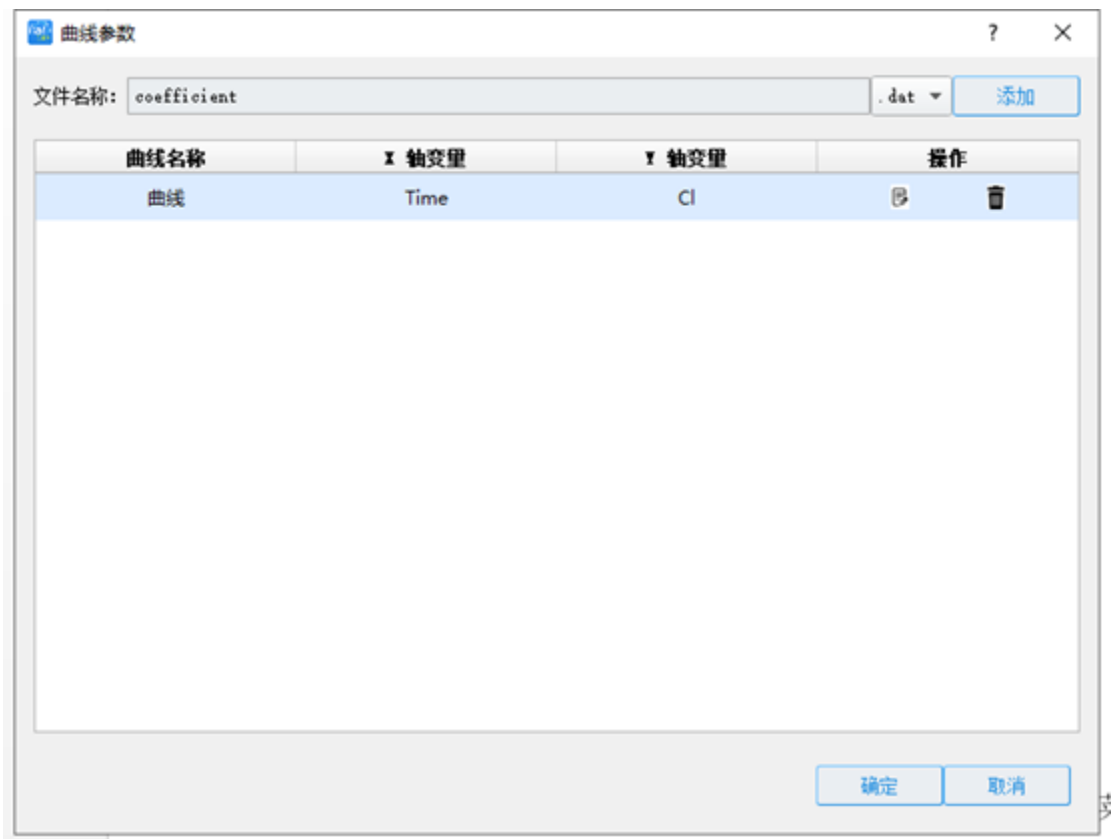
二维文件名

添加 x 轴和 y 轴变量名称，如下图所示：



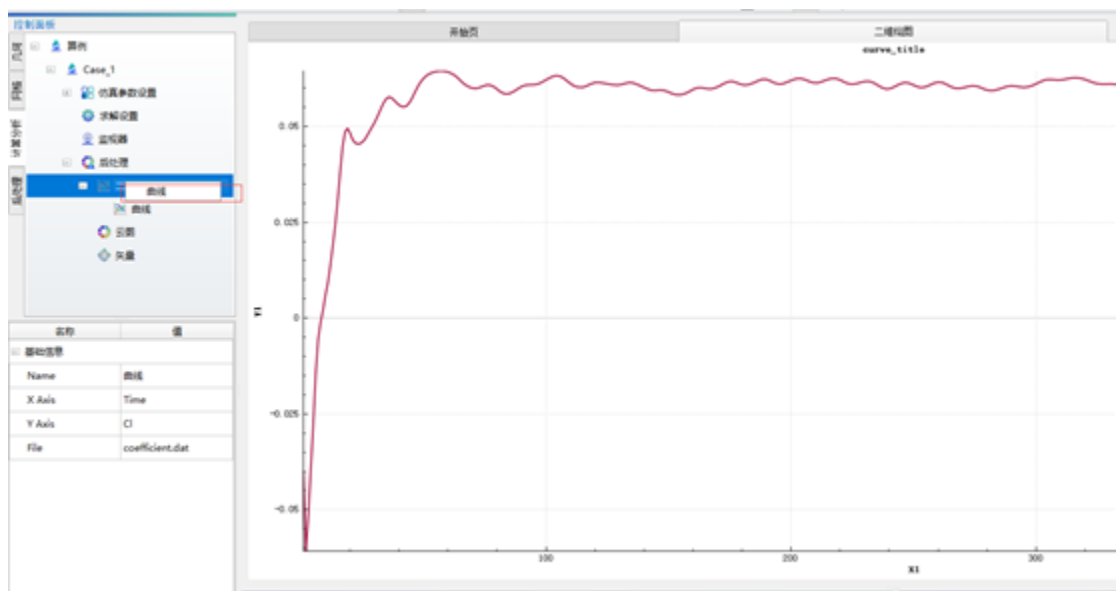
变量名称

曲线参数设置如下图所示：



曲线参数

在“二维曲线”节点上，单击“右键”显示出对应的二维曲线，如下图所示：



二维曲线显示

4.3.11 求解器管理

案例求解器设置，点击菜单栏【求解器】-【求解器管理】项，软件将弹出求解器管理界面。



求解器管理

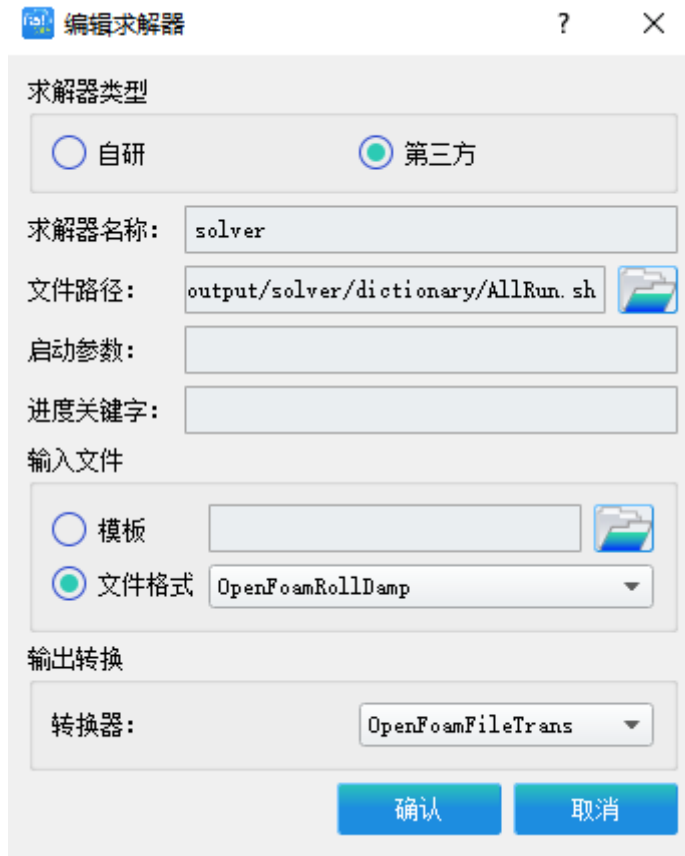
点击求解器管理界面的



按键，软件将弹出添加求解器界面，按照下图所示添加求解器信息。需要注意的是这里的求解器路径要选择则 Solver.exe 可执行文件所在的文件路径。点击界面中的



按键完成求解器的添加。



编辑求解器

- “文件格式”OpenFoamRollDamp 为我们注册的求解器输入文件生成的方法
- “转换器”OpenFoamFileTrans 为我们求解完成后进行文件转换注册的方法。

此时的求解器管理界面如下图所示，点击界面右上角的



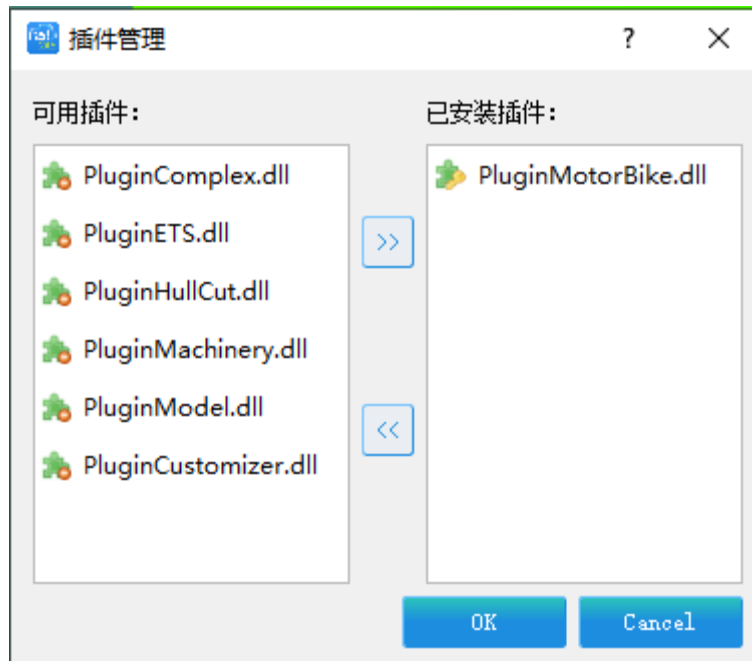
按键，完成求解器的设置。

4.4 OpenFoam 插件开发

moduleBase	2020-03-05 10:06	文件夹	
ParaClassFactory	2020-03-05 9:33	文件夹	
PluginComplex	2020-03-05 10:06	文件夹	
PluginCustomizer	2020-03-05 10:06	文件夹	
PluginETS	2020-03-05 11:07	文件夹	
PluginHull	2020-03-05 10:06	文件夹	
PluginMachinery	2020-03-05 10:06	文件夹	
PluginManager	2020-03-05 10:06	文件夹	
PluginModel	2020-03-05 10:07	文件夹	
PluginMotorBike	2020-03-10 9:46	文件夹	
PostWidgets	2020-03-05 10:06	文件夹	
ProjectTree	2020-03-10 17:42	文件夹	
ProjectTreeExtend	2020-03-05 10:06	文件夹	
python	2020-03-05 10:06	文件夹	
qrc	2020-03-05 9:28	文件夹	
SelfDefObject	2020-03-05 10:06	文件夹	
settings	2020-03-05 10:06	文件夹	
SolutionDataIO	2020-03-05 9:32	文件夹	
SolverControl	2020-03-05 10:06	文件夹	
XGenerateReport	2020-03-05 10:06	文件夹	
cgns.pri	2020-03-05 9:29	Qt Project Includ...	1 KB
Copy_DLLs.bat	2020-03-05 9:28	Windows 批处理...	22 KB
Copy_Pys.bat	2020-03-05 9:28	Windows 批处理...	1 KB
Copy_Pys.sh	2020-03-05 9:28	SH 文件	1 KB
Copy_SOs.sh	2020-03-05 9:28	SH 文件	17 KB
Create_Linux64_Project.sh	2020-03-05 9:28	SH 文件	2 KB
Create_X64_Project.bat	2020-03-05 9:28	Windows 批处理...	3 KB
FastCAE.sdf	2020-03-10 17:42	SQL Server Com...	115,456 KB
FastCAE.sln	2020-03-05 11:21	Microsoft Visual...	34 KB
FastCAE.v12.suo	2020-03-10 17:42	Visual Studio Sol...	117 KB
log.docx	2020-03-05 9:29	Microsoft Word ...	36 KB
occ.pri	2020-03-05 9:29	Qt Project Includ...	22 KB
PluginMotorBike.zip	2020-03-10 17:40	360压缩 ZIP 文件	849 KB
python.pri	2020-03-05 9:29	Qt Project Includ...	1 KB
ReadMe.txt	2020-03-05 9:29	文本文档	2 KB
Run_MSVC.bat	2020-03-05 9:29	Windows 批处理...	1 KB
Translate.bat	2020-03-05 9:28	Windows 批处理...	2 KB
vtk.pri	2020-03-05 9:29	Qt Project Includ...	34 KB

插件代码存放目录

项目文件夹名为 PluginMotorBike，与 FastCAE 其他项目放到相同路径，此项目源码可从网站下载，编译后做为 FastCAE 的一个插件，实现求解器参数文件生成和求解计算后文件的转换。



加载插件

4.4.1 头文件说明

我们的插件继承自插件基类 PluginBase, 因此基类的纯虚函数需要在我们我们的插件类中实现。这里我们把我们的插件类名字命名为 PluginOpenFoamExtend。此时类所在的头文件 PluginOpenFoamExtend.h 内容如下:

```
namespace Plugins
{
static GUI::MainWindow* mMainWindow;

class OPENFOAMPLUGINAPI PluginOpenFoamExtend:public PluginBase
{
Q_OBJECT
public:
PluginOpenFoamExtend(GUI::MainWindow* m);
PluginOpenFoamExtend();
//加载插件
bool install() override;
//卸载插件
bool uninstall() override;
public:
```

```

static int invaluse;

static GUI::MainWindow* getMainWindow();

private:
//发送控制台消息
void sendMessageToConsole(QString hint);

signals:
void treePrintMessageToMessageWindowSignal(ModuleBase::Message message);

};

}

extern "C"

{

void          OPENFOAMPLUGINAPI          Register(GUI::MainWindow*m,
QList<Plugins::PluginBase*>*plugins);

void showmsg(QString inputtext);

//求解器文件写出，给求解器使用
bool OPENFOAMPLUGINAPI WriteOut(QString path, ModelData::ModelDataBase*d);

//求解完成后的文件转换
bool OPENFOAMPLUGINAPI transfer(QString);

//复制字典文件到 case 目录下
bool OPENFOAMPLUGINAPI copyDirectoryFiles(const QString &fromDir, const QString
&toDir, bool coverFileIfExist);

//求解计算完成后复制后处理文件到 case 下目录
bool OPENFOAMPLUGINAPI transfileToDest(const QString &fromDir, const QString &toDir,
int type);

//发送消息到控制台
void OPENFOAMPLUGINAPI sendMessageToConsole(QString hintInfo);

//生成 openfoam 字典里读取的文本文件
bool          OPENFOAMPLUGINAPI          writeinputText(QString          filename,
ModelData::ModelDataBase*d);

}

```

4.4.2 C++ 文件说明

此时类所在的 C++ 文件 PluginOpenFoamExtend.cpp 内容如下：

```
namespace Plugins
{
    PluginOpenFoamExtend::PluginOpenFoamExtend(GUI::MainWindow* m)
    {
        mMainWindow = m;
        _describe = "OpenFoam plugin";
    }

    bool PluginOpenFoamExtend::install()
    {
        //注册求解器输入文件生成函数
        IO::IOConfigure::RegisterInputFile("OpenFoamRollDamp", WriteOut);
        //注册求解完成后文件转换函数
        IO::IOConfigure::RegisterOutputTransfer("OpenFoamFileTrans", transfer);
        return true;
    }

    bool PluginOpenFoamExtend::uninstall()
    {
        IO::IOConfigure::RemoveInputFile("OpenFoamRollDamp");
        return true;
    }

    GUI::MainWindow* PluginOpenFoamExtend::getMainWindow()
    {
        return mMainWindow;
    }
}

void Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* ps)
{

```

```

        Plugins::PluginBase* p = new Plugins::PluginOpenFoamExtend(m);
        ps->append(p);
    }
    bool WriteOut(QString path, ModelData::ModelDataBase* d)
    {
        //生成输入文件
        return true;
    }
    bool transfer(QString path)
    {
        //求解完成后把后处理 vtk 文件复制到 result 目录下
    }
    bool OPENFOAMPLUGINAPI copyDirectoryFiles(const QString &fromDir, const QString
    &toDir, bool coverFileIfExist)
    {
        //新建算例时复制字典文件到 case 目录下
    }
    void OPENFOAMPLUGINAPI sendMessageToConsole(QString hintInfo)
    {
        //发送消息到控制台
        GUI::MainWindow* mainwindow=nullptr;
        mainwindow = Plugins::PluginOpenFoamExtend::getMainWindow();
        if (mainwindow == nullptr) return;
        ModuleBase::Message message;
        message.type = ModuleBase::MessageType::Warning_Message;
        message.message = hintInfo;
        emit mainwindow->printMessageToMessageWindow(message);
    }
    bool OPENFOAMPLUGINAPI writeinputText(QString filename, ModelData::ModelDataBase*d)
    {

```

```

        //生成关联字典文件的输入参数文件
    }

    bool OPENFOAMPLUGINAPI transfileToDest(const QString &fromDir, const QString &toDir,
    int type)
    {
        //复制后处理文件到指定目录
    }

    在基类 PluginBase 中有一个 Protected 成员变量 QString _describe, 它是用于描述当前
    插件信息的字符串变量。这里我们在 FluentPlugin 类的构造函数中对其赋值。

    _describe = "OpenFoam plugin";

```

4.4.3 插件主要函数说明

当前 FastCAE2.0 插件支持的输入输出扩展函数包括:

```

//注册写出的后缀与方法

static void RegisterInputFile(QString suffix, WRITEINPFILE fun);

//注册结果文件转换方法

static void RegisterOutputTransfer(QString name, TRANSFEROUTFILE fun);

```

需要注意的是 RegisterInputFile 函数注册完成后, 当在求解器设置成我们所扩展的格式时, 该函数会在求解器求解之前进行调用。而 RegisterOutputTransfer 函数注册完成后, 会在完成求解器求解生成结果文件后调用。

根据我们的需求我们需要在**求解器求解之前**完成求解器输入文件 initparameter 生成。因此我们选择 RegisterInputFile 方法作为我们拓展的方法。因此我们在头文件中 C 函数部分添加:

```
bool OPENFOAMPLUGINAPI WriteOut(QString path, ModelData::ModelDataBase*d);
```

在 CPP 文件中添加:

```

bool WriteOut(QString path, ModelData::ModelDataBase* d)
{
    return true;
}

```

并在 WriteOut 函数中添加相应的功能代码, 具体代码可以参见本段落最终头文件和 CPP 文件全部内容部分。

根据我们的需求我们需要在**求解**完成后把结果文件复制到固定目录下用于后处理展示, 因此我们选择 RegisterOutputTransfer 方法作为我们拓展的方法。因此我们在头文件中 C 函数部分添加:

```
bool OPENFOAMPLUGINAPI transfer(QString);
```

在 CPP 文件中添加:

```
bool transfer(QString path)
{
    QStringList rowList;
    QString sourcePath = path + "/postProcessing/cuttingPlane";
    QString destPath = path + "/Result/CuttingPlane/";
    if (!transfileToDest(sourcePath, destPath, 0))
    {
        sendMessageToConsole(QStringLiteral("从 PostProcessing/cuttingPlane 目录复制文件到 Result/cuttingPlane 出错"));
        return false;
    }
    sourcePath = path + "/postProcessing/sets/streamLines/";
    destPath = path + "/Result/StreamLines/";
    if (!transfileToDest(sourcePath, destPath, 1))
    {
        sendMessageToConsole(QStringLiteral("从 PostProcessing/sets/streamLines 目录复制文件到 Result/cuttingPlane 出错"));
        return false;
    }
    QString fileName = path + "/postProcessing/forceCoeffs1/0/coefficient.dat";
    QFile file_y(fileName);
```

```
if (!file_y.exists())

    sendMessageToConsole

    (QStringLiteral("postProcessing/forceCoeffs1/0/目录下，不存在 coefficient.dat
文件转换出错"));

return false;

if (!file_y.open(QIODevice::ReadOnly))

    return false;

rowList.clear();

QString lineStr;

while (!file_y.atEnd())

{

    lineStr = file_y.readLine().trimmed();

    if (lineStr != "")

    {

        if (!lineStr.contains("# Time"))

        {

            continue;

        }

        lineStr = "Time Cm Cd Cl Cl(f) Cl(r)";

        rowList.append(lineStr);

        break;

    }

}

while (!file_y.atEnd())

{

    lineStr = file_y.readLine();

    lineStr = lineStr.replace(" ", " ");

    rowList.append(lineStr);

}

QString curFileName;
```



```

    curFileName = path + "/Result/coefficient.dat";

    QFile file(curFileName);

    if (!file.open(QIODevice::WriteOnly | QIODevice::Text))

        return false;

    QTextStream stream(&file);

    for (int i = 0; i < rowList.count(); i++)

    {

        stream << rowList.at(i) << endl;

    }

    return true;

}

```

并在 transfer 函数中添加相应的功能代码，具体代码可以参见本段落最终头文件和 CPP 文件全部内容部分。

4.4.4 插件全部代码

插件最终头文件内容：

```

#ifndef PLUGINOPENFOAMEXTEND_H
#define PLUGINOPENFOAMEXTEND_H

#include "OpenFoamPluginAPI.h"
#include "PluginManager/pluginBase.h"
#include "DataProperty/modelTreeItemType.h"
#include "mainWindow/mainWindow.h"
#include "DataProperty/ParameterTable.h"
#include "DataProperty/ParameterSelectable.h"
#include "DataProperty/ParameterInt.h"
#include "DataProperty/ParameterBool.h"
#include "DataProperty/ParameterDouble.h"

namespace IO
{

    class SolverIO;

}

```

```
class IOConfigure;

namespace ModelData
{
    class ModelDataBase;
}

namespace GUI
{
    class MainWindow;
}

namespace Plugins
{
    static GUI::MainWindow* mMainWindow;

    class OPENFOAMPLUGINAPI PluginOpenFoamExtend:public PluginBase
    {
        Q_OBJECT

    public:
        PluginOpenFoamExtend(GUI::MainWindow* m);

        PluginOpenFoamExtend();

        //加载插件

        bool install() override;

        //卸载插件

        bool uninstall() override;

    public:
        static int inval;

        static GUI::MainWindow* getMainWindow();

    private:
        //发送控制台消息

        void sendMessageToConsole(QString hint);

    signals:
        void treePrintMessageToMessageWindowSignal(ModuleBase::Message message);
    };
}
```

```

};

}

extern "C"

{

    void OPENFOAMPLUGINAPI Register(GUI::MainWindow*m,
QList<Plugins::PluginBase*>*plugins);

    void showmsg(QString inputtext);

    //求解器文件写出，给求解器使用

    bool OPENFOAMPLUGINAPI WriteOut(QString path, ModelData::ModelDataBase*d);

    //求解完成后的文件转换

    bool OPENFOAMPLUGINAPI transfer(QString);

    //复制字典文件到 case 目录下

    bool OPENFOAMPLUGINAPI copyDirectoryFiles(const QString &fromDir, const QString
&toDir, bool coverFileIfExist);

    //求解计算完成后复制后处理文件到 case 下目录

    bool OPENFOAMPLUGINAPI transfileToDest(const QString &fromDir, const QString
&toDir, int type);

    //发送消息到控制台

    void OPENFOAMPLUGINAPI sendMessageToConsole(QString hintInfo);

    //生成 openfoam 字典里读取的文本文件

    bool OPENFOAMPLUGINAPI writeinputText(QString filename,
ModelData::ModelDataBase*d);

    QString getParameterValue(QString paraname, ModelData::ModelDataBase*d);

    DataProperty::ParameterTable* getParameterTable(QString paraname,
ModelData::ModelDataBase*d);

    DataProperty::ParameterSelectable* getParameterSelectable(QString paraname,
ModelData::ModelDataBase* d);

}

#endif // PLUGINOPENFOAMEXTEND_H

```

插件最终 CPP 文件内容:

```
#include "PluginOpenFoamExtend.h"

class IOConfigure;

#include <QDebug>

#include "mainWindow/mainWindow.h"

#include "ModelData/modelDataFactory.h"

#include "ModelData/modelDataSingleton.h"

#include "ModelData/modelDataBase.h"

#include "IO/IOConfig.h"

#include <QDir>

#include <QApplication>

#include <QFile>

#include <vtkSTLWriter.h>

#include <vtkSmartPointer.h>

#include <QTextCodec>

#include "meshData/meshSingleton.h"

#include "meshData/meshKernal.h"

#include <vtkDataSet.h>

#include <QTextStream>

#include <QTextCodec>

#include "IO/SolverIO.h"

#include <QtMath>

#include <QMessageBox>

namespace GUI

{

    class MainWindow;

}

namespace Plugins

{
```

```

PluginOpenFoamExtend::PluginOpenFoamExtend(GUI::MainWindow* m)
{
    mMainWindow = m;
    _describe = "OpenFoam plugin";
}

bool PluginOpenFoamExtend::install()
{
    IO::IOConfigure::RegisterInputFile("OpenFoamRollDamp", WriteOut);
    IO::IOConfigure::RegisterOutputTransfer("OpenFoamFileTrans", transfer);

    return true;
}

bool PluginOpenFoamExtend::uninstall()
{
    IO::IOConfigure::RemoveInputFile("OpenFoamRollDamp");
    IO::IOConfigure::RemoveInputFile("OpenFoamFileTrans");
    return true;
}

GUI::MainWindow* PluginOpenFoamExtend::getMainWindow()
{
    return mMainWindow;
}

}

void Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* ps)
{
    //Plugins::PluginOpenFoamExtend::invalue = 1;
}

```

```

        Plugins::PluginBase* p = new Plugins::PluginOpenFoamExtend(m);

        ps->append(p);
    }

    bool WriteOut(QString path, ModelData::ModelDataBase* d)
    {
        transfer(path);

        QString filename;

        QString casepath;

        casepath = d->getPath() + "/";

        qDebug() << d->getName();

        qDebug() << "writeTest";

        QString binpath = QCoreApplication::applicationDirPath();

        QDir dir;

        QString resultpath;

        resultpath = casepath + "Result/";

        if (!dir.exists(resultpath))
        {
            bool res = dir.mkpath(resultpath);
        }

        if (!copyDirectoryFiles(binpath + "/../solver/dictionary", casepath, true))
        {
            sendMessageToConsole(QObject::tr("Copy dictionary fail,please check!"));

            return false;
        }

        if (!writeinputText((casepath + "initparameter"),d))
        {
            return false;
        }

        return true;
    }
}

```

```
bool transfer(QString path)
{
    QStringList rowList;

    QString sourcePath = path + "/postProcessing/cuttingPlane";
    QString destPath = path + "/Result/CuttingPlane/";
    if (!transfileToDest(sourcePath, destPath, 0))
    {
        sendMessageToConsole(QStringLiteral("从 PostProcessing/cuttingPlane 目录复制文件到 Result/cuttingPlane 出错"));
        return false;
    }
    sourcePath = path + "/postProcessing/sets/streamLines/";
    destPath = path + "/Result/StreamLines/";
    if (!transfileToDest(sourcePath, destPath, 1))
    {
        sendMessageToConsole(
            QStringLiteral("从 PostProcessing/sets/streamLines 目录复制文件到 Result/cuttingPlane 出错"));
        return false;
    }
    QString fileName = path + "/postProcessing/forceCoeffs1/0/coefficient.dat";
    QFile file_y(fileName);
    if (!file_y.exists())
        sendMessageToConsole(
            QStringLiteral("postProcessing/forceCoeffs1/0/目录下, 不存在 coefficient.dat 文件转换出错"));
        return false;

    if (!file_y.open(QIODevice::ReadOnly))
        return false;
```

```

rowList.clear();

QString lineStr;
while (!file_y.atEnd())
{
    lineStr = file_y.readLine().trimmed();
    if (lineStr != "")
    {
        if (!lineStr.contains("# Time"))
        {
            continue;
        }
        lineStr = "Time      Cm      Cd      Cl      Cl(f)
Cl(r)";

        rowList.append(lineStr);
        break;
    }
}

while (!file_y.atEnd())
{
    lineStr = file_y.readLine();
    lineStr = lineStr.replace(" ", " ");
    rowList.append(lineStr);
}

QString curFileName;
curFileName = path + "/Result/coefficient.dat";
QFile file(curFileName);
if (!file.open(QIODevice::WriteOnly | QIODevice::Text))return false;
QTextStream stream(&file);
for (int i = 0; i < rowList.count(); i++)
{

```



```

        stream << rowList.at(i) << endl;

    }

    return true;
}

bool OPENFOAMPLUGINAPI copyDirectoryFiles(const QString &fromDir, const QString
&toDir, bool coverFileIfExist)
{
    QDir sourceDir(fromDir);
    QDir targetDir(toDir);

    if (!targetDir.exists()){    /**< 如果目标目录不存在，则进行创建 */
        if (!targetDir.mkdir(targetDir.absolutePath()))
            return false;
    }

    QFileInfoList fileInfoList = sourceDir.entryInfoList();
    foreach(QFileInfo fileInfo, fileInfoList){
        if (fileInfo.fileName() == "." || fileInfo.fileName() == "..")
            continue;

        if (fileInfo.isDir())
        {    /**< 当为目录时，递归的进行 copy */
            if (!copyDirectoryFiles(fileInfo.filePath(),
                targetDir.filePath(fileInfo.fileName()), coverFileIfExist))
                return false;
        }

        else{    /**< 当允许覆盖操作时，将旧文件进行删除操作 */

            if (coverFileIfExist && targetDir.exists(fileInfo.fileName())){
                targetDir.remove(fileInfo.fileName());

                /// 进行文件 copy
            }
        }
    }
}

```

```

        if (!QFile::copy(fileInfo.filePath(),
            targetDir.filePath(fileInfo.fileName()))){
            return false;
        }
    }

    return true;
}

void OPENFOAMPLUGINAPI sendMessageToConsole(QString hintInfo)
{
    GUI::MainWindow* mainwindow=nullptr;
    mainwindow = Plugins::PluginOpenFoamExtend::getMainWindow();
    if (mainwindow == nullptr) return;
    ModuleBase::Message message;
    message.type = ModuleBase::MessageType::Warning_Message;
    message.message = hintInfo;
    emit mainwindow->printMessageToMessageWindow(message);
}

bool OPENFOAMPLUGINAPI writeinputText(QString filename, ModelData::ModelDataBase*d)
{
    QString splitchar = " ";
    QString paravalue;
    QStringList list;
    QString startTime;
    QString endTime;

```

```
paravalue = getParameterValue(QStringLiteral("开始步"), d);
```

```
list.append(QStringLiteral("starttime%1%2").arg(splitchar).arg(paravalue));
```

```
paravalue = getParameterValue(QStringLiteral("结束步"), d);
```

```
list.append(QStringLiteral("endtime%1%2").arg(splitchar).arg(paravalue));
```

```
paravalue = getParameterValue(QStringLiteral("求解器名称"), d);
```

```
list.append(QStringLiteral("application%1%2").arg(splitchar).arg(paravalue));
```

```
paravalue = getParameterValue(QStringLiteral("文件生成间隔"), d);
```

```
list.append(QStringLiteral("writeInterval%1%2").arg(splitchar).arg(paravalue));
```

```
//blockmesh
```

```
DataProperty::ParameterTable* pNx = getParameterTable(QStringLiteral("各方向网格数量"), d);
```

```
if (pNx != nullptr)
```

```
{
```

```
    if (pNx->getRowCount() > 0 && pNx->getColumnCount() > 2)
```

```
    {
```

```
list.append(QStringLiteral("xgridnumber%1%2").arg(splitchar).arg(pNx->getValue(0, 0)));
```

```
list.append(QStringLiteral("ygridnumber%1%2").arg(splitchar).arg(pNx->getValue(0, 1)));
```

```
list.append(QStringLiteral("zgridnumber%1%2").arg(splitchar).arg(pNx->getValue(0, 2)));
```

```
    }
```

```
}
```

```

int rowcount;

DataProperty::ParameterTable* pNy = getParameterTable(QStringLiteral("顶点坐标"), d);

if (pNy != nullptr)
{
    rowcount = pNy->getRowCount();
    for (int i = 0; i < rowcount; i++)
    {
        double x, y, z;

        x = pNy->getValue(i, 0);
        y = pNy->getValue(i, 1);
        z = pNy->getValue(i, 2);

        list.append(QStringLiteral("x%1%2%3").arg(i + 1).arg(splitchar).arg(x));
        list.append(QStringLiteral("y%1%2%3").arg(i + 1).arg(splitchar).arg(y));
        list.append(QStringLiteral("z%1%2%3").arg(i + 1).arg(splitchar).arg(z));
    }
}

```

//decomposepardict

```

DataProperty::ParameterTable* px = getParameterTable(QStringLiteral("各方向分解数量"), d);

if (px != nullptr)
{
    if (px->getColumnCount()> 2&&px->getRowCount()>0)
    {
        list.append(QStringLiteral("nx%1%2").arg(splitchar).arg(px->getValue(0, 0)));
        list.append(QStringLiteral("ny%1%2").arg(splitchar).arg(px->getValue(0, 1)));
        list.append(QStringLiteral("nz%1%2").arg(splitchar).arg(px->getValue(0, 2)));
    }
}

```

```

    }

    paravalue = getParameterValue(QStringLiteral("分解域的总数"), d);
    list.append(QStringLiteral("core%1%2").arg(splitchar).arg(paravalue));

    DataProperty::ParameterSelectable *pMethod = getParameterSelectable(QStringLiteral("
分解方法"), d);
    if (pMethod != nullptr)
    {
        int index = pMethod->getCurrentIndex();

        QString methodstr;
        if (index == 0)
        {
            methodstr = "hierarchical";
        }
        else
        {
            methodstr = "ptscotch";
        }

        list.append(QStringLiteral("decomposemethod%1%2").arg(splitchar).arg(methodstr));
    }

    //snappyHexMeshDict:
    paravalue = getParameterValue(QStringLiteral("是否切割网格"), d);

    list.append(QStringLiteral("castellatedMesh%1%2").arg(splitchar).arg(paravalue));

    paravalue = getParameterValue(QStringLiteral("是否进行网格贴合"), d);

    list.append(QStringLiteral("snap%1%2").arg(splitchar).arg(paravalue));

```

```

paravalue = getParameterValue(QStringLiteral("添加网格边界层"), d);

list.append(QStringLiteral("addLayers %1%2").arg(splitchar).arg(paravalue));

paravalue = getParameterValue(QStringLiteral("最小表面细化等级"), d);

list.append(QStringLiteral("minlevel%1%2").arg(splitchar).arg(paravalue));

paravalue = getParameterValue(QStringLiteral("最大表面细化等级"), d);

list.append(QStringLiteral("maxlevel%1%2").arg(splitchar).arg(paravalue));

paravalue = getParameterValue(QStringLiteral("边界层添加迭代数"), d);

list.append(QStringLiteral("nLayerIter%1%2").arg(splitchar).arg(paravalue));

//controldict

QFile file(filename);
if (!file.open(QIODevice::WriteOnly | QIODevice::Text))return false;
QTextStream stream(&file);
for (int i = 0; i < list.count(); i++)
{
    stream << list.at(i)+";" << endl;
}
return true;
}

```

```

QString getParameterValue(QString paraname, ModelData::ModelDataBase*d)
{
    QString t;
    DataProperty::ParameterBase* p = d->getParameterByName(paraname);
    if (p != nullptr)
    {
        if (p->getParaType() == DataProperty::Para_String)
        {
            t= p->valueToString();
        }
        else if (p->getParaType() == DataProperty::Para_Int)
        {
            DataProperty::ParameterInt* pInt=(DataProperty::ParameterInt*)(p);
            t = QString("%1").arg(pInt->getValue());
        }
        else if (p->getParaType() == DataProperty::Para_Double)
        {
            DataProperty::ParameterDouble* pDouble
(DataProperty::ParameterDouble*)(p);
            t = QString("%1").arg(pDouble->getValue());
        }
        else if (p->getParaType() == DataProperty::Para_Bool)
        {
            DataProperty::ParameterBool* pBool = (DataProperty::ParameterBool*)(p);
            return pBool->valueToString();
            if (pBool->getValue())
            {
                t = QString("%1").arg("true");
            }
        }
    }
}

```

```

        else
        {
            t = QString("%1").arg("false");
        }

    }

    return t;
}

else
{
    return "0.0000";
}
}

```

```

DataProperty::ParameterTable*      getParameterTable(QString      paraname,
ModelData::ModelDataBase*d)

```

```

{
    DataProperty::ParameterTable* result = nullptr;

    DataProperty::ParameterBase* p = d->getParameterByName(paraname);

    if (p != nullptr)
    {
        if (p->getParaType() == DataProperty::Para_Table)
        {
            result = (DataProperty::ParameterTable*)(p);
        }
    }

    return result;
}

```

```

DataProperty::ParameterSelectable*      getParameterSelectable(QString      paraname,

```



```
ModelData::ModelDataBase* d)
```

```
{
```

```
    DataProperty::ParameterSelectable* result = nullptr;
```

```
    DataProperty::ParameterBase* p = d->getParameterByName(paraname);
```

```
    if (p != nullptr)
```

```
    {
```

```
        if (p->getParaType() == DataProperty::Para_Selectable)
```

```
        {
```

```
            result = (DataProperty::ParameterSelectable*)(p);
```

```
        }
```

```
    }
```

```
    return result;
```

```
}
```

```
void showmsg(QString inputtext)
```

```
{
```

```
    QMessageBox msgBox;
```

```
    msgBox.setText(inputtext);
```

```
    msgBox.exec();
```

```
}
```

```
bool OPENFOAMPLUGINAPI transfileToDest(const QString &fromDir, const QString &toDir,  
int type)
```

```
{
```

```
    int i = 0;
```

```
    QStringList dirlist;
```

```
    QDir sourceDir(fromDir);
```

```
    sourceDir.setFilter(QDir::Dirs);
```

```
    sourceDir.setSorting(QDir::Name);
```

```
    QDir targetDir(toDir);
```

```
if (!targetDir.exists()){    /**< 如果目标目录不存在，则进行创建 */
    if (!targetDir.mkdir(targetDir.absolutePath()))
        return false;
}

QString destFileName;
QFileInfoList fileInfoList = sourceDir.entryInfoList();
int fileno = 0;
foreach(QFileInfo fileInfo, fileInfoList)
{
    if (fileInfo.fileName() == "." || fileInfo.fileName() == "..")
        continue;

    QString dirname = fileInfo.filePath();
    if (fileInfo.isDir())
    {
        QDir fileDir(fileInfo.filePath());

        fileDir.setFilter(QDir::Files);

        QFileInfoList fileList = fileDir.entryInfoList();

        destFileName = "";

        for (int j = 0; j < fileList.count(); j++)
        {
            if (fileList.at(j).fileName() == "." || fileList.at(j).fileName() == "..") continue;
            if (type == 0)
            {
                if (fileList.at(j).fileName().contains("p_y"))
                {
```

```

        destFileName
        =
QString("%1%2%3%4").arg(toDir).arg("p").arg(fileno).arg(".vtk");

        fileno++;

    }

}

else

{

    if (fileList.at(j).fileName().contains("track0_p"))

    {

destFileName = QString("%1%2%3%4").arg(toDir).arg("streamline").arg(fileno).arg(".vtk");

        fileno++;

    }

}

if (targetDir.exists(fileList.at(j).fileName()))

{

    targetDir.remove(fileList.at(j).fileName());

}

QString filename;

filename = fileList.at(j).filePath();

if (!QFile::copy(fileList.at(j).filePath(), destFileName));

{

    //return false;

}

}

}

return true;

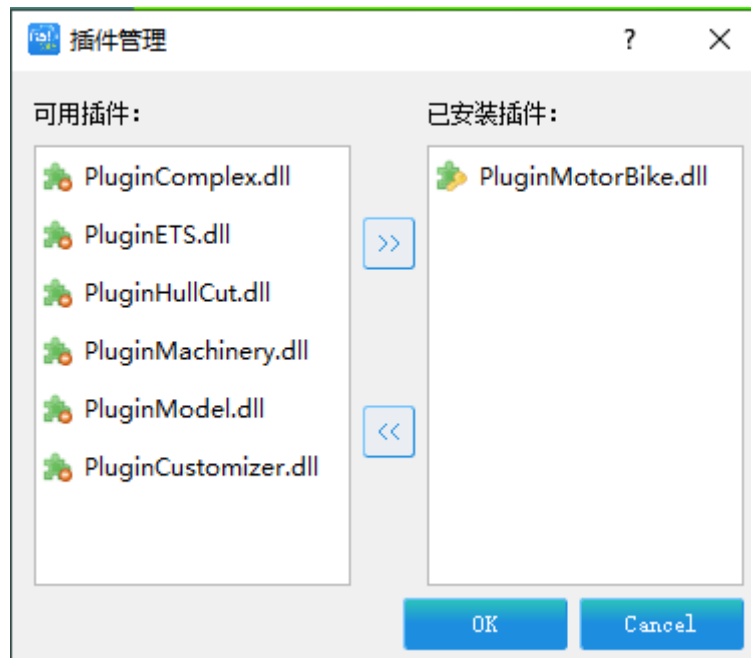
}

```

4.5 软件使用

4.5.1 插件加载

插件的加载，如下图所示：



加载插件

4.5.2 创建算例

鼠标左键点击【控制面板】-【计算分析】-【算例】节点，点击鼠标右键软件将弹出【算例】节点的右键菜单。



创建算例

1.点击【算例】节点弹出的右键菜单中的【创建算例】菜单项，软件会弹出创建算例对话框。这里我们使用默认的名称，直接点击按键完成算例的创建。如果有需要用户可以输入其他的算例名称。



2.此时算例下的树形结构如下图所示。点击【算例】下的【仿真参数设置】节点，我们可以看到仿真参数设置节点下的定制的参数。用户可以根据需求，自行调整参数值。



编辑参数

3.点击【算例】下的【求解设置】节点，我们可以看到求解设置节点下的定制的参数。用户可以根据需求，自行调整参数值。



编译求解参数

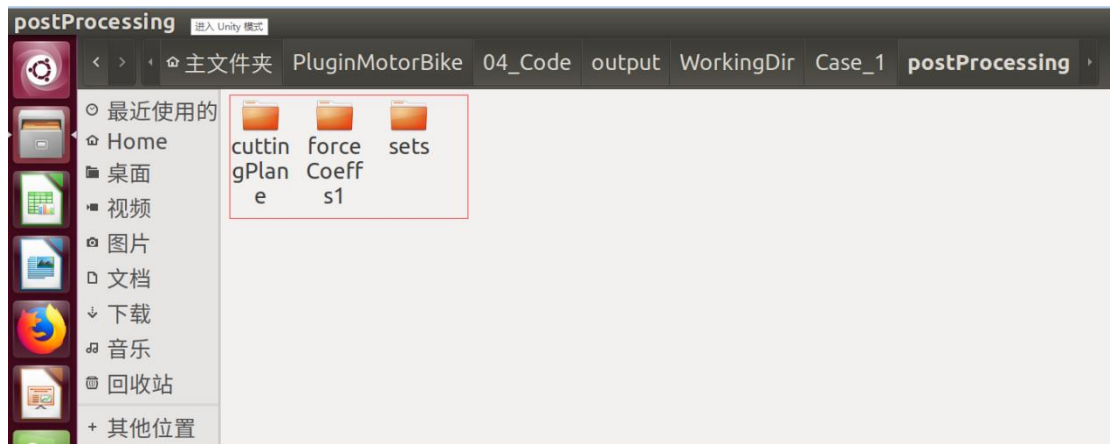
4.5.3 求解计算

点击工具栏的按钮，软件将启动求解。在求解器完成后用户可以按照生成 Python 脚本方式加载后处理结果文件进行显示。

4.5.4 后处理展示

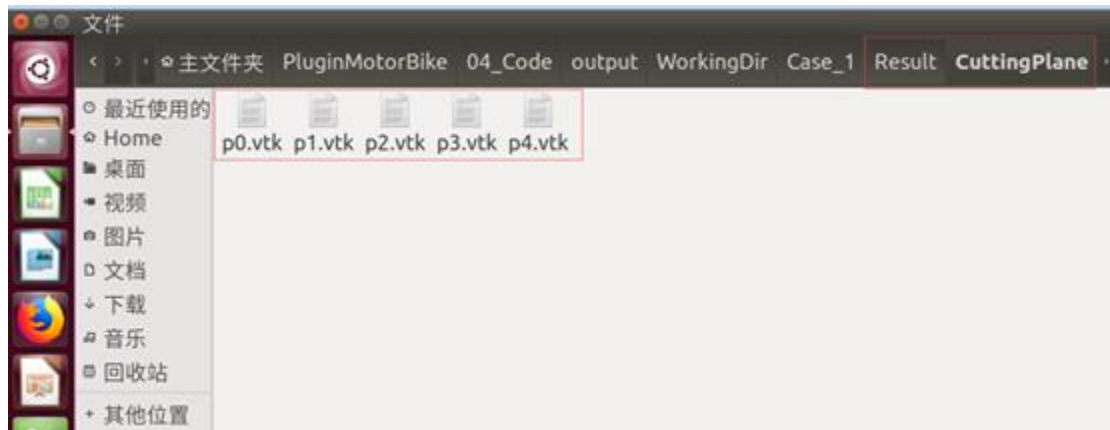
4.5.4.1 结果云图展示

本算例求解完成后，后处理文件默认保存在 postProcessing 目录下，cuttingPlane 和 sets 保存 VTK 结果文件，forceCoeffs1 保存二维曲线文件，如下图所示：



OpenFoam 后处理保存路径

由于这种目录结构不适合 FastCAE 集中动画展示，通过插件的 transfer 函数求解完成后将相关文件复制到算例路径下 Result 目录下，如下图所示：

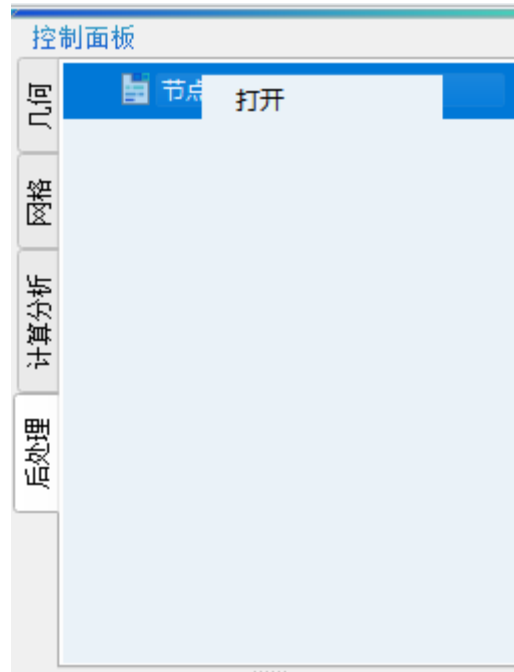


摩托结果文件

code > work > Project > VtkFrame > OpenFoam > FastCaeMotorBike > 04_Code > output > WorkingDir > Case_1 > Result > StreamLines			
名称	修改日期	类型	大小
streamline0.vtk	2020-03-06 17:31	VTK 文件	963 KB
streamline1.vtk	2020-03-06 17:39	VTK 文件	1,348 KB
streamline2.vtk	2020-03-06 17:46	VTK 文件	988 KB
streamline3.vtk	2020-03-06 17:53	VTK 文件	1,028 KB
streamline4.vtk	2020-03-06 18:00	VTK 文件	981 KB

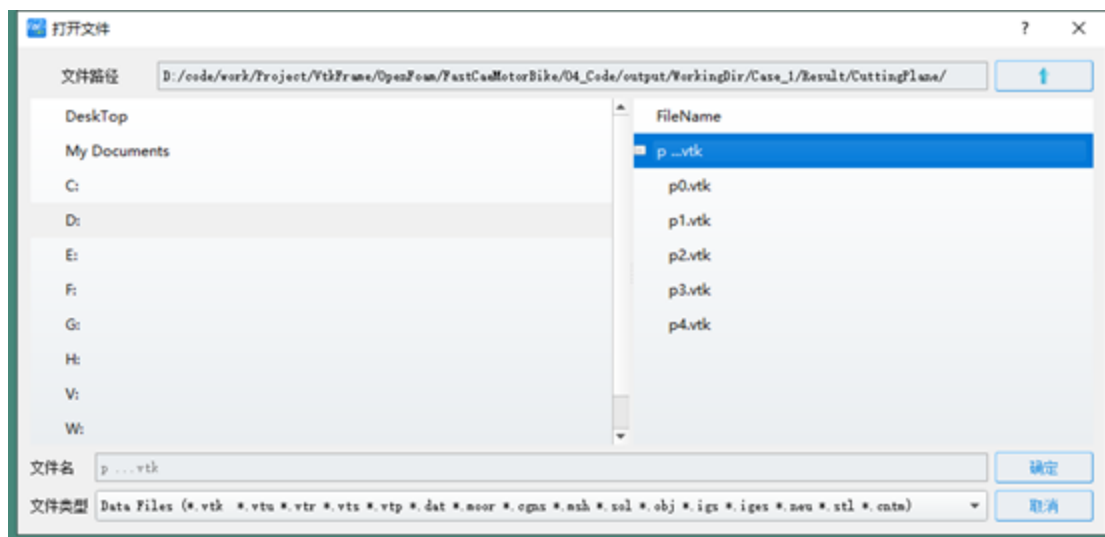
流线结果文件

新建算例后，我们打开后处理菜单->三维渲染窗体，点击“打开”，如下图所示：



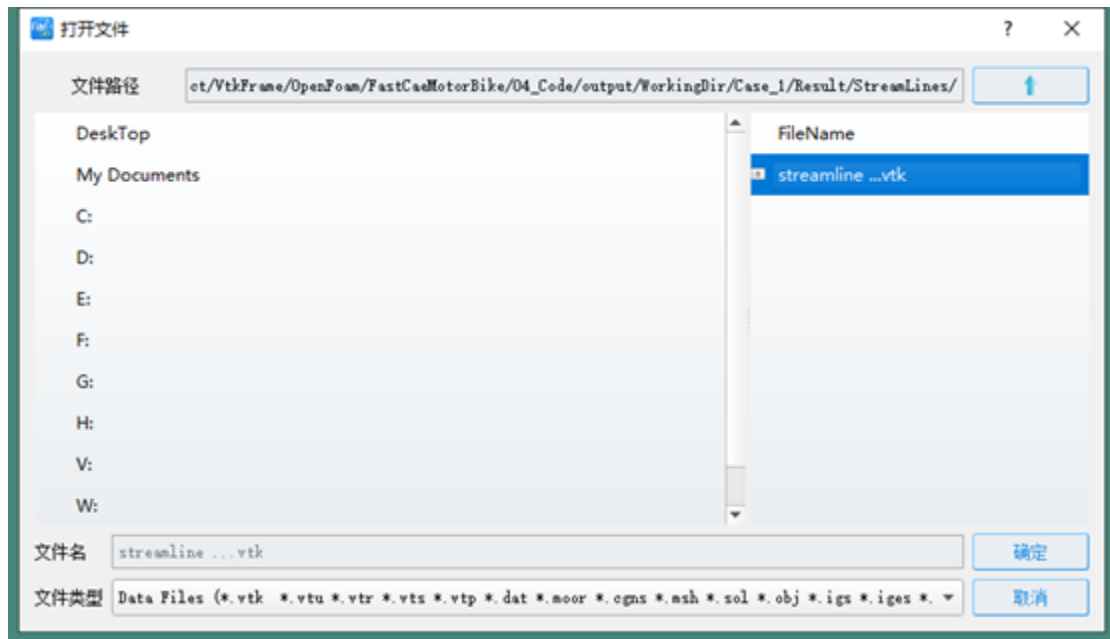
打开文件夹

选择一组文件加载到树型节点：



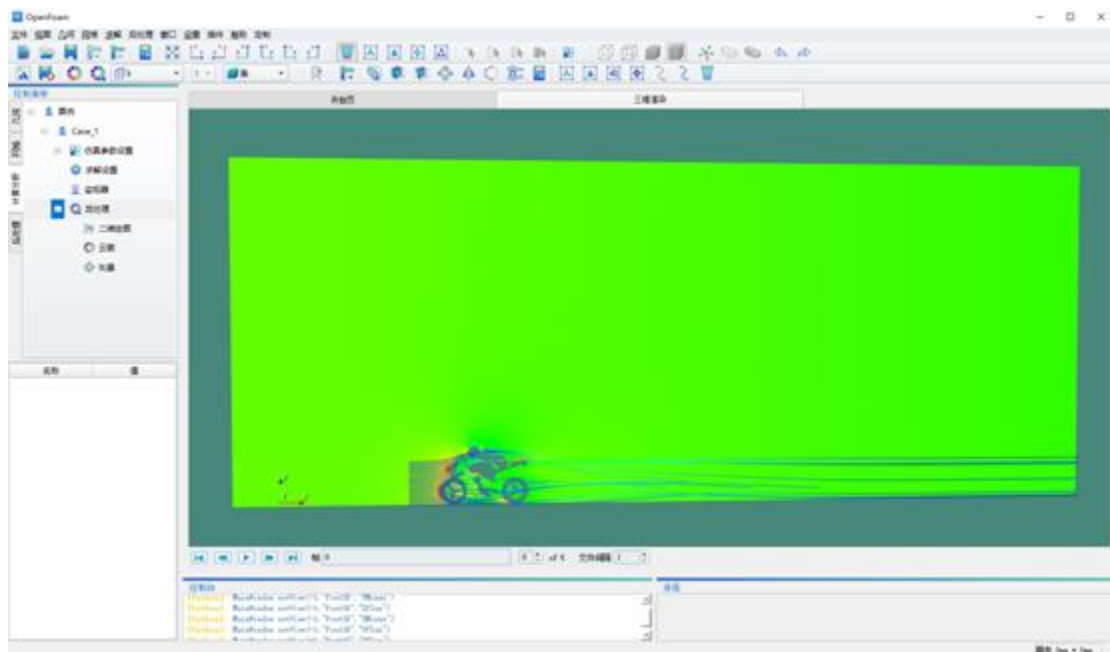
加载网格文件

再选择一组文件加到树型节点：



加载另一组结果文件

点击“应用”按钮，显示出图形。



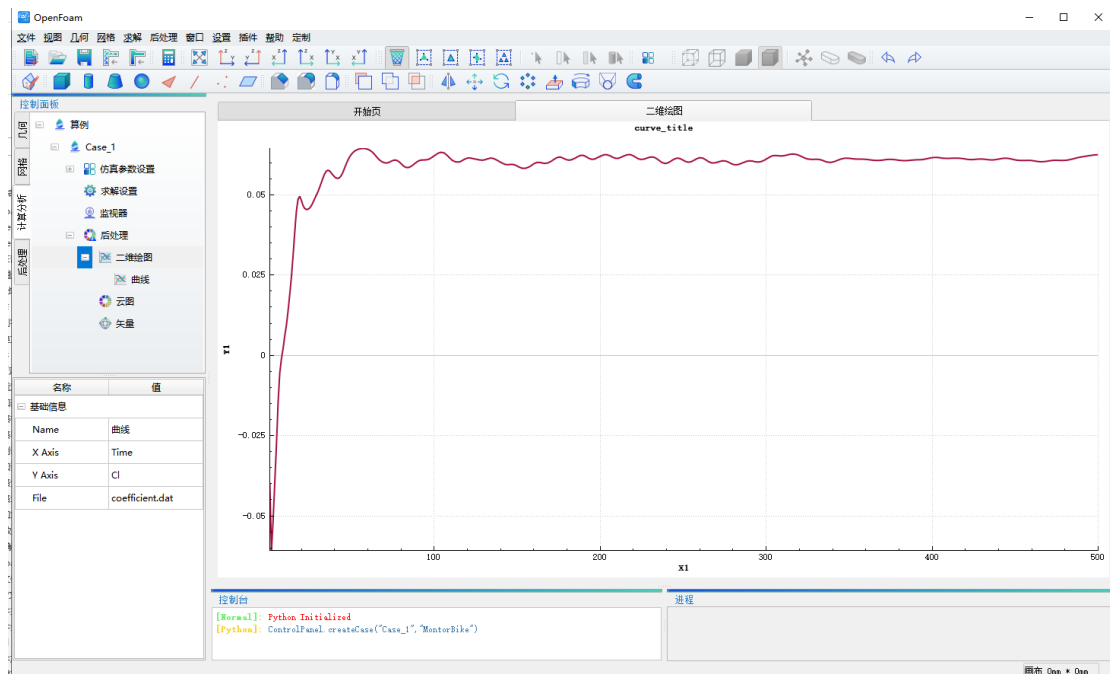
结果云图展示

4.5.4.2 二维曲线展示

在“二维绘图”节点上单击鼠标右键，显示出二维曲线，如下图所示：



显示二维曲线操作



二维曲线

Abaqus 集成示例

FastCAE-Abaqus 集成示例（插件拓展集成）

一、概述

1.1 案例介绍

本案例采用 FastCAE 的插件开发方式集成 Abaqus 模态分析功能，实现网格导入，仿真参数设置，求解参数设置，驱动 Abaqus 后台求解及结果可视化等流程。

本案例基于 FastCAE 提供的插件拓展机制，开发了 Auaqus 适配插件，实现了模态分析的全流程个性化定制，定义数据适配接口实现与 Abaqus 的数据传递，形成了基于 Abaqus 的专业定制模态分析软件。

本案例名称为 Abaqus 集成案例，其中所有资料包含源码和相关文档均可以在本站“[下载](#)”菜单栏目内进行下载。

1.2 集成目标

- 使用 FastCAE 集成 Abaqus 实现模态分析功能的重新封装，固化分析流程，减少参数接口。
- 通过本案例能够使用户快速了解使用 FastCAE2.0 进行插件开发方式的进行开发，为 Abaqus 软件集成与二次开发提供指导。

1.3 环境说明

- 开发环境
- Windows 10
- VS2013 Community(VS 2013 其他版本也可以)
- Qt 5.4.2-msvc-opengl-x64
- FastCAE2.0
- ABAQUS 6.14-4

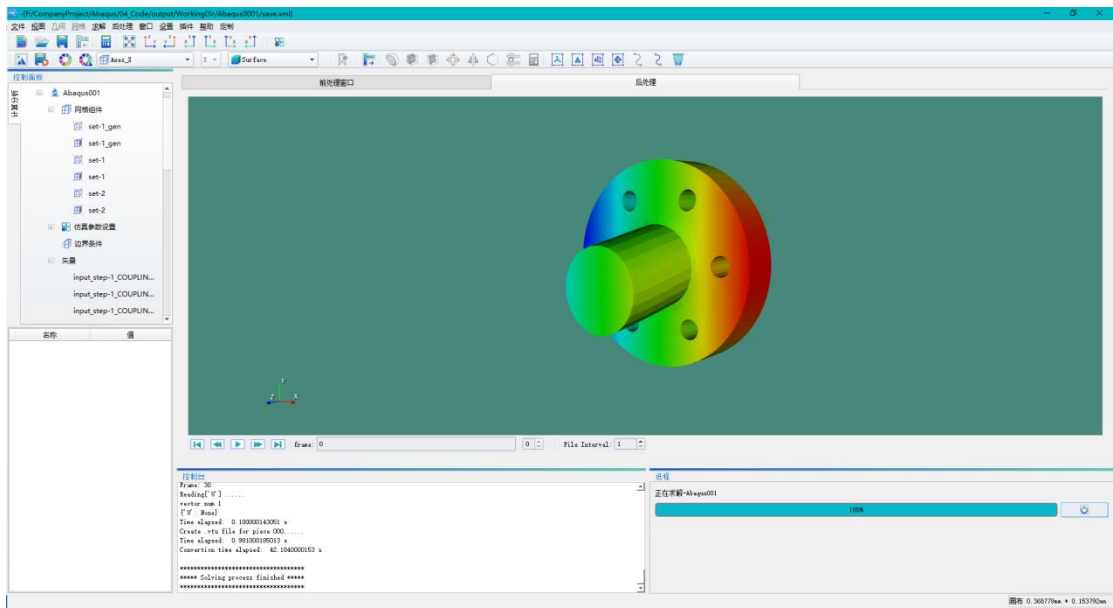
1.4 注意事项

- 建议开展本案例学习之前，要有一定 C++ 基础和 Abaqus 使用经验。
- 本次案例采用导入 Abaqus 中的 Inp 格式文件加载网格信息，并且在案例中尽量避

免在使用 FastCAE 过程中代码存放位置及文件存放位置中包括中文路径。

二、软件实现效果

本案例最终形成了基于 Abaqus 的专业定制模态分析软件，该软件数值计算由 Abaqus 程序实现，前后处理功能与流程通过 FastCAE 定制实现。前后处理的个性化流程定制以及前后处理与数值计算程序的数据交换通过 Abaqus 适配插件实现。软件界面如下图所示：

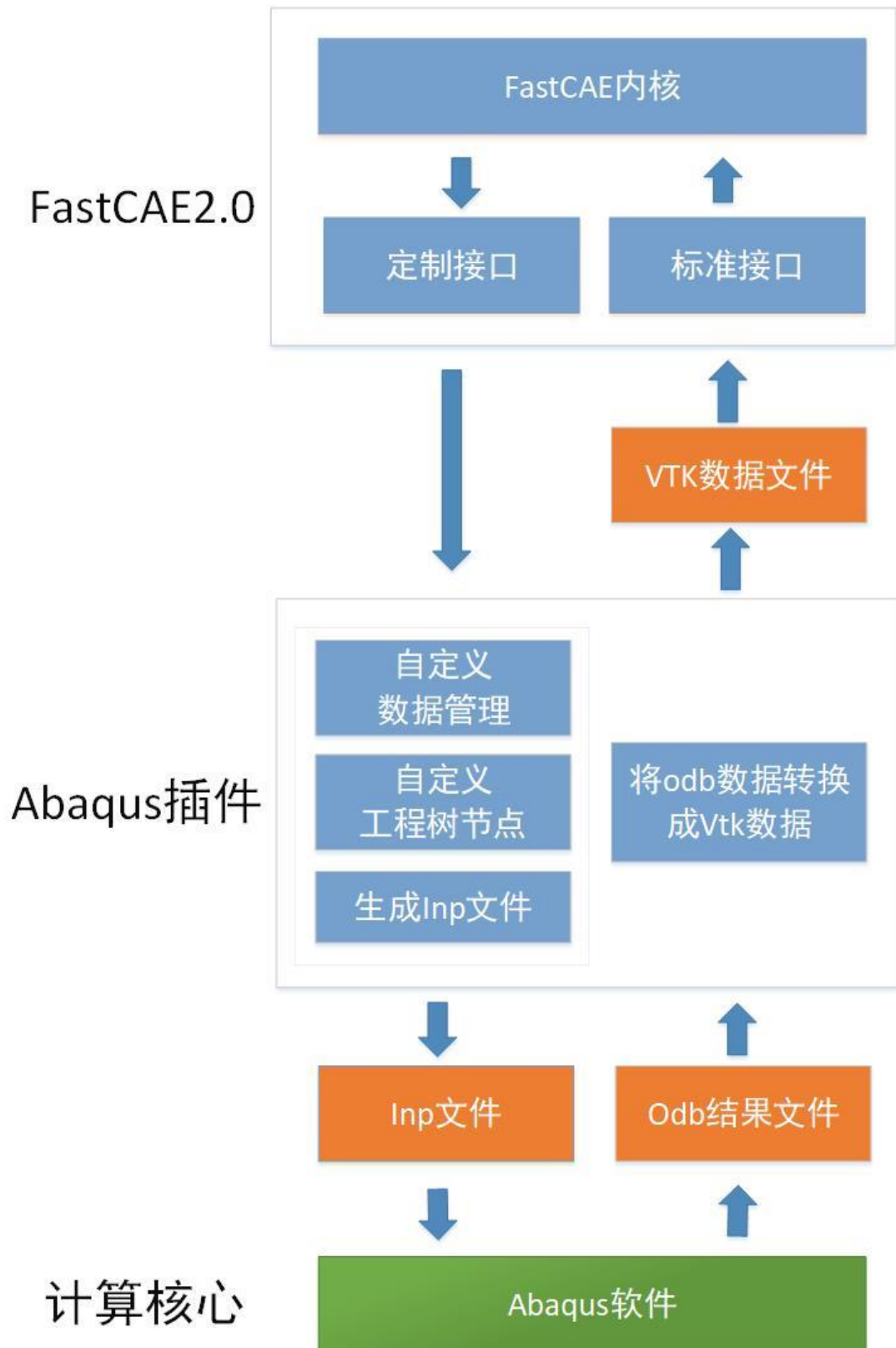


软件效果图

该专业定制的模态分析软件固化了分析流程，减少了大量的数据接口，在保证分析精度的前提下，降低了学习成本，提高了仿真分析的效率与可靠性。

三、原理说明

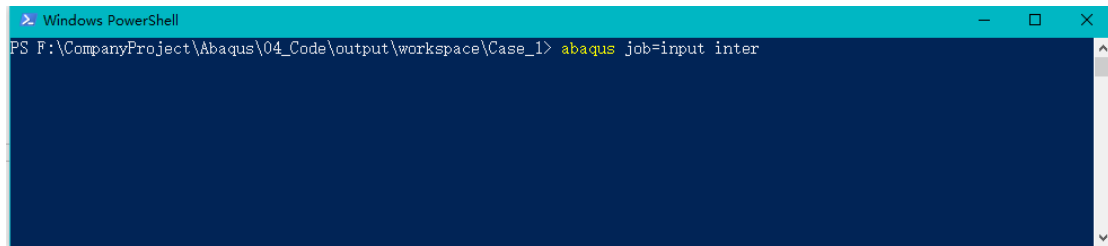
本案例的核心集成原理，通过 FastCAE 平台软件定制界面，并生成 Abaqus 运行所需的脚本文件，后台驱动 Abaqus 软件进行科学计算并对计算结果进行展示。



案例原理图

3.1 Abaqus 部分

- Abaqus 软件支持通过命令行的方式使用脚本进行启动运行。



Abaqus 命令

如上图，Abaqus 就会在后台加载 input.inp 文件进行运算。

Abaqus 命令行参数 job 表示提交分析作业。Inter 表示交互控制,显示启动计算情况。

3.2 FastCAE 部分

- 根据|原理说明 -|Abaqus 部分内容，我们可以分析得出|FastCAE 部分需要调用 Abaqus 通过传参的形式让 Abaqus 在后台运行，即我们在|FastCAE 部分需要完成软件参数的设置、网格的导入及展示、边界条件设置、inp 文件内容的生成的、后处理结果展示的工作。

其中，FastCAE 部分及**软件框架**将提供网格的导入及展示。这部分功能不需要用户重新开发，只需要进行配置就能够使用。软件参数的设置、网格的导入及展示、边界条件设置、后处理结果展示功能以及 inp 文件生成功能将由插件完成。

四、操作开发步骤

首先我们需要进行插件开发，将当前 FastCAE2.0 版本不支持的功能通过插件的形式进行开发，通过平台加载的形式完成这部分功能的支持。

通过插件定义一个自定义类型数据类，用来存储复杂数据。

通过插件定义重新定义工程树，满足流程需求。

将数据进行输出生成求解所需 Inp 格式文件。

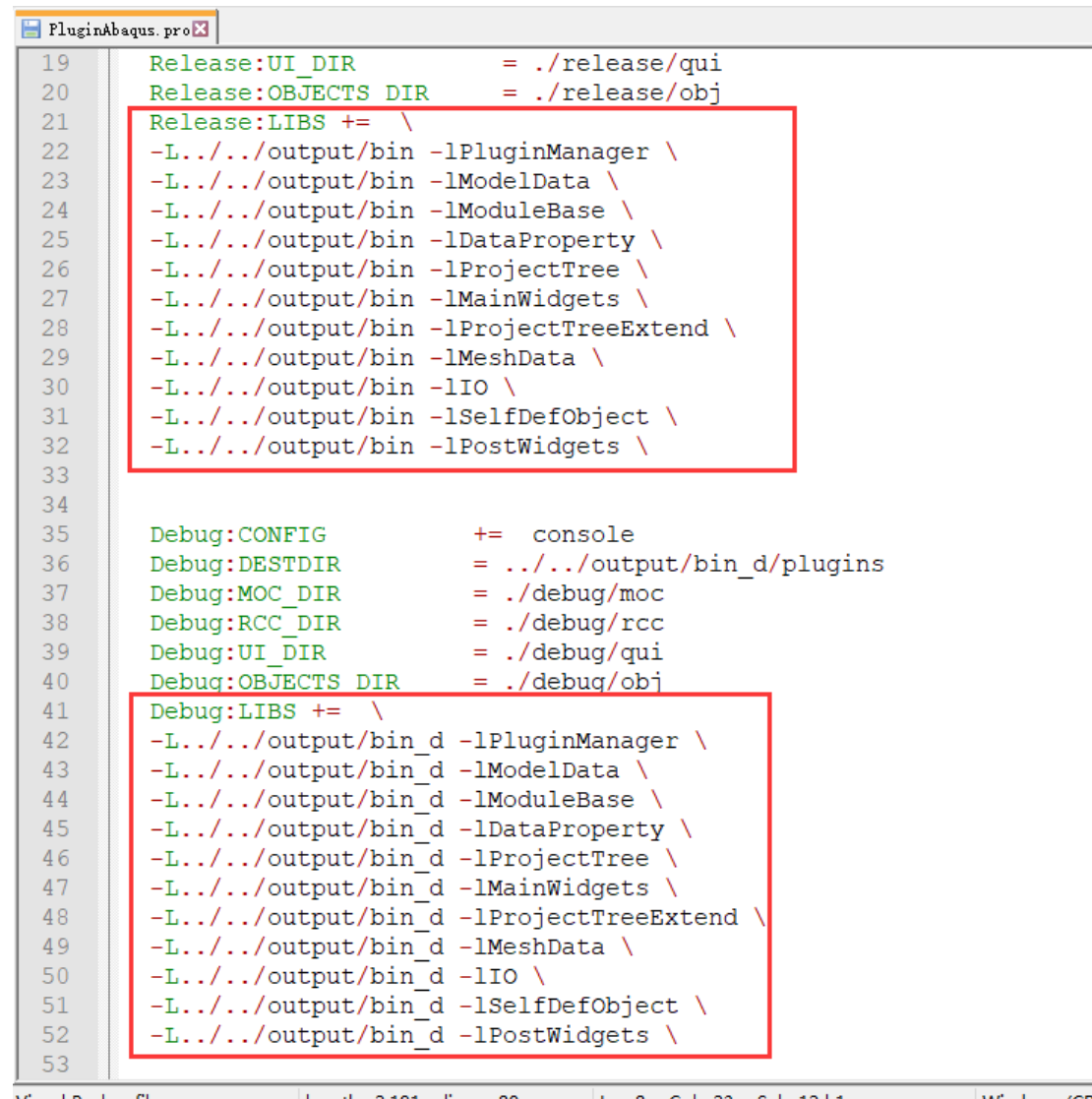
由于我们开发的插件中具有将 odb 转换成 vtk 功能，这就需要在完成插件开发后，根据需求需要引入 “odb2vtk.py”脚本文件。

然后我们需要按照我们软件的要求对软件的界面及功能进行定制。定制完成后，我们就可以开始我们所开发软件的使用旅程了！

4.1 Abaqus 插件开发

插件开发前准备

本次插件开发代码中，引用了许多 FastCAE 解决方案中的项目，需在 PluginAbaqus.pro 文件中声明引用其项目，如下图所示。



```
19 Release:UI_DIR           = ./release/qui
20 Release:OBJECTS_DIR      = ./release/obj
21 Release:LIBS += \
22 -L../output/bin -lPluginManager \
23 -L../output/bin -lModelData \
24 -L../output/bin -lModuleBase \
25 -L../output/bin -lDataProperty \
26 -L../output/bin -lProjectTree \
27 -L../output/bin -lMainWidgets \
28 -L../output/bin -lProjectTreeExtend \
29 -L../output/bin -lMeshData \
30 -L../output/bin -lIO \
31 -L../output/bin -lSelfDefObject \
32 -L../output/bin -lPostWidgets \
33
34
35 Debug:CONFIG             += console
36 Debug:DESTDIR            = ../output/bin_d/plugins
37 Debug:MOC_DIR            = ./debug/moc
38 Debug:RCC_DIR            = ./debug/rcc
39 Debug:UI_DIR             = ./debug/qui
40 Debug:OBJECTS_DIR        = ./debug/obj
41 Debug:LIBS += \
42 -L../output/bin_d -lPluginManager \
43 -L../output/bin_d -lModelData \
44 -L../output/bin_d -lModuleBase \
45 -L../output/bin_d -lDataProperty \
46 -L../output/bin_d -lProjectTree \
47 -L../output/bin_d -lMainWidgets \
48 -L../output/bin_d -lProjectTreeExtend \
49 -L../output/bin_d -lMeshData \
50 -L../output/bin_d -lIO \
51 -L../output/bin_d -lSelfDefObject \
52 -L../output/bin_d -lPostWidgets \
53
```

Pro 项目文件

PluginAbaqus 项目文件路径存放在 FastCAE 工程下，保持相对路径一致，如下图。

	BCBase	2020/3/11 16:50	文件夹
	ConfigOptions	2020/3/11 16:50	文件夹
	CurveAnalyse	2020/3/11 16:50	文件夹
	DataProperty	2020/3/11 16:50	文件夹
	DllsForDesigner	2019/11/28 11:13	文件夹
	geometry	2020/3/11 16:50	文件夹
	GeometryCommand	2020/3/13 9:29	文件夹
	GeometryWidgets	2020/3/13 9:29	文件夹
	Gmsh	2020/3/11 16:50	文件夹
	GraphicsAnalyse	2020/3/11 16:50	文件夹
	HeuDataSrcIO	2020/3/11 16:50	文件夹
	IO	2020/3/11 16:53	文件夹
	License	2020/3/11 16:50	文件夹
	main	2020/3/11 16:50	文件夹
	MainWidgets	2020/3/13 9:29	文件夹
	mainWindow	2020/3/13 12:17	文件夹
	Material	2020/3/11 16:50	文件夹
	meshData	2020/3/11 16:50	文件夹
	ModelData	2020/3/13 9:29	文件夹
	moduleBase	2020/3/11 16:50	文件夹
	ParaClassFactory	2020/3/11 16:50	文件夹
	PluginAbaqus	2020/3/11 10:52	文件夹
	PluginComplex	2019/11/28 11:49	文件夹
	PluginCustomizer	2020/3/11 16:11	文件夹
	PluginETS	2020/3/4 14:17	文件夹
	PluginFluent	2020/3/4 16:53	文件夹
	PluginHull	2020/3/11 16:11	文件夹

存放路径

插件通用部分代码修改

我们的插件继承自插件基类 PluginBase, 因此基类的纯虚函数需要在我们我们的插件类中实现。这里我们把我们插件类名字命名为 PluginBase。此时类所在的头文件 PluginBase.h 内容如下:

```
namespace Plugins
{
    class PLUGINABAQUSAPI PluginAbaqus : public PluginBase
    {
        Q_OBJECT
    }
}
```



```

public:
    PluginAbaqus(GUI::MainWindow* m);
    ~PluginAbaqus();

    //加载插件
    bool install() override;
    //卸载插件
    bool uninstall() override;
    //翻译
    void reTranslate(QString lang) override;
private:
    GUI::MainWindow* _mainwindow{};

};
}
extern "C"
{
    //插件注册
    void PLUGINABAQUSAPI Register
        (GUI::MainWindow* m, QList<Plugins::PluginBase*>* plugs);
}

```

此时类所在的 CPP 文件 PluginBase.cpp 内容如下：

```

void Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* ps)
{
    Plugins::PluginBase* p = new Plugins::PluginAbaqus(m);
    ps->append(p);
}
namespace Plugins

```

```

{
    PluginAbaqus::PluginAbaqus(GUI::MainWindow* m)
    {
        _mainwindow = m;
    }

    PluginAbaqus::~~PluginAbaqus()
    {
    }

    bool PluginAbaqus::install()
    {
        return true;
    }

    bool PluginAbaqus::uninstall()
    {
        return true;
    }
}

```

在基类 PluginBase 中有一个 Protected 成员变量 QString_describe，它是用于描述当前插件信息的字符串变量。这里我们在 PluginAbaqus 类的构造函数中对其赋值。

```
_describe = " Abaqus plugin ";
```

插件需要完成函数选择

当前 FastCAE2.0 插件支持的输入输出扩展函数包括:

```
//注册写出的后缀与方法
```

```
static void RegisterInputFile(QString suffix, WRITEINPFILE fun);
```

```
//注册结果文件转换方法
```

```
static void RegisterOutputTransfer(QString name, TRANSFEROUTFILE fun);
```

```
//注册网格文件导入的方法
```

```
static void RegisterMeshImporter(QString suffix, IMPORTMESHFUN fun);
```

//注册网格导出的方法

```
static void RegisterMeshExporter(QString suffix, EXPORTMESHFUN fun);
```

需要注意的是 RegisterInputFile 函数注册完成后，当在求解器设置成我们所扩展的格式时，该函数会在求解器求解之前进行调用。而 RegisterOutputTransfer 函数注册完成后，会在完成求解器求解生成结果文件后调用。RegisterMeshImporter 和 RegisterMeshExporter 方法是为新格式的网格文件导入导出使用的函数。

根据我们的需求，本次案例中使用了导入 inp 文件读取网格数据和我们需要在**求解器求解之前**完成 Inp 文件生成。因此我们选择 RegisterInputFile、RegisterMeshImporter 方法作为我们拓展的方法。因此我们在头文件中 C 函数部分添加：

//求解器文件写出

```
bool PLUGINABAQUSAPI WriteOut(QString path, ModelData::ModelDataBase* d);
```

在 CPP 文件中添加：

```
bool WriteOut(QString path, ModelData::ModelDataBase* d)
{
    qDebug() << d->getName();
    qDebug() << "writeTest";

    ModelData::AbaqusModel* model = static_cast<ModelData::AbaqusModel*>(d);

    QString strURL = model->getPath() + "/input.inp";
    InpWriteManager inp(model);
    inp.writeInit(strURL);

    strURL = model->getPath() + "/ODB2VTK+.cfg";
    ODB2VTKConfig cfg(model);
    cfg.writeConfig(strURL);

    QString sourcePath = QCoreApplication::applicationDirPath() + "../Solve";
```

```

QString sourceFile = sourcePath + "/ODB2VTK+.py";
QString targetFile = model->getPath() + "/ODB2VTK+.py";
QFile file(sourceFile);

if (file.exists())
{
    QFile::copy(sourceFile, targetFile);
}

QString strVtkUrl = model->getPath() + "/vtk";
QDir dir;
if (!dir.exists(strVtkUrl))
{
    dir.mkpath(strVtkUrl);
}

return true;
}

```

WriteOut 函数中,我们要完成求解前所需要准备的输出数据,其中包括了读取配置参数生成 inp 文件以及求解后生成的 odb 文件转换成 vtk 格式数据的配置文件,都在求解前生成。主要实现是通过 InpWriteManager 读取 AbaqusModel 中的数据,将其输出成 input.inp 文件,ODB2VTKConfig 读取 AbaqusModel 数据生成 python 脚本 ODB2VTK+.py 需要配置文件 ODB2VTK+.cfg。上述中 InpWriteManager、AbaqusModel、ODB2VTKConfig 会在本文后续中介绍。在 WriteOut 函数中指定了 ODB2VTK+.py 路径为 applicationDirPath() + "../Solve",也就是 output 文件夹下的 Solve 中,所以我们需要把 ODB2VTK+.py 拷贝改路径下。

根据我们的需求,我们需要自己维护一个复杂的数据类,用于记录 inp 中使用的相关参数。因此我们选择 registerType 方法注册自定义数据类。因此我们在头文件中 C 函数部分添加:

```
//创建数据类
```

```
bool PLUGINABAQUSAPI CreateModel(int t, QPair<int, ModelData::ModelDataBase*>* p);
```

在 CPP 文件中添加:

```
bool CreateModel(int t, QPair<int, ModelData::ModelDataBase*>* p)
{
    if (t == ProjectTreeType::PluginDefType + 101)
    {
        auto m = new ModelData::AbaqusModel((ProjectTreeType)t);
        p->first = t;
        p->second = m;
        return true;
    }
    return false;
}
```

其中 AbaqusModel 是自定义数据类，内容部分后续进行介绍。

接下来是重新定义工程树，通过选择 registerType 方法注册自定义工程树类。因此我们在头文件中 C 函数部分添加:

```
//创建树
bool PLUGINABAQUSAPI CreateTree
(int, GUI::MainWindow* m, QPair<int, ProjectTree::ProjectTreeBase*>*);
1
2
3
Plain Text
```

在 CPP 文件中添加:

```
bool CreateTree(int tp, GUI::MainWindow* m, QPair<int, ProjectTree::ProjectTreeBase*>* t)
{
```

```
    if (tp == PluginDefType + 101)
    {
        t->first = tp;

        t->second = new ProjectTree::AbaqusTree(m);

        return true;
    }

    return false;
}
```

插件功能开发

插件 PluginAbaqus 最终头文件内容：

```
#ifndef PLUGINABAQUS_H
#define PLUGINABAQUS_H

#include "PluginAbaqusAPI.h"
#include "PluginManager\pluginBase.h"
#include <QObject>

namespace GUI
{
    class MainWindow;
}

namespace ProjectTree
{
    class ProjectTreeBase;
}

namespace ModelData
{

```

```

class ModelDataBase;

}

namespace Plugins
{

    class PLUGINABAQUSAPI PluginAbaqus : public PluginBase
    {
        Q_OBJECT

    public:

        PluginAbaqus(GUI::MainWindow* m);

        ~PluginAbaqus();

        //加载插件

        bool install() override;

        //卸载插件

        bool uninstall() override;

        //翻译

        void reTranslate(QString lang) override;

    private:

        GUI::MainWindow* _mainwindow{};

    };

}

extern "C"
{

    //插件注册

    void PLUGINABAQUSAPI Register

        (GUI::MainWindow* m, QList<Plugins::PluginBase*>* plugs);

```

```

//创建数据类
bool PLUGINABAQUSAPI CreateModel(int t, QPair<int, ModelData::ModelDataBase*> *);

//创建树
bool PLUGINABAQUSAPI CreateTree
    (int, GUI::MainWindow* m, QPair<int, ProjectTree::ProjectTreeBase*> *);

//求解器文件写出
bool PLUGINABAQUSAPI WriteOut(QString path, ModelData::ModelDataBase* d);

//网格导入方法
bool PLUGINABAQUSAPI importMeshFun(QString filename);
}

#endif // PLUGINABAQUS_H

```

插件 PluginAbaqus 最终 CPP 文件内容：

```

#include "pluginabacus.h"
#include "mainWindow/mainWindow.h"
#include "ModelData/modelDataFactory.h"
#include "ModelData/modelDataSingleton.h"
#include "ModelData/modelDataBase.h"
#include "MainWidgets/ProjectTreeFactory.h"
#include "abaqustree.h"
#include "PluginAbaqus\abacusmodel.h"
#include "IO/IOConfig.h"
#include "qDebug"
#include "inpwritermanager.h"

```



```
#include "QProcess"

#include "odb2vtkconfig.h"

#include "QDir"

#include "QCoreApplication"

#include <QApplication>

#include <QTranslator>

#include <assert.h>


Plugins::PluginAbaqus::PluginAbaqus(GUI::MainWindow* m)
{
    _mainwindow = m;
    _describe = " Abaqus plugin ";
}


Plugins::PluginAbaqus::~PluginAbaqus()
{
}


bool Plugins::PluginAbaqus::install()
{
    ModelData::ModelDataFactory::registerType(PluginDefType + 101, "PluginAbaqus",
CreateModel);

    MainWidget::ProjectTreeFactory::registerType(PluginDefType + 101, CreateTree);
    IO::IOConfigure::RegisterInputFile("inp", WriteOut);

    return true;
}
```

```

bool Plugins::PluginAbaqus::uninstall()
{
    ModelData::ModelDataFactory::removeType(PluginDefType + 101);
    MainWidget::ProjectTreeFactory::removeType(PluginDefType + 101);
    IO::IOConfigure::RemoveInputFile("inp");

    return true;
}

void Plugins::PluginAbaqus::reTranslate(QString lang)
{
    auto app = static_cast<QApplication*>(QCoreApplication::instance());
    app->removeTranslator(_translator);
    if (lang.toLowerCase() == "chinese")
    {
        bool ok = _translator->load(":/translation/pluginabacus_zh");
        assert(ok);
        app->installTranslator(_translator);
    }
}

void Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* plugs)
{
    Plugins::PluginBase * p = new Plugins::PluginAbaqus(m);
    plugs->append(p);
}

bool CreateModel(int t, QPair<int, ModelData::ModelDataBase*>* p)

```

```

{

    if (t == ProjectTreeType::PluginDefType + 101)
    {
        auto m = new ModelData::AbaqusModel((ProjectTreeType)t);
        p->first = t;
        p->second = m;
        return true;
    }
    return false;
}

bool CreateTree(int tp, GUI::MainWindow* m, QPair<int, ProjectTree::ProjectTreeBase*>* t)
{
    if (tp == PluginDefType + 101)
    {
        t->first = tp;
        t->second = new ProjectTree::AbaqusTree(m);
        return true;
    }
    return false;
}

bool WriteOut(QString path, ModelData::ModelDataBase* d)
{
    qDebug() << d->getName();
    qDebug() << "writeTest";
}

```

```
ModelData::AbaqusModel* model = static_cast<ModelData::AbaqusModel*>(d);

QString strURL = model->getPath() + "/input.inp";
InpWriteManager inp(model);
inp.writeInit(strURL);

strURL = model->getPath() + "/ODB2VTK+.cfg";
ODB2VTKConfig cfg(model);
cfg.writeConfig(strURL);

QString sourcePath = QCoreApplication::applicationDirPath() + "../Solve";
QString sourceFile = sourcePath + "/ODB2VTK+.py";
QString targetFile = model->getPath() + "/ODB2VTK+.py";
QFile file(sourceFile);

if (file.exists())
{
    QFile::copy(sourceFile, targetFile);
}

QString strVtkUrl = model->getPath() + "/vtk";
QDir dir;
if (!dir.exists(strVtkUrl))
{
    dir.mkpath(strVtkUrl);
}

return true;
}
```

```
bool importMeshFun(QString)

{

    return true;

}
```

AbaqusModel 数据类开发

我们的自定义数据类继承自数据类扩展类 ModelDataBaseExtend, 因此基类的纯虚函数需要在我们自定义类中实现。头文件 AbaqusModel.h 内容如下:

```
#ifndef ABAQUSMODEL_H
#define ABAQUSMODEL_H

#include "PluginAbaqusAPI.h"
#include "ModelData/modelDataBaseExtend.h"
#include "stepdata.h"
#include "bcddata.h"

struct MaterialStruct
{
    QString strMaterialName = "";
    QString strSetName = "";
    QString strDensity = "";
    QString strModulus = "";
    QString strRatio = "";
};

namespace ModelData
{
```

```

class PLUGINABAQUSAPI AbaqusModel :public ModelDataBaseExtend
{
public:
    AbaqusModel(ProjectTreeType t);
    ~AbaqusModel();

    QHash<QString, StepData*>* getStepDataHash() const { return _stepDataHash; }
    QHash<QString, BCData*>* getBcDataHash() const { return _bcDataHash; }
    QList<QString>* getStepNameList() const { return _stepNameList; }
    MaterialStruct* getMaterialStruct() const { return _materialStruct; }

private:
    QList<QString>* _stepNameList = nullptr;
    QHash<QString, StepData*>* _stepDataHash = nullptr;
    QHash<QString, BCData*>* _bcDataHash = nullptr;
    MaterialStruct* _materialStruct = nullptr;

protected:
    QDomElement& writeToProjectFile(QDomDocument* doc, QDomElement* ele)
override;

    void readDataFromProjectFile(QDomElement* e) override;

private:
    void writeStepNameList(QDomDocument* doc, QDomElement* ele);
    void writeMaterialStruct(QDomDocument* doc, QDomElement* ele);
    void writeStepDataHash(QDomDocument* doc, QDomElement* ele);
    void writeBCDataHash(QDomDocument* doc, QDomElement* ele);

    void clearStepDataHash();
    void clearBCDataHash();

};

```

```
}  
  
#endif // ABAQUSMODEL_H
```

其中 StepData 是分析步类、BCData 边界参数类，内容会在后面介绍。因为继承数据类 ModelDataBaseExtend，需要实现基类的纯虚函数 writeToProjectFile、readDataFromProjectFile 实现保存项目参数和读取项目参数功能。

BCData 数据类开发

BCData 数据类记录边界相关数据参数类，主要记录边界名称、边界类型名称、分析步名称、组件名称以及边界类型。以及数据保存与读入功能的实现。

头文件内容：

```
#ifndef BCDATA_H  
  
#define BCDATA_H  
  
#include "PluginAbaqusAPI.h"  
#include "QString"  
#include <QDomElement>  
  
namespace BCEnumType  
{  
    enum SymmetryType  
    {  
        XSYMM,  
        YSYMM,  
        ZSYMM,  
        XASYMM,  
        YASYMM,  
        ZASYMM,  
        PINNED,  
        ENCASTRE,  
    }  
}
```

```
};  
  
}  
  
struct DisplacementStruct  
{  
    bool bU1 =false;  
    bool bU2 =false;  
    bool bU3 = false;  
    bool bUR1 = false;  
    bool bUR2 = false;  
    bool bUR3 = false;  
};  
  
class PLUGINABAQUSAPI BCData  
{  
public:  
    BCData();  
    ~BCData();  
  
private:  
    QString strBCName;//边界名称  
    QString strBCType;//边界类型  
    QString strStepName;//分析步名称  
    QString strSetName;//组件名称  
  
    DisplacementStruct displacement;  
    BCEnumType::SymmetryType symmetryType;  
};  
  
#endif // BCDATA_H
```


StepData 数据类开发

StepData 数据类记录分析步相关数据参数类，主要分析步名称、分析步类型、分析步(Frequency)基础数据以及分析步其他数据，以及数据保存与读入功能的实现。

头文件内容：

```
#ifndef STEPDATA_H
#define STEPDATA_H

#include "PluginAbaqusAPI.h"
#include "QString"
#include <QDomElement>

namespace StepEnumType
{
    enum NlgeomType
    {
        Off,
        On,
    };

    enum EigensolverType
    {
        Lanczos,
        Subspace,
        AMS,
    };

    enum EigenvaluesType
    {
        AllinFrequencyRange,
    };
}
```

```

        Value,
    };

    enum BlockSizeType
    {
        Default,
        ValueSize,
    };

    enum NormalizeType
    {
        Displacement,
        Mass,
    };
}

struct StepBasicStruct
{
    //Basic
    QString value_description = "";

    StepEnumType::EigensolverType          radio_Eigensolver
StepEnumType::EigensolverType::Lanczos;

    StepEnumType::EigenvaluesType          radio_Eigenvalues
StepEnumType::EigenvaluesType::AllinFrequencyRange;

    QString value_Eigenvalues = " ";

```

```

    bool check_FrequencyShift = false;
    QString value_FrequencyShift = " ";

    bool check_MinFrequency = false;
    QString value_MinFrequency = " ";

    bool check_MaxFrequency = false;
    QString value_MaxFrequency = " ";

    bool check_IncludeAcoustic = false;

    StepEnumType::BlockSizeType          radio_BlockSize          =
StepEnumType::BlockSizeType::Default;
    QString value_BlockSize = " ";

    StepEnumType::BlockSizeType          radio_MaxBlockLanczos    =
StepEnumType::BlockSizeType::Default;
    QString value_MaxBlockLanczos = " ";

    bool check_UseSIMBased = false;
    bool check_IncludeResidual = false;

};

struct StepOtherStruct
{
    StepEnumType::NormalizeType          radio_Normalize          =
StepEnumType::NormalizeType::Displacement;

    bool check_Evaluate = false;

```

```

        double value_Evaluate = 0.0f;

};

class StepData
{
public:
    StepData();
    ~StepData();

private:
    void writeStepBasic(QDomDocument* doc, QDomElement* ele);
    void writeStepOther(QDomDocument* doc, QDomElement* ele);

private:
    StepBasicStruct stepBasic;//分析步基础数据
    StepOtherStruct stepOther;//分析步其他数据
    QString stepName;//分析步名称
    QString stepTypeName;//分析步类型名称

};

#endif // STEPDATA_H

```

AbaqusTree 工程树类开发

AbaqusTree 工程树类是通过自定义对工程树进行编辑，主要是增加新的节点和左右鼠标键功能。AbaqusTree 继承 ProjectTreeWithBasicNode 工程树基类，通过实现基类中的虚函数，达到编辑工程树实现效果。

ProjectTreeWithBasicNode 头文件内容：

```
#ifndef ABAQUESTREE_H
#define ABAQUESTREE_H

#include "ProjectTree/ProjectTreeWithBasicNode.h"
#include "PluginAbaqusAPI.h"

namespace GUI
{
    class MainWindow;
}

class CustomPostWidget;

namespace ProjectTree
{
    enum TreeCustomItemType
    {
        none,
        StepCreate,
        MaterialWidget,
        BCCreate,
        CustomVector,
        PostWindow,
    };

    class PLUGINABAQUSAPI AbaqusTree : public ProjectTreeWithBasicNode
    {
        Q_OBJECT
    }
}
```

```
public:
    AbaqusTree(GUI::MainWindow* m);
    ~AbaqusTree();

protected:

    virtual void initBasicNode(QTreeWidgetItem* root) override;

    //鼠标右键事件
    virtual void contextMenu(QMenu* menu) override;

    //鼠标左键单击事件
    virtual void singleClicked() override;

    //鼠标左键双击事件
    virtual void doubleClicked() override;

    //创建树
    virtual void createTree(QTreeWidgetItem* root, GUI::MainWindow* mainwindow)
override;

    //更新网格子树
    virtual void updateMeshSubTree() override;

    virtual void reTranslate() override;

    virtual void updateTree() override;

private:

    void disableItems();

    CustomPostWidget* _customPostWidget = nullptr;

    QTreeWidgetItem* _customVectortitem{};

private slots:
```

```

        void refresh_slot();

        void close3DPostWindow(Post::PostWindowBase*);

    };
}

#endif // ABAQUESTREE_H

```

通过实现 `initBasicNode` 函数进行对工程树节点追加以及隐藏的功能，在函数中添加了“StepCreate”、“SetMaterial”、“Vector”三个节点。通过 `disableItems` 函数将一些初始的树节点进行隐藏。也可以通过

实现如下：

```

void ProjectTree::AbaqusTree::initBasicNode(QTreeWidgetItem* root)
{
    ProjectTreeWithBasicNode::initBasicNode(root);

    disableItems();

    QTreeWidgetItem* stepCreateWidgetitem =
        new QTreeWidgetItem(_simulationSettingItem, TreeItemType::SelfDefineItem);
    stepCreateWidgetitem->setText(0, tr("StepCreate"));
    stepCreateWidgetitem->setData(0, Qt::UserRole + 2, StepCreate);

    QTreeWidgetItem* setMaterialitem =
        new QTreeWidgetItem(_simulationSettingItem, TreeItemType::SelfDefineItem);
    setMaterialitem->setText(0, tr("SetMaterial"));
    setMaterialitem->setData(0, Qt::UserRole + 2, MaterialWidget);

    _customVectortitem = new QTreeWidgetItem(_root, TreeItemType::SelfDefineItem);
    _customVectortitem->setText(0, tr("Vector"));
    _customVectortitem->setData(0, Qt::UserRole + 2, CustomVector);
}

```

```

}

void ProjectTree::AbaqusTree::disableItems()
{
    QStringList disableList;
    disableList << "Monitor"
        << "Vector"
        << "Boundary Type"
        << "Solver Setting"
        << "Post";

    setDisableItems(disableList);
}

```

通过实现 singleClicked 函数进行对工程树节点追加以及鼠标左键点击功能,通过点击实现“创建分析步”、“设置材料”、“边界条件”窗口等功能。

实现如下：

```

void ProjectTree::AbaqusTree::singleClicked()
{
    this->getCurrentItemData();

    qDebug() << _name << "    single click";

    TreeltemType type = (TreeltemType)_currentItem->type();
    int index = _currentItem->data(0, Qt::UserRole + 2).toInt();

    if (StepCreate == index)
    {
        ModelData::AbaqusModel* model =
static_cast<ModelData::AbaqusModel*>(_data);

```



```

        StepCreateWidget stepCreate(model, _mainWindow);

        stepCreate.exec();
    }

    else if (MaterialWidget == index)
    {
        ModelData::AbaqusModel*          model
static_cast<ModelData::AbaqusModel*>(_data);

        SetMaterialWidget materialWidget(model, _mainWindow);

        materialWidget.exec();
    }

    else if (ProjectBoundaryCondation == type)
    {
        ModelData::AbaqusModel*          model
static_cast<ModelData::AbaqusModel*>(_data);

        BCCreateWidget bCCreate(model, _mainWindow);

        bCCreate.exec();
    }

    else if (PostWindow == index)
    {
        QString strName = _currentItem->text(0);

        if (_customPostWidget == nullptr)
        {
            ModelData::AbaqusModel*          model
static_cast<ModelData::AbaqusModel*>(_data);

            _customPostWidget = new CustomPostWidget(_mainWindow, model,
model->getID());
        }

        _customPostWidget->drawImage(strName);

        emit openPostWindowSig(_customPostWidget);
    }

```

```

        emit showPostWindowInfo(_data->getID(), Post::PostWindowType::D3);
    }
    else
    {
        ProjectTreeWithBasicNode::singleClicked();
    }
}

```

通过实现 contextMenu 函数进行对工程树节点追加以及鼠标右键点击功能,通过右键菜单单击实现“刷新”结果数据功能。当计算完毕后,在算例路径下生成可视化数据,通过“刷新”检索加载数据到工程树“矢量”节点下。

实现如下:

```

void ProjectTree::AbaqusTree::contextMenu(QMenu* menu)
{
    qDebug() << _name << "   Right click";

    QAction* action = nullptr;
    TreeItemType type = (TreeItemType)_currentItem->type();
    int index = _currentItem->data(0, Qt::UserRole + 2).toInt();
    switch (type)
    {
        case ProjectBoundaryCondation:
            break;
        case SelfDefineltem:
            if (index == CustomVector)
            {
                action = menu->addAction(tr("Refresh"));
                connect(action, SIGNAL(triggered()), this, SLOT(refresh_slot()));
            }
    }
}

```

```

        break;

    default:
        ProjectTreeWithBasicNode::contextMenu(menu);
    }
}

void ProjectTree::AbaqusTree::refresh_slot()
{
    ModelData::AbaqusModel* model = static_cast<ModelData::AbaqusModel*>(_data);
    QString strURL = model->getPath() + "/vtk";

    QDir dir(strURL);

    if (!dir.exists())

        return ;

    QStringList nameFilters;

    nameFilters << "*.vtu";

    QStringList listFiles = dir.entryList(nameFilters, QDir::Files | QDir::Readable, QDir::Name);

    int count = listFiles.count();
    _customVectortitem->takeChildren();
    for (int i = 0; i < count; i++)
    {
        QTreeWidgetItem* postWindowitem =

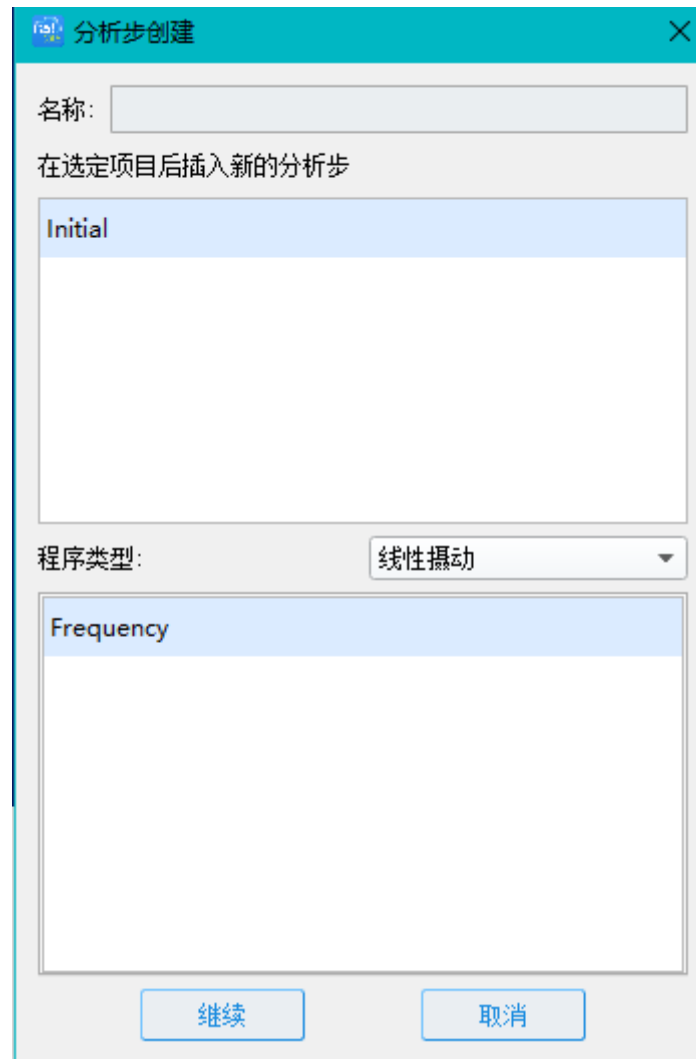
```

```
new QTreeWidgetItem(_customVectortitem,
TreeItemType::ProjectPostCounterChild);
    postWindowitem->setText(0, listFiles.at(i));
    postWindowitem->setData(0, Qt::UserRole + 2, PostWindow);
}

}
```

StepCreateWidget 类开发

分析步创建窗口，用于创建某种类型的分析步骤，在“Initial”后追加新增的分析步骤，当存在多个分析步骤的时候，可以选择在某一分析步后插入新增项。程序类型中可根据分类选择分析的类型，每个分类中存在一个或多个分析类型，根据选择的分析类型不同，分析参数界面也会有所不同。本案例讲解，屏蔽了其余的选项，仅保留流程使用的必要选项。Ui 设计代码见 StepCreateWidget.ui 文件。



分析步创建

StepCreateWidget 头文件内容:

```
#ifndef STEPCREATEWIDGET_H
#define STEPCREATEWIDGET_H

#include "PluginAbaqusAPI.h"
#include "SelfDefObject/QFDialog.h"

namespace Ui {class StepCreateWidget;};

namespace ModelData
{

```

```
class AbaqusModel;
}

namespace GUI
{
    class MainWindow;
}

class QListWidgetItem;

class PLUGINABAQUSAPI StepCreateWidget : public QDialog
{
    Q_OBJECT

public:
    StepCreateWidget(ModelData::AbaqusModel* model, GUI::MainWindow* m);
    ~StepCreateWidget();

protected:
    virtual void showEvent(QShowEvent *)override;

private:
    Ui::StepCreateWidget *ui;
    ModelData::AbaqusModel * _abaqusModel = nullptr;
    GUI::MainWindow* _mainWindow{};

    void init();
    void initStepList();//初始化分析步列表
    void initProcedure();//初始化下拉框列表
```

```

void initGeneral();//初始化通用类型列表

void initLinearPerturbation();//初始化线性摄动列表

void initConnect();//初始化信号槽


QString checkDataValue();

private slots:


void comboBoxTypeChange_slot(const QString&);

void bcListDoubleClick_slot(QListWidgetItem *item);


void on_pushButton_Continue_clicked();

void on_pushButton_Cancel_clicked();

};

#endif // STEPCREATEWIDGET_H

```

其中在 comboBoxTypeChange_slot 函数中根据 UI 中默认显示“线性摄动”分析分类，在“程序类型”列表中显示“线性摄动”分类下的分析类型，在 initLinearPerturbation 函数中，实现了显示“Frequency”分析类型。

具体实现如下:

```

QString StepCreateWidget::checkDataValue()
{
    QString strError;

    QString strStepName = ui->lineEdit_StepName->text().trimmed();

    if (strStepName == "")
    {

```

```
        strError.append(tr("The Step Name is Empty."));
    }
    return strError;
}

void StepCreateWidget::comboBoxTypeChange_slot(const QString& strText)
{
    if (strText.contains(tr("General")))
    {
        ui->tabWidget->setCurrentIndex(0);
    }
    else if (strText.contains(tr("Linear perturbation")))
    {
        ui->tabWidget->setCurrentIndex(1);
    }
}
```

Stepeditwidget 界面类开发

在分析步编辑界面中，因不同分析类型，分析编辑界面参数差异较大，本次案例仅展示“Frequency”类型的参数界面。StepeditwidgetUI 界面显示如下：

StepEditWidget

Name: step-1

Type: Frequency

Basic Other

Description:

Nlgeom: Off

Eigsolver: ☒ Lanczos ☐ Subspace ☐ AMS

Number of eigenvalues requested: ☒ All in frequency range
☐ Value:

☐ Frequency shift (cycles/time)**2:

☐ Minimum frequency of interest(cycles/time):

☐ Maximum frequency of interest(cycles/time):

☐ Include acoustic-structural coupling where applicable

Block size: ☒ Default ☐ Value:

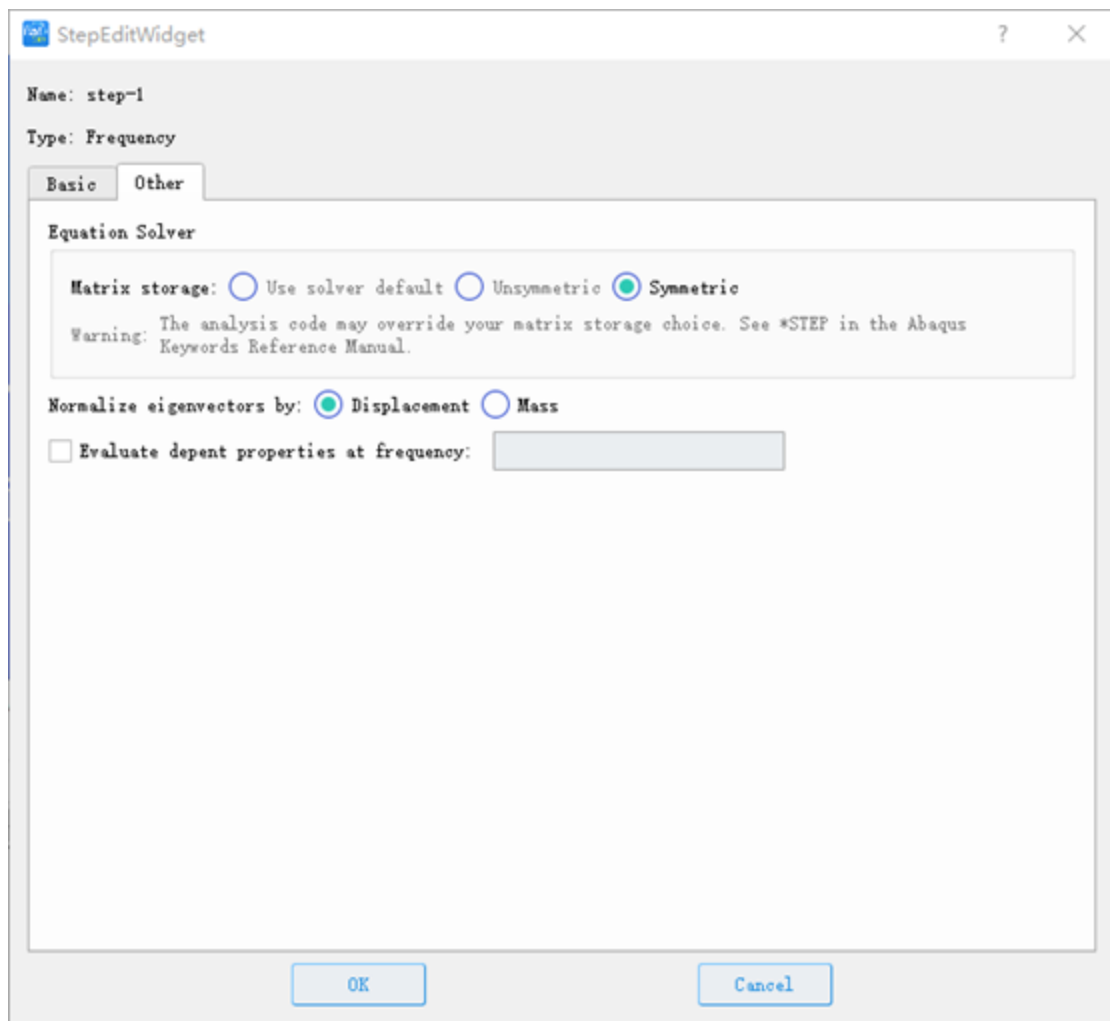
Maximum number of block Lanczos steps: ☒ Default ☐ Value:

☐ Use SDM-based linear dynamics procedures

☐ Include residual modes

OK Cancel

StepEditWidget Basic 界面



StepEditWidget Other 界面

Stepeditwidget 界面类的主要功能是对分析参数的设置，“Frequency”分析类型中包含的参数如上图所示，本次案例使用到的参数有“Description:”和“Number of eigenvalues requested:Value”，其他参数均使用默认值。设定参数完毕后，会将分析步及参数存储到数据类 AbaqusModel 中。

Stepeditwidget 界面类头文件内容：

```
#ifndef STEPEDITWIDGET_H
#define STEPEDITWIDGET_H

#include <QDialog>

namespace Ui {class StepEditWidget;};

namespace ModelData
```

```
{
    class AbaqusModel;
}

class StepData;

class StepEditWidget : public QDialog
{
    Q_OBJECT

public:
    StepEditWidget(ModelData::AbaqusModel* model, QDialog *parent = 0);
    ~StepEditWidget();

    void SetStepNameAndType(QString strName,QString strType,int index);

    void SetStepNameAndType(QString strName, int index);

private:
    Ui::StepEditWidget *ui;
    ModelData::AbaqusModel* _abaqusModel = nullptr;
    StepData* _stepData = nullptr;
    int _StepIndex = 0;

    void initStepNameAndType(QString strName);//初始化分析步骤名称类型
    void initBasic();//初始化基础界面
    void initOther();//初始化其他界面
    void init();

private slots:
```

```

void on_pushButton_OK_clicked();//保存数据

void on_pushButton_Cancel_clicked();//取消

void radioButtonClicked_slot(int);//单选信号槽

void checkBoxClicked_slot();//复选信号槽

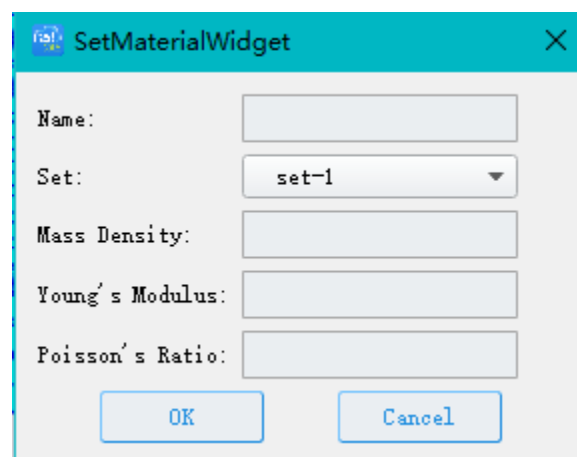
};

#endif // STEPEDITWIDGET_H

```

SetMaterialWidget 界面类开发

SetMaterialWidget 界面类是维护材料类型的界面,本案例中使用到的参数有“Mass Density”、“Young's Modulus”、“Poisson's Ratio”以及材料名称和组件。其作用是给某组件设定材料属性。设定参数完毕后,会将材料参数存储到数据类 AbaqusModel 中。



SetMaterialWidget 界面

SetMaterialWidget 界面类头文件内容:

```

#ifndef SETMATERIALWIDGET_H
#define SETMATERIALWIDGET_H

#include "PluginAbaqusAPI.h"
#include "SelfDefObject/QFDialog.h"

namespace Ui {class SetMaterialWidget;};

```

```
namespace ModelData
{
    class AbaqusModel;
}

namespace GUI
{
    class MainWindow;
}

class PLUGINABAQUSAPI SetMaterialWidget : public QDialog
{
    Q_OBJECT

public:
    SetMaterialWidget(ModelData::AbaqusModel* model, GUI::MainWindow* m);
    ~SetMaterialWidget();

protected:
    virtual void showEvent(QShowEvent *)override;

private:
    Ui::SetMaterialWidget *ui;
    ModelData::AbaqusModel* _abaqusModel = nullptr;
    GUI::MainWindow* _mainWindow{};

    void initList();//初始化下来列表
```

```
QString checkData();

void initData();//初始化数据

private slots:

void on_pushButton_OK_clicked();
void on_pushButton_Cancel_clicked();

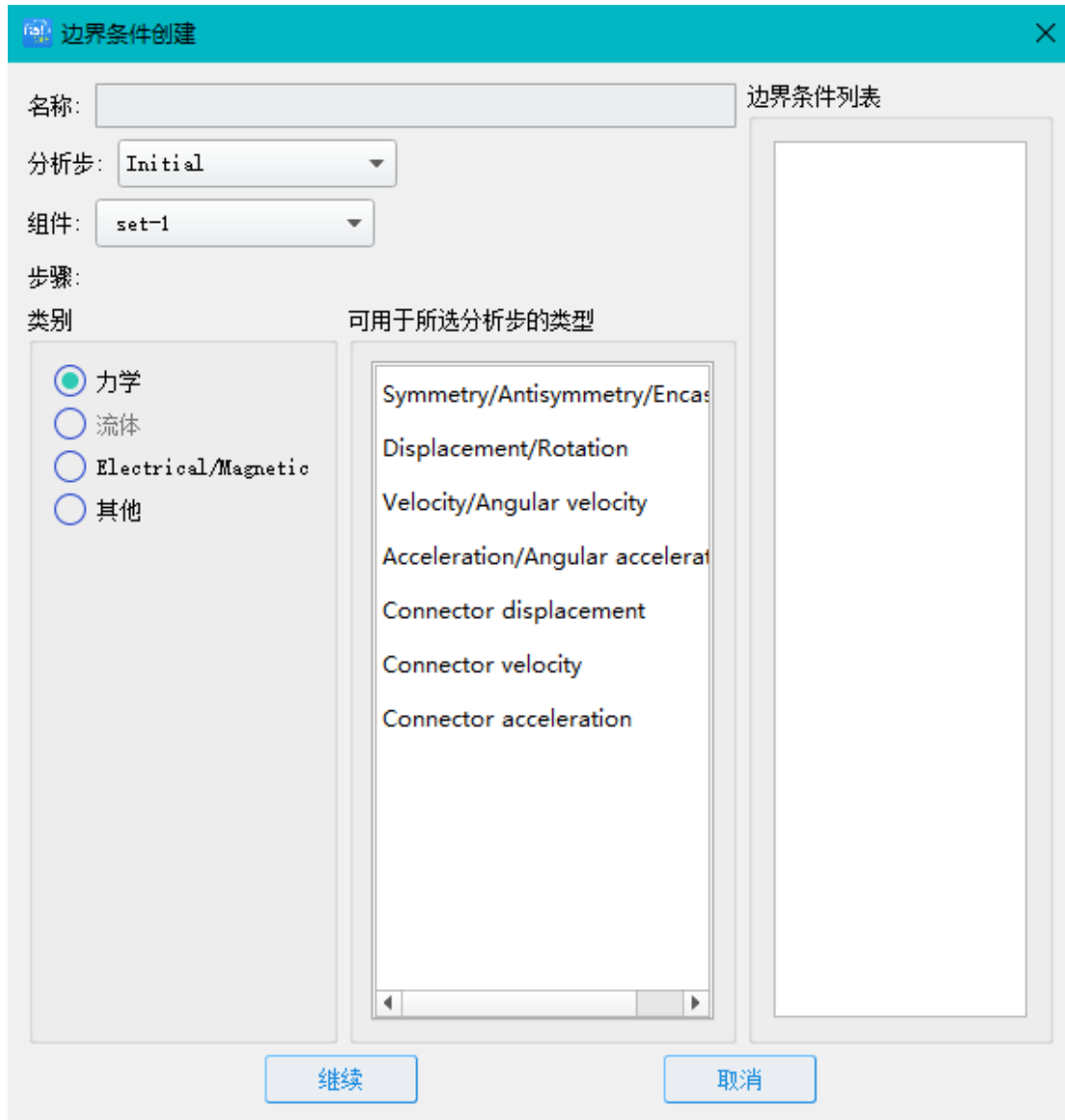
};

#endif // SETMATERIALWIDGET_H
```

BCCreateWidget 界面类开发

BCCreateWidget 界面类是提供选择设定边界条件类型界面类，可以设定在某分析步骤中某组件的边界类型，类型分类有力学、流体等，本案例仅演示案例使用到的力学中用到的两个类型：“Symmetry/Antisymmetry/Encastre”、“Displacement/Rotation”。

用户通过设定“分析步”、选择“组件”以及边界类型为模型设定边界条件，根据边界条件类型的不同，显示参数界面。右侧列表中显示已设定的边界条件。



边界条件界面

BCCreateWidget 界面类头文件内容：

```
#ifndef BCCREATEWIDGET_H
#define BCCREATEWIDGET_H

#include "PluginAbaqusAPI.h"
#include "SelfDefObject/QFDialog.h"

namespace Ui {class BCCreateWidget;}
```

```
namespace ModelData
{
    class AbaqusModel;
}

class QListWidgetItem;

namespace GUI
{
    class MainWindow;
}

class PLUGINABAQUSAPI BCCreateWidget : public QDialog
{
    Q_OBJECT

public:
    BCCreateWidget(ModelData::AbaqusModel* model, GUI::MainWindow* m);
    ~BCCreateWidget();

protected:
    virtual void showEvent(QShowEvent *)override;

private:
    Ui::BCCreateWidget *ui;
    ModelData::AbaqusModel* _abaqusModel = nullptr;
    GUI::MainWindow* _mainWindow{};

    void init();//初始化
}
```



```

void initConnect();//初始化信号槽

void initRadio();//初始化单选按钮

void initMechanical(int stepType);//初始化力学

void initElectrical();//初始化电气

void initOther(int stepType);//初始化其他

void initStep();//初始化分析步

void initSet();//初始化组件

void initList();//初始化列表

QString checkData();//数据校验

void openEditWidget(QString strBCName, QString strStepName, QString strSetName,
QString strBCTypeName);//打开编辑界面

private slots:

    void radioCheck_slot(bool b);

    void bcListDoubleClick_slot(QListWidgetItem *item);

    void on_pushButton_Continue_clicked();

    void on_pushButton_Cancel_clicked();

};

#endif // BCCREATEWIDGET_H

```

BCEditWidget 界面类开发

BCEditWidget 界面类，根据边界类型的不同，显示不同参数界面，编辑完毕后，将边界参数存储到自定义数据类 AbaqusModel 中。

 边界编辑
 ? ×

名称: bc-1

类型: Symmetry/Antisymmetry/Encastre

分析步: Initial

组件: set-1

☒ XSYMM ($U1 = UR2 = UR3 = 0$)
☐ YSYMM ($U2 = UR1 = UR3 = 0$)
☐ ZSYMM ($U3 = UR1 = UR2 = 0$)
☐ XASSYMM ($U2 = U3 = UR1 = 0$; Abaqus/Standard only)
☐ YASSYMM ($U1 = U3 = UR2 = 0$; Abaqus/Standard only)
☐ ZASSYMM ($U1 = U2 = UR3 = 0$; Abaqus/Standard only)
☐ PINNED ($U1 = U2 = U3 = 0$)
☐ ENCASTRE ($U1 = U2 = U3 = UR1 = UR2 = UR3 = 0$)

确定

取消

边界编辑界面

 边界编辑
 ? ×

名称: bc-2

类型: Displacement/Rotation

分析步: Initial

组件: set-2

☐ U1
☐ U2
☐ U3
☐ UR1
☐ UR2
☐ UR3

确定

取消

边界编辑界面

BCEditWidget 界面类头文件内容：

```
#ifndef BCEDITWIDGET_H
#define BCEDITWIDGET_H

#include <QDialog>

namespace Ui {class BCEditWidget;};

namespace ModelData
{
    class AbaqusModel;
}

class QButtonGroup;

class BCEditWidget : public QDialog
{
    Q_OBJECT

public:
    BCEditWidget(ModelData::AbaqusModel* model, QDialog *parent = 0);
    ~BCEditWidget();

    void initDataValue(QString strBCName,QString strStepName,QString
strSetName,QString strBCTypeName);

private:
    Ui::BCEditWidget *ui;

    ModelData::AbaqusModel* _abaqusModel = nullptr;

    QButtonGroup* _group = nullptr;
```

```

        void init();//初始化

        void initRadioGroup();//初始化单元组

        void initModelData();//初始化数据

private slots:

        void on_pushButton_OK_clicked();

        void on_pushButton_Cancel_clicked();

};

#endif // BCEDITWIDGET_H

```

InpWriteManager 类开发

InpWriteManager 类是提供了将自定义数据类 AbaqusModel 中的数据转换为 Abaqus 的 Inp 格式文件功能，目前仅实现本案例所需数据的转换功能，并不能够支持任意数据转换功能。

InpWriteManager 类头文件内容：

```

#ifndef INPWITEMANAGER_H
#define INPWITEMANAGER_H

#include "QFile"
#include "QTextStream"
#include "PluginAbaqusAPI.h"

namespace ModelData
{
    class AbaqusModel;
}

class BCData;

```

```
class PLUGINABAQUSAPI InpWriteManager
{
public:
    InpWriteManager(ModelData::AbaqusModel* model);
    ~InpWriteManager();
    void writeInit(QString fileURL);//根据 Abaqus 格式生成 inp 文件
private:

    ModelData::AbaqusModel* _abaqusModel = nullptr;

    void writeHeading(QTextStream* out);
    void writePart(QTextStream* out);
    void writeNode(QTextStream* out);
    void writeElement(QTextStream* out);
    void writeNset(QTextStream* out, QString strSetName, bool isInstance);
    void writeSection(QTextStream* out);
    void writeAssembly(QTextStream* out);
    void writeMaterial(QTextStream* out);
    void writeStep(QTextStream* out);
    void writeBC(QTextStream* out, QString strStepName);
    void writeOutput(QTextStream* out);

    void writeDisplacement(QTextStream* out,BCData* bcData);
    void writeSymmetry(QTextStream* out, BCData* bcData);

};

#endif // INPWRITEMANAGER_H
```

InpWriteManager 类中 writeNode 函数展示了如何获取网格组件中的网格节点数据并进行输出。

```
void InpWriteManager::writeNode(QTextStream* out)
{
    MeshData::MeshData* md = MeshData::MeshData::getInstance();
    if (md->getKernalCount() == 0)
    {
        return;
    }
    MeshData::MeshKernal* kernal= md->getKernalAt(0);
    if (kernal != nullptr)
    {
        *out << "*Node" << "\r\n";

        int count = kernal->getPointCount();
        for (int i = 0; i < count; i++)
        {
            double* coor = kernal->getPointAt(i);

            int pointIndex = i;

            QString strCoor0 = QString::number(coor[0], 'f', 11);
            QString strCoor1 = QString::number(coor[1], 'f', 11);
            QString strCoor2 = QString::number(coor[2], 'f', 11);

            QString s = QString("%1,%2,%3,%4")
                .arg(pointIndex + 1, 7, 10, QLatin1Char(' '))
                .arg(strCoor0, 17, QLatin1Char(' '))
                .arg(strCoor1, 17, QLatin1Char(' '))
                .arg(strCoor2, 17, QLatin1Char(' '));

            *out << s << "\r\n";
        }
    }
}
```

```
    }  
}  
}
```

InpWriteManager 类中其他函数是按照 Abaqus 格式进行模拟输出生成 Inp 文件。

ODB2VTKConfig 类开发

ODB2VTK+.py 是 python 通过 Abaqus 命令解析 odb 文件将内部数据按照 vtk 格式进行转译,其中主要是读取了网格、节点数据以及位移数据等。ODB2VTKConfig 类是根据算例路径及算例参数生成的配置文件, 其中包含了 odb 文件名称、路径、以及读取的数据类型等命令配置。

头文件内容:

```
#ifndef ODB2VTKCONFIG_H  
#define ODB2VTKCONFIG_H  
  
#include "QFile"  
#include "QTextStream"  
#include "PluginAbaqusAPI.h"  
  
namespace ModelData  
{  
    class AbaqusModel;  
}  
  
class PLUGINABAQUSAPI ODB2VTKConfig  
{  
public:  
    ODB2VTKConfig(ModelData::AbaqusModel* model);  
    ~ODB2VTKConfig();  
  
    void writeConfig(QString fileUrl);  
};
```

```

private:

    ModelData::AbaqusModel* _abacusModel = nullptr;

    void initPath(QTextStream* out);
    void initMesh(QTextStream* out);
    void initPartition(QTextStream* out);
    void initSetting(QTextStream* out);
    void initOutput(QTextStream* out);
    void initMultiprocessing(QTextStream* out);
    void initScale(QTextStream* out);

};

#endif // ODB2VTKCONFIG_H
}

```

CustomPostWidget 类开发

CustomPostWidget 类继承 PostWindowBase 基类, 通过 drawImage 函数将要显示的 vtU 文件路径传递给后处理窗口进行显示。

头文件内容:

```

#ifndef CUSTOMPOSTWIDGET_H
#define CUSTOMPOSTWIDGET_H

#include <QWidget>
#include "PostWidgets/PostWindowBase.h"
#include "PluginAbaqusAPI.h"

```



```
namespace Ui {class CustomPostWidget;};

namespace Post
{
    class Post3DWindow;
}

namespace ModelData
{
    class AbaqusModel;
}

class PLUGINABAQUSAPI CustomPostWidget : public Post::PostWindowBase
{
    Q_OBJECT

public:
    CustomPostWidget
        (GUI::MainWindow* mainwindow, ModelData::AbaqusModel* model, int projectid);
    ~CustomPostWidget();

    void drawImage(QString fileName);

private:
    Ui::CustomPostWidget *ui;
    Post::Post3DWindow *_post3DWindow = nullptr;
    ModelData::AbaqusModel* _model = nullptr;
    int _projectID = 0;

    void initPost3D();
}
```

```
};
```

```
#endif // CUSTOMPOSTWIDGET_H
```

4.2 软件使用

1.我们将开始运行我们开发的程序进行求解工作。鼠标左键点击“控制面板”-“计算分析”-“算例”节点，点击鼠标右键软件将弹出“算例”节点的右键菜单。



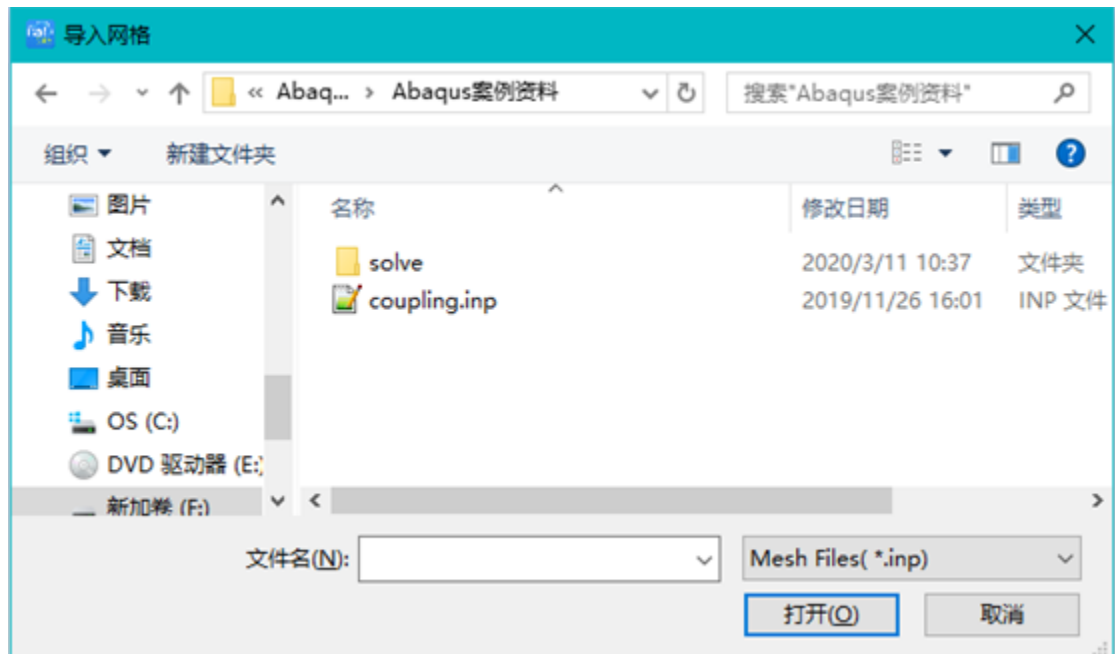
控制面板

2.点击“算例”节点弹出的右键菜单中的“创建算例”菜单项，软件会弹出创建算例对话框。这里我们使用默认的名称，直接点击“确认”按键完成算例的创建。如果有需要用户可以输入其他的算例名称。



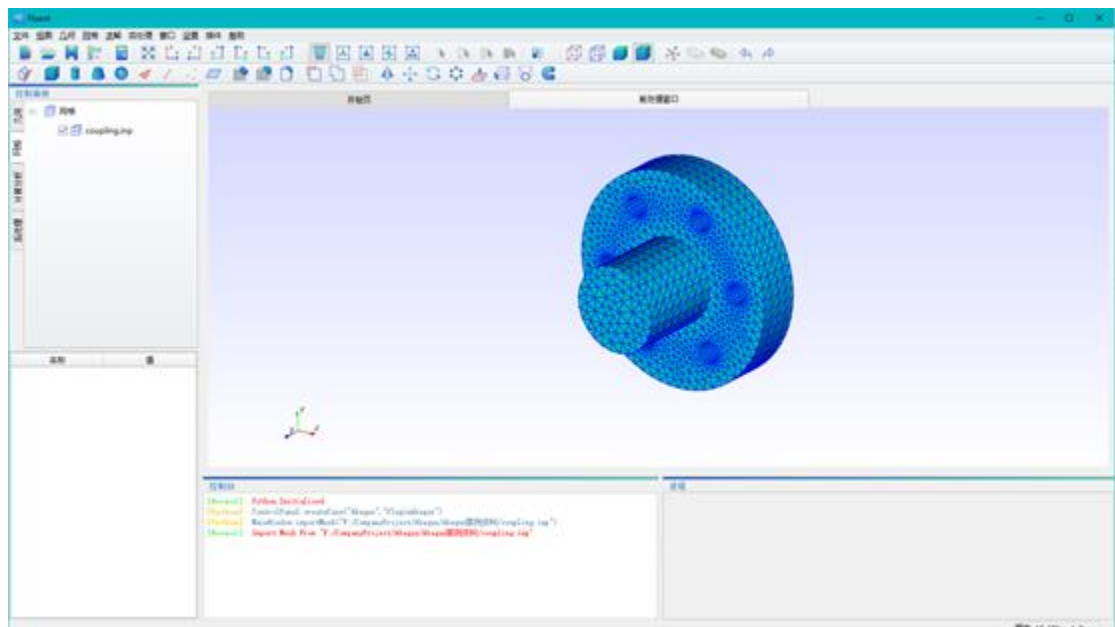
创建算例

3.选择菜单“文件”下的“导入网格”，选择 inp 文件进行导入。



导入网格

4.导入完毕后主界面显示如下图所示。



主界面

5.此时算例下的树形结构如下图所示。点击“算例”下的“仿真参数设置”节点，展开后，可

以看到分析步创建和设置材料节点。



控制面板

6.点击分析步创建，显示分析步创建。名称填写“step-1”,选择分析步“Initial”和选择“线性摄动”下的“Frequency”点击“继续”。

分析步创建

名称: step-1

在选定项目后插入新的分析步

Initial

程序类型: 线性摄动

Frequency

继续

取消

分析步创建

7.点击分析步编辑界面中，在“描述”中填写：Model analysis of a coupling,在请求的特征值个数中选择“数值”并填写：30，其他参数均为默认值，点击“确定”。

分析步编辑

名称: step-1

类型: Frequency

基本信息 其他

描述: Model analysis of a coupling

几何非线性: 关

特征值求解器: ☒ Lanczos ☐ 子空间 ☐ AMS

请求的特征值个数: ☐ 下述频率范围内的所有值

☒ 数值: 30

☐ 频率变化(周期/时间)**2:

☐ 关注的最小频率(周期/时间):

☐ 关注的最大频率(周期/时间):

☐ 包括适用的声学结构耦合

块大小: ☒ 默认 ☐ 数值:

block Lanczos分析步的最大个数: ☒ 默认 ☐ 数值:

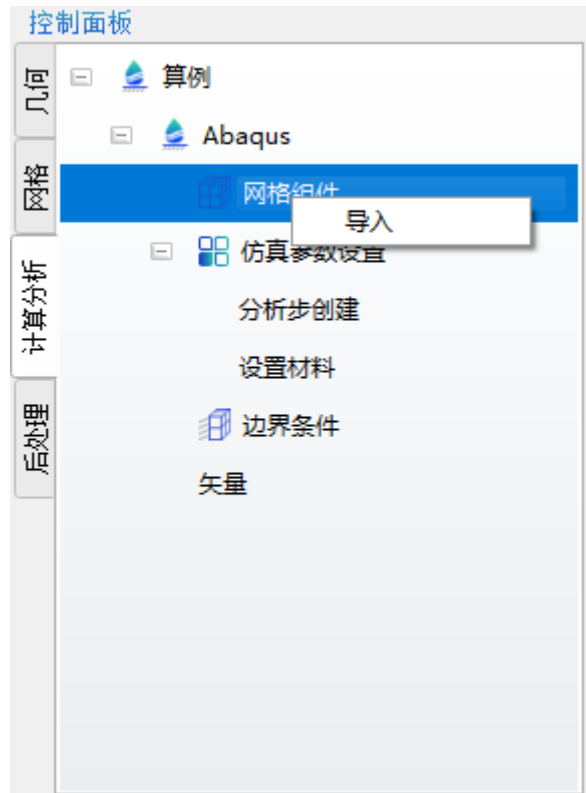
☐ 使用基于SIM的线性动力学步骤

☐ 包括残余模式

确定 取消

分析步编辑

8.在工程树中选择“网格组件”右键点击“导入”，如下图。



控制面板



导入界面

9.将“可选”中的所有组件选中点击“>>”将其移动到“已选”列表中，点击“确定”。



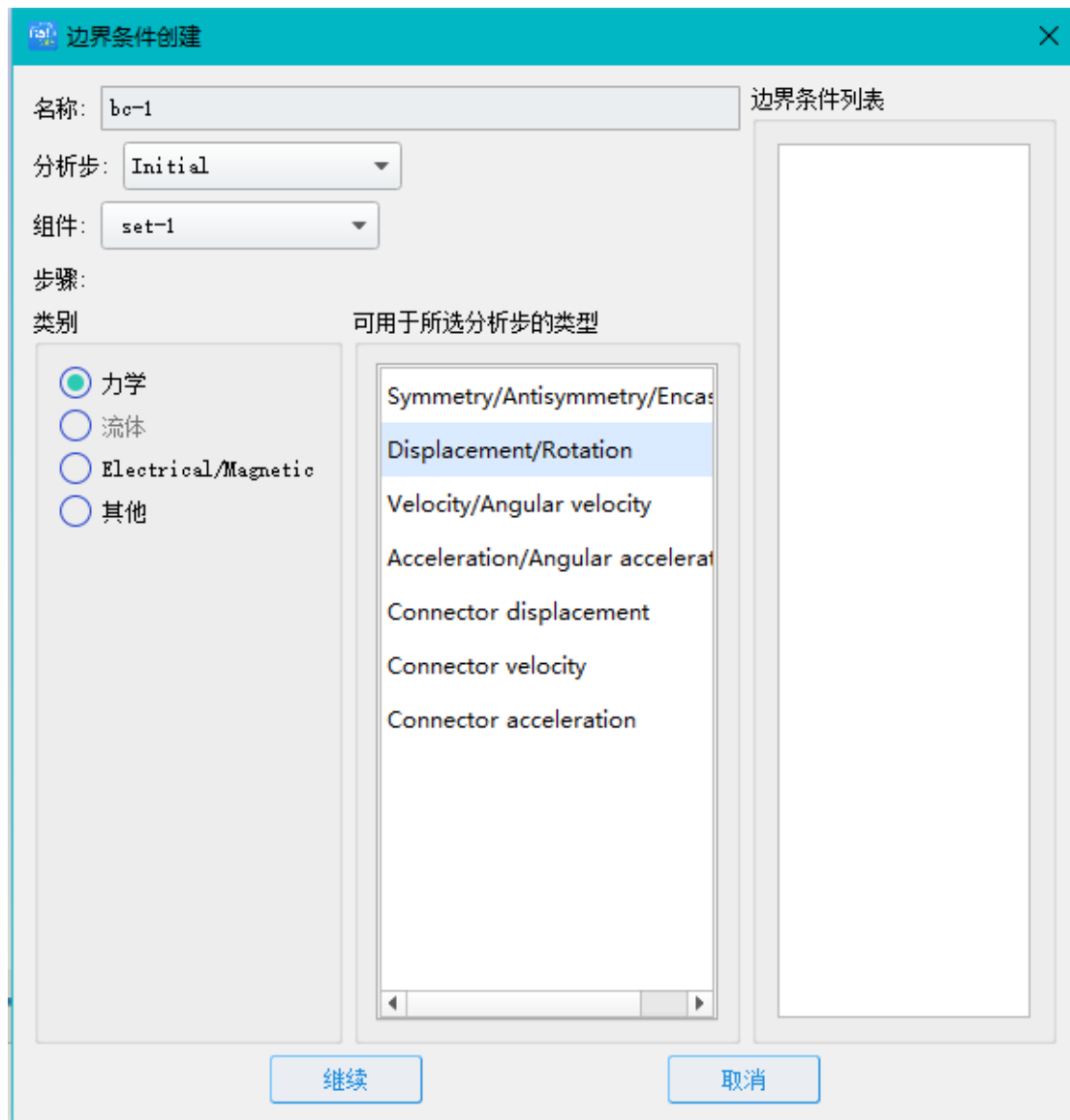
导入界面

10. 点击工程树中“设置材料”，分别设置以下参数数值，如图：



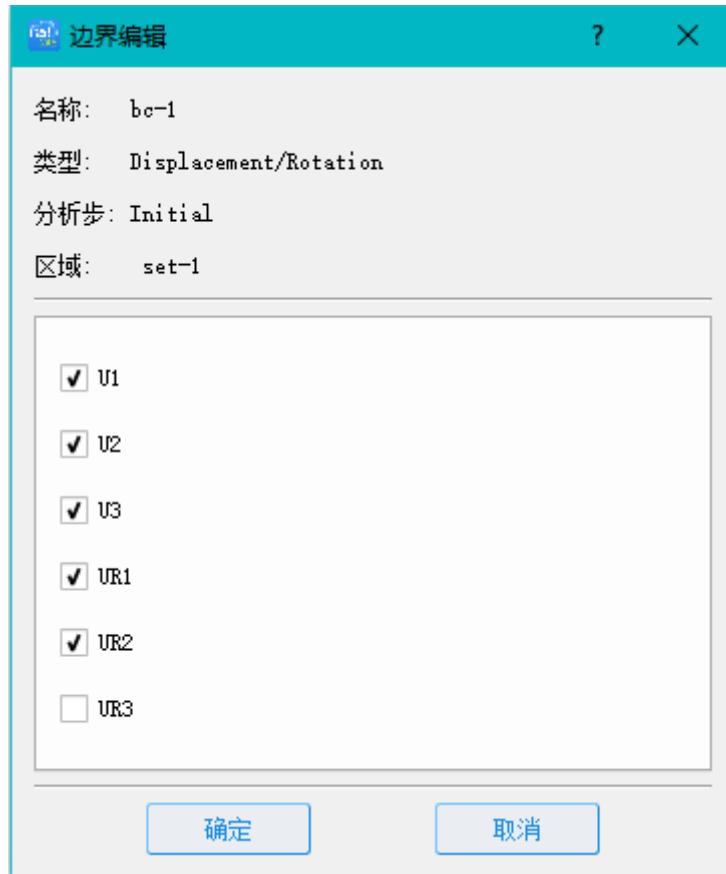
设置材料界面

11. 点击工程树中“边界条件”，打开边界条件创建界面，在界面中填写名称“bc-1”，选择分析步“Initial”，组件选择“set-1”，类别选择“力学”下的“Displacement/Rotation”，点击“继续”。



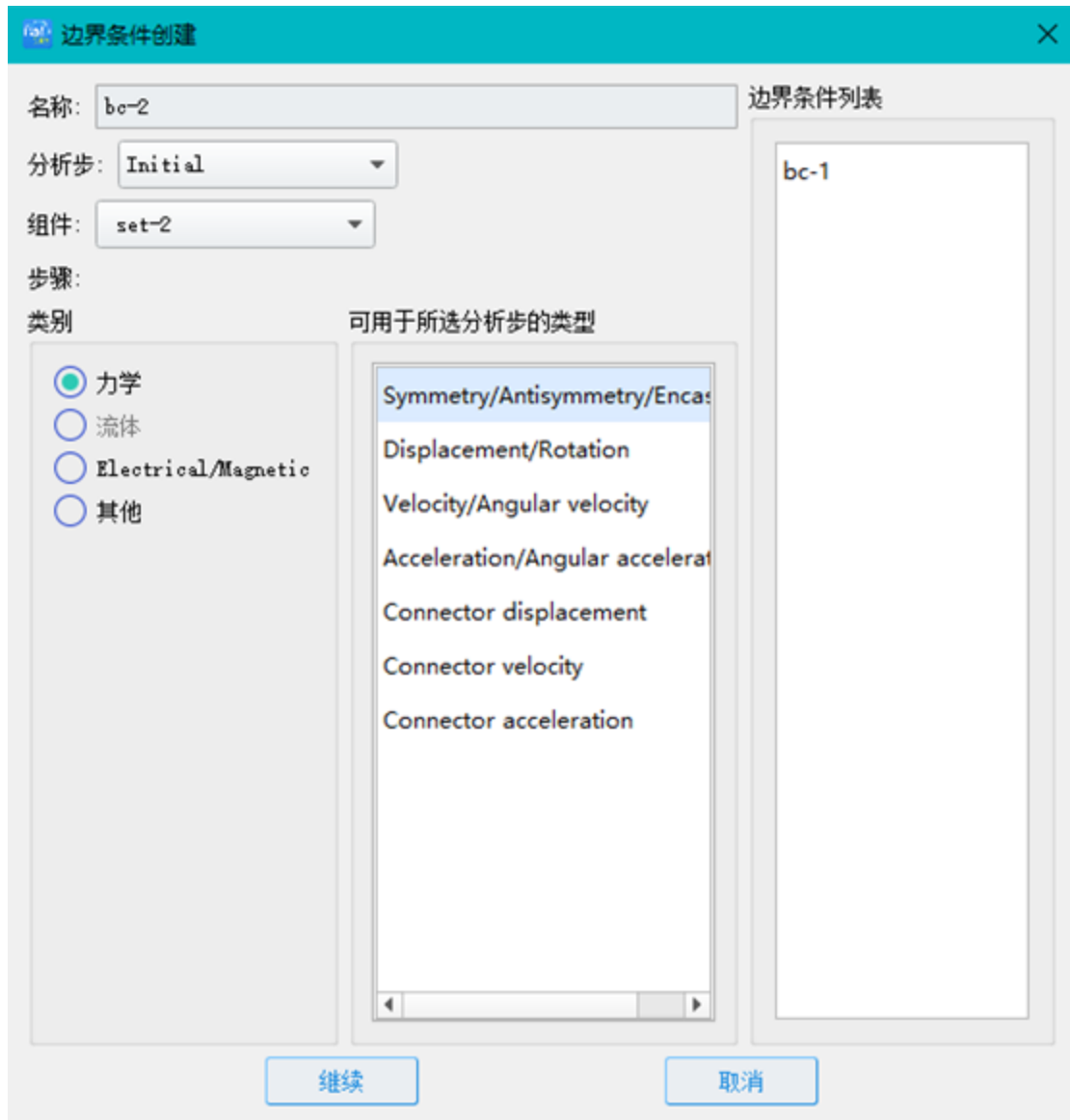
边界条件创建

12.在边界编辑界面中，选中“U1”、“U2”、“U3”、“UR1”、“UR1”，点击“确定”，如下图。



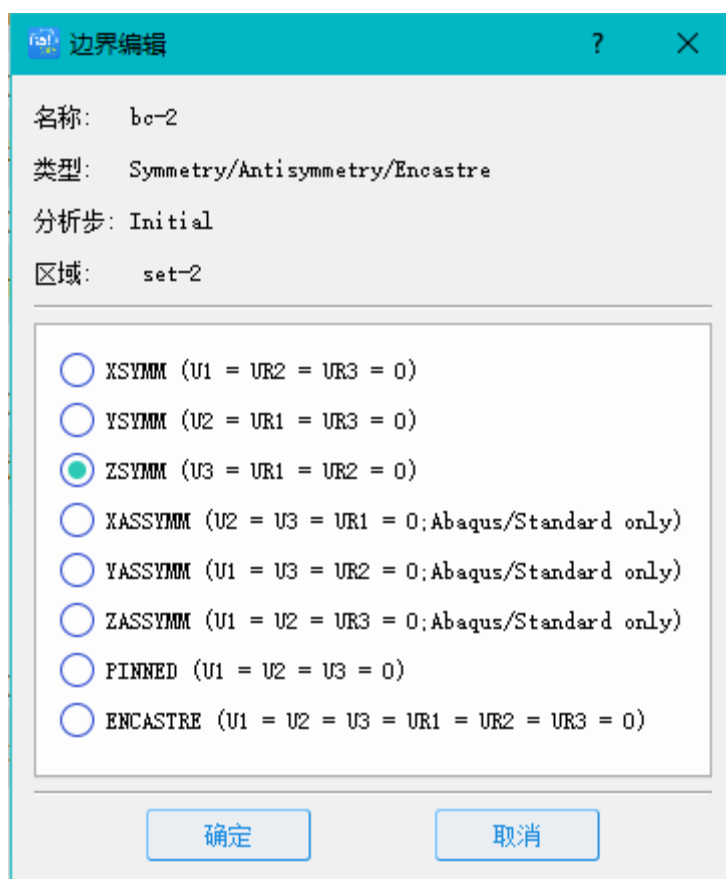
边界编辑

13.同样为组件“set-2”设置边界条件。再次点击工程树中的“边界条件”在边界条件创建界面中，名称填写“bc-2”，分析步选择“Initial”，组件选择“set-2”，类别选择“力学”下的“Symmetry/Antisymmetry/Encastre”，点击“继续”。



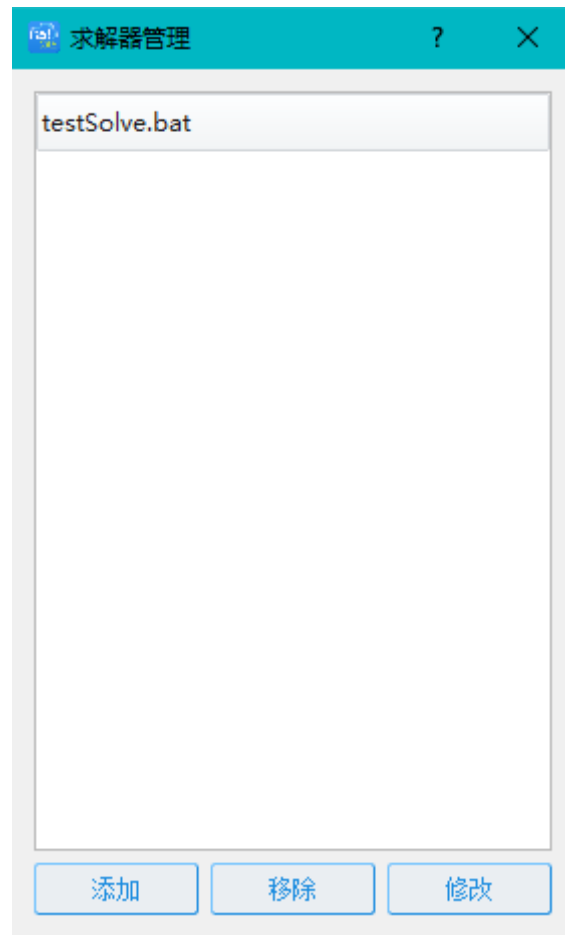
边界条件创建界面

14.在边界编辑界面中，选择“ZSYMM”类型，点击“确定”。



边界编辑界面

15.所有需要设置的计算参数设置完毕后，在菜单中点击“求解”下的“求解器管理”设置求解器参数，点击“添加”。



求解器管理界面



编辑求解器界面

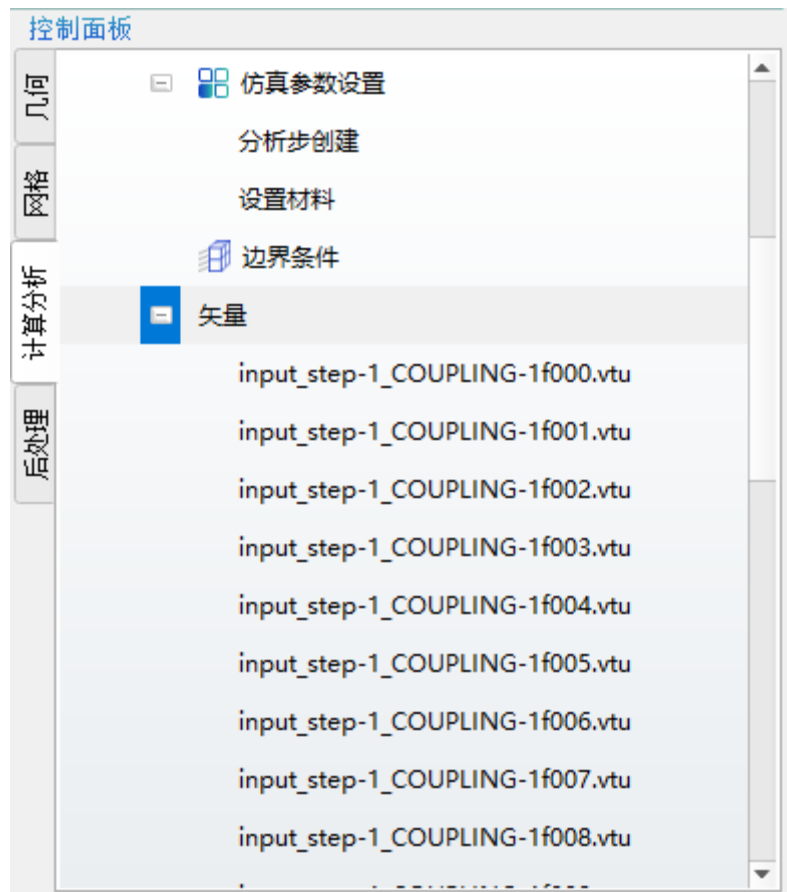
16.在“编辑求解器”界面中，设定“求解器类型”为“自研”，求解器名称为“testSolve.bat”,选择该文件所在的路径,"testSolve.bat"文件内容如下：

```
call abaqus job = input inter  
call abaqus python odb2vtk+.py
```

17.“输入文件”类型选择“文件格式”中的 inp,点击确认及设置完毕求解器参数。

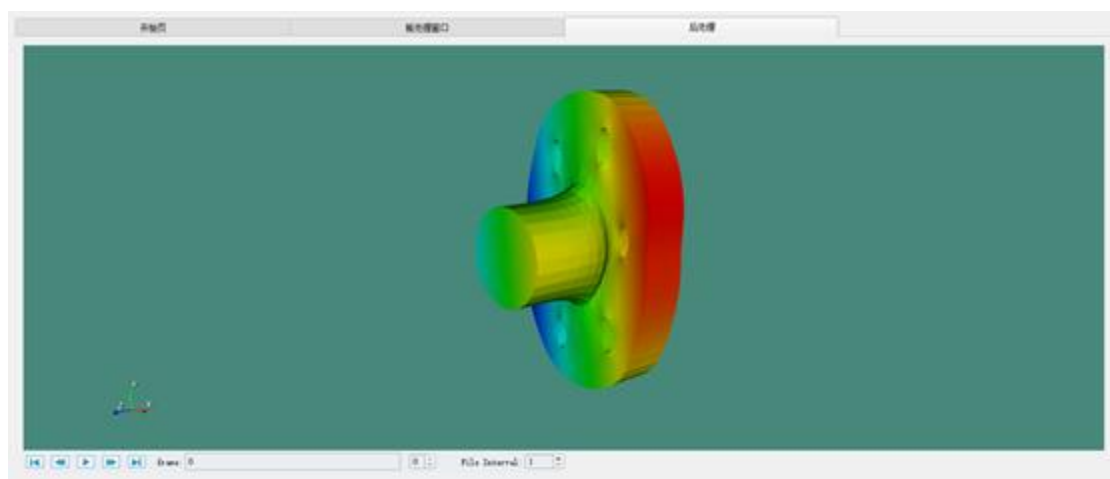
18.点击工具栏的“求解”按键，软件将启动求解。

19.求解结束后，选择工程树“矢量”鼠标右键“刷新”。加载后处理文件，如图。



控制面板界面

20.点击“矢量”节点下 vtu 文件，可查看矢量三维云图。



后处理显示界面